

VECTOR

Naples Beach 2004:

- | | |
|----------------------------------|-----|
| • APL2000 & Dyalog in Florida | 64 |
| • Ajay reviews APLX for Windows | 42 |
| • Hui surveys the Tower of Hanoi | 78 |
| • Devon McCormick on Monty Hall | 85 |
| • Gene takes the Google Test | 116 |
| • Richard Hill uses J from Excel | 133 |



*The Journal of the
British APL Association*

ISSN 0955-1433

www.vector.org.uk

Vol.21 No.1 Autumn 2004

A specialist Group of the British Computer Society

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file or an HTML document. APL code can be accepted in workspaces from I-APL, APL+Win, IBM APL2 or Dyalog APL/W, or in documents from Windows Write (use the APL2741 TrueType font, available free from the website), and MS Word (any version, ideally using the Vector template which is also available from the website).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 2004–2005

Category	Fee	Vectors	Passes
UK Private	£20	1	1
Overseas Private	£22	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	5	5
Overseas Corporate	£110	5	
Sustaining	£500	10	5
Non-voting Member (Student, OAP, unemployed)	£10	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates (excl VAT) are £250 per full page, £125 for half-page or less (there is a £75 surcharge per page if spot colour is required).

Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to Gill Smith, Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Tel: 01439-788385.

Email: apl385@compuserve.com.

Editorial:

by Stephen Taylor

Since our last issue we've lost our most valued reader, Dr Kenneth E. Iverson. Many people have contributed to APL, which is now several dialects; Ken Iverson was father to them all.

The ambiguity is deliberate. Many people have now expressed their gratitude for the gift of these languages, from which we have paying work that remains a recurrent source of pleasure. And many have expressed their gratitude for the gift of Ken Iverson's teaching.

A quarter of a century ago in London he taught a course for APL instructors. I learned from him that I had to take myself out of the learning process: my biggest obstacle to teaching effectively is my own desire to teach, to be the man with the answers. (This must have been the Albertan school of Zen.) When a student asked why 2^*3 gives the wrong answer, I was not to explain the difference between the times and power functions, but ask instead what the answer 'should' have been, and through Ken's famous use of Socratic method lead the student to identify and revise the false premise. Ken's ideal of a teacher was not the man with all the answers, but an irritating and provocative Socrates, one part guide, three parts goad.

I learned to appreciate and eventually to share his fierce, wolfish joy at seeing a student find his own way to a larger understanding. It was a life-changing lesson, which I later took with me into psychotherapy.

Play that pays: Ken Iverson blessed those of us who program in the APLs with paying work that is also study and play. VECTOR 21.2 will celebrate his life and work; this issue is rich in games and puzzles.

Besides our regular crossword puzzle, Brian Schott writes on using J to construct them. Roger Hanoi rescales the venerable Tower of Hanoi with J, and Devon McCormick tackles the 'Monty Hall' problem in APL. Michael Horton translates his fascination with Celtic knotwork into disarmingly simple J verbs. Eugene McDonnell found a puzzle set by Google to catch engineers, and sets about solving it with J. Norman Thomson follows Eugene's recent article on word ladder puzzles with algorithms to find short paths through graphs.

In our technical section Dan Baronet continues his series on development tools, and David Crossley offers more thoughts on 'execute inverse'. He also describes how to launch an APL workspace, and Richard Hill describes how to use J notation in Excel as an alternative to learning a lot of verbose VBA you might prefer not to know.

Go play.

Contents

		Page
Editorial:	Stephen Taylor	2
The WildHeart Column:	Stefano Lanzavecchia	3
APL NEWS		
Quick Reference Diary		6
British APL Association News		
APL Product Guide	Gill Smith	7
The Education Vector		
Zark Newsletter Extracts	edited by Jonathan Barman	22
Crossword and Puzzle		31
J-ottings 41: Shortest Paths and Related Algorithms	Norman Thomson	34
Review: APLX Beta 2.0.1 for Windows	Ajay Askoolum	42
RECENT MEETINGS		
APL2000 and Dyalog at Naples Beach	Adrian Smith	64
APL Germany in Ingolstadt	Adrian Smith	74
GENERAL ARTICLES		
The Tower of Hanoi	Roger Hui	78
Further Surprises Involving a Man and a Goat		
The "Monty Hall" Problem Solved ...	Devon McCormick	85
Celtic Knotwork in J	Michael Horton	90
ACROSS and DOWN	Brian Schott	99
TECHNICAL SECTION		
At Play with J: The Google Test	Gene McDonnell	116
Tools, Part 3: Object Comparison	Dan Baronet	123
Launching a Workspace in Windows	David Crossley	131
Using J in Excel	Richard Hill	133
More on Execute Inverse	David Crossley	140
Index to Advertisers		143

The WildHeart Column

by Stefano Lanzavecchia

"I have been to places that were too cold, never to one that was too hot"

Anonymous

One morning you wake up to discover that the cover on your bed feels thinner and your feet are unmistakably cold. This is a sure sign that summer is over and it's time to hunt for a sweater in the winter closet. And if you live in a country like the one where I live, you will not fail to notice this sad but necessary change, unless you are like a disturbing majority of the Italian population. My compatriots seem to live seasons in a way that I find hard to understand. Let us follow a typical year in Milano from the point of view of the average citizen. When King Winter reigns over the land, he has two possibilities: either dress like the cold climate would suggest with warm and comfortable clothes or follow some silly fashion trend and risk to freeze every day and catch nasty colds to spread around to his or her family and colleagues. In both cases he or she will complain bitterly of the inclement season, dreaming of running away in some exotic country where a tan in mid-February does not mean going to a beauty parlour for an UV treatment. Oscillating between apathy and bad mood, he will try to drink away some of his sorrows waiting for the sweet sun's kisses to return. Fair enough.

As the year progresses there will be no Spring since, as everybody says, seasons are all messed up, and one glorious day Summer will suddenly sweep away the last scarf and once again it will be time for a decision. A few wise souls will remember their miserable past days and will gratefully thank the rising temperature that frees them from the chains of Winter. Alas, the worrying majority of the others will, instead, start cursing the change and hurriedly switch their air-conditioners on, oblivious of the fact that when the sky was grey and cloudy in the cold season, 20 degrees in a room seemed barely enough to survive and required, in addition to a woolly pullover and the comforting emission of a raging hot heater, a good dose of foul vocabulary to let out the frustration of those bound by a cruel destiny. Now for every degree over the 21 threshold, despite the t-shirts, the shorts and the sandals, they grab their remotes for the conditioner and lower the temperature a bit more, to make sure that the room they have to spend all the day in, sweating the brains out, never passes the 18 degrees limit. And the conditioner, enslaved by their will, will vomit rivers of ice-cold air through filters that were probably never cleaned after the first installation. The citizen will praise the ingenuity of science for creating such a useful machine and will only have to

suffer the warm, soft and caring embrace of summer when crossing the street, out of the door of the office and into his modern full-optional car, and out of the car toward his morgue-cold flat.

Luckily enough, not everybody is like that. Recently I met an Italian lady who spent two years living (and working) in Egypt (Sharm-El-Sheik to be precise) and at the end of the stay, in preparation for the move to the new destination, she sold her car. At the question "does the air-conditioning system work" she candidly replied "I don't know". To the eye-popped enquirer she explained that she had never felt the need to switch it on and try it out.

I believe there is a fundamentally wrong assumption in the approach of Italians to hot weather. I agree there are some pathological conditions that prevent a body from being able to stand high temperatures and humidity, and the persons affected should protect themselves using anything science can help them with. But I cannot find a real reason to justify the collective hysteria. I think it's all a matter of perspective. If a child is told that feeling warm is bad, he or she will grow up with a foolish fear of thermometers rising. If he is told that sweating always causes smells and is socially unacceptable, he will develop an insane rejection for any form of moisture on his and his neighbours' body. The truth is different, though. In the case of a normal person, periodic exposure to warm climates bathed in shining sun will help relaxation, improve the mood and enhance self-esteem. Sweating will help the body get rid of various kinds of poisons, balance the content of salts and minerals and, once the toxins are gone, the sweat is almost scentless. Also, in a warm environment where the thermal difference between the inside and the outside of a building is around 5 degrees even if outside the temperature is above 30 degrees, there is very little risk of colds and other various air-conditioning induced diseases, not to mention *legionella* and company, kindly provided by the lethally neglected filters. A friend of mine is convinced that the curse of the Nile, which forces most visitors there to spend too many days of their precious vacations on a toilet, is not caused by the germs in the water but by the freezing cold halls in the hotels. Even I would not go that far, but I find this an interesting interpretation.

Maybe I don't know enough English people to extract meaningful statistics; nevertheless amongst my acquaintances I have noticed some undeniable wisdom in their approach to summer. They seem to enjoy the nice weather when it's there and will even find some form of nostalgic thought in the smell of the melting asphalt of a town like Milano during Summer. Still they would rather be in a place like Lanzarote, officially a Spanish island, but in practice an English (British, Welsh and Irish) colony. And who could blame them? Who would not want to be lazing with an icy pint of lager, under the shade in a transplanted Irish pub on the

promenade of Puerto del Carmen? To be honest, not only English people seem to be able to enjoy warm weather much better than we, who should be used to it, can, but they also seem to be able to stand cold weather without making such a big fuss out of it and I envy them for that. But here I digress.

Weather conditions and seasons have a sure impact on software developers. I, for one, am much more productive during summer. Some of the best code I ever wrote was developed during an amazingly hot August that I had the luck to spend at the office mostly on my own, which meant that I could keep the conditioner switched off all the time. The embrace of the warm air and even the sweat that was refreshing my body made me feel safe and protected and I could concentrate on the work at hand without having to worry about the environment around me. I would like to stress the fact that being sweaty does not necessarily mean being dirty and a shower at the end of the day can be something to look forward to without falling into the obsession of longing for it since the first hour of work at the office. Why was the code I wrote *good*? First of all it was rather elegant, because I had managed to concentrate deep enough to design many aspects of the final result beforehand. Second, it was almost bugless because I had had the time to do thorough testing. Third it was naturally extensible, as was proved in the few months following completion when I was asked to add certain features I had not foreseen. My claim is that all of this was possible because of a proper Summer: unshielded hot weather combined with the long hours of light that allowed me to stay late at the office without having to worry about getting home when it was already dark.

Of course to be able to benefit from the effect of Summer, one must be put in the conditions for this to happen. If short-sighted management or self-inflicted masochism forces one to attend a working day of summer in Milano with suit and tie, I can see that enjoying it would be quite hard. Maybe one day I am going to understand why a software developer strangled by a tie in a shirt with patched armpits should be perceived as a better professional than the same software developer with a clean and fancy t-shirt and a fitting pair of blue-jeans. I am afraid in my case prejudices are hard to overcome so I will not expect a miracle any time soon.

At this point my readers should have an idea about who the anonymous author of the opening quote is, so I will not procrastinate with a useless revelation my wishes to everybody for a great and, hopefully, unusually warm Autumn.

Quick Reference Diary

Date	Venue	Event
March 17-18 2005	Finland	Forest Seminar VIII
November	Naples, Florida	APL 2000 Users' Meeting

Dates for Future Issues of VECTOR

	Vol.21 No.2	Vol.21 No.3	Vol.21 No.4
Copy date	in press	March	May
Ad booking		*	
Ad Copy		*	
Distribution	March 2005	April 2005	June 2004

Vector Back Numbers

Back numbers of Vector are available from:

**British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO62 4JJ**

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Please note that Vol.1 No.2 is now out of stock.

The Vector Product Guide

compiled by Gill Smith

VECTOR's exclusive Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete Systems (Hardware & Software)
- APL and J Interpreters
- APL-based Packages
- Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses
- World Wide Web and FTP Sites

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

*Your listing here is absolutely free, will be updated on request, and is also carried on the Vector web site, with a hotlink to your own site. It is the most complete and most used APL address book in the world.
Please help us keep it up to date!*

All contributions and updates to the Vector Product Guide should be sent to:
Gill Smith, Brook House, Gilling East, York, YO62 4JJ. Tel: 01439-788385,
Email: apl385@compuserve.com

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL Borealis Inc.	Dyalog APL	poa	Distributor of Dyalog APL products from Dyadic
	APL2000	poa	Distributor of APL2000 products
APL Systems IDC SL	APL2000 APL interpreters and tools	poa	Distributor of APL2000 products for the UK Consulting and maintenance for APL applications
Beautiful Systems	Dyalog APL/W for Windows	poa	US Distributor of Dyalog APL products from Dyadic.
	Dyalog APL for Unix	poa	See Dyadic listing for product details.
Dinosoft Oy	Dyalog APL/W for Windows	poa	Finnish distributor of Dyalog APL products.
	Dyalog APL for Unix	poa	See Dyadic's listing for product details.
Dittrich & Partner	APL+Win	poa	Cognos/APL2000 Inc products
	Dyalog APL	poa	Dyadic Systems Ltd. products
	IBM APL products	poa	
Dyalog	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle interface available for IBM, Sun and Xenix versions.
DynArray	DICE for Windows	poa	Software development kit which includes an APL interpreter as a DLL and the ability to run and link existing and new APL code to non APL code such as VB, C/C++, Java and integration with various Windows software applications and database packages such as MS Office.
I-APL Ltd	I-APL/PC or clones	8	ISO conforming interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	8	As above
IBM APL Products	TryAPL2	free	APL2 for educational or demonstration use. Write, fax or Email to APL Products; specify disk size desired.
	Workstation APL2 V2 Version 2	\$1500	AIX, Linux, Solaris, Windows Product 5724-B74, Part Number 45P7514
	APL2 Version 2	poa	Product No. 5688-228, Full APL2 system for S/370 and S/390
	APL2 Application Env't Vn2	poa	Product No. 5688-229. Runtime environment for APL2 packages
Insight Systems	Cognos/APL2000 Inc	poa	Leading distributor of APL2000 products in Denmark
	Dyalog Ltd.	poa	Leading distributor of Dyalog APL products in Denmark
	IBM	poa	Leading distributor of IBM APL & GraphX products in Denmark
J Austria	J	poa	Distributor for Austria and Switzerland
	Dyalog APL	poa	Distributor
	Causeway Products	poa	Distributor
	Structural Analysis Software	poa	Complete package by IG Zenkner&Handel to perform structural analysis/engineering calculations. Also suitable for dynamic problems, e.g. earthquake simulation.
JSoftware Inc.	J on the Web online registration ...		
	J Professional (online reg.)	\$895	Includes manual set and one year of updates
	J Standard (online reg.)	Free	Free for download only

Books and accessories			
	J Dictionary	\$50	
	J Phrases	\$50	
	J Primer	\$50	
	Fractals, Visualization and J	\$80	
	Concrete Math	\$40	
	Exploring Math	\$50	
Lescasse Consulting	APL+PC	poa	Lescasse Consulting is the exclusive APL2000 distributor in France and also distributes in Switzerland and Belgium. Call for price quotes.
	APL+Unix	poa	
	APL+DOS	poa	
	APL+Win	poa	
	Dyalog APL/W	poa	French distributor for Dyalog
MasterWork Software	Manugistics Products and ISI	poa	New Zealand distributor
MicroAPL	APLX for Windows/MacOS	499	Cross-platform APL development environment with GUI programming facilities. Interpreter modelled on APL2. Available for Windows 95/98/ME/NT/2000/XP, Mac OS 9 and Mac OS X.
	APLX Server Edition	poa	For running large multi-user APL applications on x86 Linux, RS/6000 AIX, and Windows NT/2000.
	APLX for Linux (commercial)	499	Linux desktop edition of APLX, compatible with Windows and MacOS versions, with full development environment and GUI programming facilities. Runs under RedHat, Debian, Mandrake and SuSe Linux distributions.
	APLX for Linux (personal)	Free	Full version of APLX, can be downloaded free for personal use.
Oasis	Dyalog APL	poa	Dyalog Ltd
	APL*PLUS	poa	Manugistics
	APL.68000	poa	MicroAPL Ltd
	APL2	poa	IBM
Omega	Zero	poa	A "small simple and fast" alternative to APL
Optima	Dyalog APL/W	985	Dyalog's best, leading edge APL interpreter
RE Time Tracker Oy	APL+PC (APL*PLUS/PC)	poa	Complete APL+ and Statgraphics product range and links to various 3rd party products.
	APL+DOS (APL*PLUS II)		
	APL+Win (APL*PLUS III), APL+Link		
	APL+UNIX		
	APL*PLUS Sharefile		
Soliton Associates	SHARP APL for OS/390	poa	for IBM OS/390 mainframes
	SHARP APL for UNIX	poa	for SunOS and IBM AIX
	SHARP APL for Linux	poa	for Intel Linux
Strand Software	Canada		
	All APL*PLUS products	poa	All APL*PLUS products including upgrades and educational.
	Dyalog and JSoftware products	poa	
	USA		
	Dyalog and JSoftware products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
ADAPTA Software	MPS	poa	Master Production Scheduling
	FBS	poa	Forecasting and Budgeting System
	DRP	poa	Distribution Requirements Planning
Adaptable Systems	FLAIR	poa	Finite loader and interactive rescheduler. Customisable full-function scheduling system. (Available outside Australia by special arrangement only.)
Adaytum	Adaytum e.Planning	poa	Adaytum e.Planning offers a Web-based solution that combines planning, forecasting, budgeting, modelling and reporting in a single, integrated application.
APL Software\Services			
	APL Utilities	poa	Software: mostly .AWS for DOS, utilities for most APL interpreters. Public domain APL*Plus v10 with on-screen documentation and interactive tutorials. APL Conference Software. Books: APL user manuals for STSC, IBM, and Sharp. Request email catalog from dick_holt@email.com.
Beautiful Systems	ASF_FILE	\$399	Dyalog APL/W auxiliary processor for access to APL*PLUS/PC APL component files (*.ASF).
	NAT_FILE	\$299	Dyalog APL/W auxiliary processor which emulates the APL*PLUS/PC quad-N native file subsystem for access to the DOS file system.
	DBF_FILE	\$299	Dyalog APL/W auxiliary processor for efficient block mode access to dBASE format files. Designed to get large amounts of data in and out of dBASE. Not suited for random access to small amounts of data (it does not handle keys).
	SF_READ	poa	Dyalog APL/W functions to read APL*PLUS data objects of any type or structure from *.SF style component files created by APL*PLUS II or III.
Causeway	CausewayPro for Dyalog/W	400	Causeway application development platform for Dyalog APL/W.
	RainPro Business Graphics	250	The ultimate graphics toolkit for the APL developer. Adds 3D charting capability. Web publishing and clipboard support to the shareware product. Charts can be included in <i>NewLeaf</i> reports. Functionally compatible across Dyalog/W and APL+Win.
	NewLeaf for Dyalog and +Win	400	Frame-based reporting tool with comprehensive table-generation and text-flow support. Offers multiple master-page capability, bitmap wrap-around and on-screen preview with pan and zoom. Fully supported on Dyalog/W and APL+Win
Cinerea AB	ORCHART	250	Organization chart package for IBM APL2/PC. Full & heavily commented source code included - free integration into other applications. NB: ASCII output with line-drawing (semi-graphic) characters for boxes.
CODEWORK	3-way TANGRAM	poa	3-way TANGRAM is a DSS-OLAP product, basically a powerful and versatile handler of multi-way tables (also known as hypercubes). It entails 46 analysis modules, including computations, cube merging, sensitivity analysis, time intelligence, free format queries, HTML and LaTeX outputs. Current version is 7.0 At the time of writing, the product is available on Dyalog APL 8.3.1 for Windows 95/98/NT. Future plans include an APL+WIN version and later a LINUX version.

CoSy	K.CoSy	\$30 % yr sub	K.CoSy is a general purpose computing and programming environment constructed, all in open code, in Arthur Whitney's very high level, yet structurally transparent Array Programming Language, K, and its tightly coupled User Interface. K.CoSy is an extremely productive environment in one of the most powerful and fastest of APL's progeny, and therefore, likewise, of all languages. K.CoSy provides a workspace-like interactive development environment previously impossible in K. Because of its unique open construction within the language itself, this environment is clearly competitive in a large domain with the APLs from the other vendors. K.CoSy notepad nature, interactivity, and open K code vocabulary make learning Arthur Whitney's K far less daunting and far more productive than its raw console, or any external scripting method. If you are a client of Kx Systems , or are investigating the possibilities, Contact us. See CoSy/K.CoSy for more information.
DynaArray	DynaWeb Server	poa	A web server providing web based access to applications running on the DICE interpreter from DynaArray, or on an IBM mainframe running APL2.
	DynaHarry	poa	A DSS system which offers the next generation capabilities for current APLDI, IC/E and IC/1 users. It comes with ROLAP capabilities, multisystem access to a wide variety of databases and data warehouses.
	DynaLink	poa	An ODBC client interface for DICE and IBM APL2 programs.
Entropy Software	Personal Information Management		Entropy specialises in Personal Information Management Systems (PIMS) which run on a PC. Most systems are inadequate in some way. This is an attempt to fulfil all the needs of George MacLeod who has been an unhappy user of a variety of systems for many years. The product is being developed in modules (Calendar, Address List, To Do List, etc) which will be available at no charge. When all the modules are complete they will be integrated together. There will be a charge for the integrated product.
HMW	4XTRA	poa	Networked, Windows/Unix based Front End and Middle Office Foreign Exchange and Money Market Dealing System. Scalable from 1 user to 120+.
	Inca	poa	Software Change Management System. Enables the user to co-ordinate development work from several sources, resolve clashes, promote work items for testing and configure releases to a live environment.
	Maya	poa	APL code file manager. A comprehensive suite of tools giving a multi-window IDE style interface to file based APL code. Offers features such as copying from file to file, object comparison, string search, style formatting, hot-spot editor for filed objects (including variables), etc.
	Aztec	poa	System shell for APL development. Manages real-time and batch applications across multiple platforms. Offers standardised error trapping, job scheduling, task communication and recovery/restart features.
	Olmec	poa	APL GUI environment, providing menu bar, tool bar, status area, navigation sidebar (with treeviews & listviews)and client area. All are configured by simple text files and require no programming. Client area has a "tab wizard" option to provide ordered transaction processing
	Nazca	poa	fast, flexible and reliable static database and editor.
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Kits 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
Insight Systems	Causeway	poa	Leading distributor of Causeway products in Denmark

All our old products are now either OEM'd, in the public domain, out of date, or all of the above. We'll be back!

JAD Software	JAD SMS	poa	JAD SMS is a multi-user software management system for Dyalog APL™ based on shared, hierarchical databases. JAD SMS databases let you keep historical versions of apl items as well as attributes such as timestamp, user name and documentation. The software includes a graphical user interface as well as specialized functions for inclusion in applications. No charge for single-user version; \$100/user for multiple users
Lescasse Consulting	APL+Win Monthly Training	\$600	Download 50+ page document about APL+ programming each month. You also get one or more workspaces full of re-usable APL code and sometimes additional files or products.
	Advanced Windows Programming	\$95	200-page book plus companion disk on interfacing APL and Delphi. Contains full coverage of Delphi-2, +Win and Dyalog.
	DLL parser for APL	\$250	Parse any Visual Basic DLL declaration file into a set of quadNA definitions. Turn constants and structures into APL variables. Available for APL+Win and Dyalog/W.
	Delphi Forms Translator	\$195	Design forms with Delphi and turn them automatically into APL programs which recreate the same form (+Win and Dyalog/W).
	APL+Link Pro	poa	ODBC interface for APL+Win
	SQAPL Pro	poa	ODBC interface for Dyalog APL/W
	RainPro	poa	Highly customisable 2D and 3D publication graphics for APL+Win and Dyalog APL/W
	NewLeaf	poa	Page layout and printing tools for APL+Win and Dyalog
	GraphX and ChartFX	poa	High-quality business graphics for APL+Win
	Formula One and Dyalog APL	\$95	100-page book + companion disk on how to use the Formula One VBX with Dyalog APL/W
Lingo Allegro	FACS	poa	EMMA-like interface to DB2 or ODBC databases
	QWIN	poa	Legacy DOS Windowing support for APL+Win
	ODBC/127	poa	IBM AP127-like ODBC Interface for APL+Win and Dyalog APL/W
Optima	ServiceLine	poa	A property management system which keeps a record of all outstanding tasks, produces an up to date list of work to-do and Scheduled reminders plus automated standard letters and basic financial control.
	TravelLine	poa	A system designed to control the workload allocation for a fleet of chauffeur vehicles plus a reminder system for fleet management and Local Authority requirements.
	BPA	poa	"Brand Performance Analysis" allows for the modelling of product/brand performance over time and comparison with competitor products.
	DBI	poa	"Database Interrogator" allows for the non technical user to ask sensible questions of a large database such as a questionnaire and obtain results tables and graphs quickly, easily and accurately.
Qualedi	Qualedi	\$850-\$5,500	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
RE Time Tracker Oy	UIT/W	poa	Comprehensive high-level Windows User Interface library for APL+Win and +H v 5.1. Comprehensive spreadsheets, replicated fields, special field types, etc. 16 and 32 bit versions available.
	AJGRAPH	poa	Graphpak-compatible 2D graphics package for +Win and +DOS. Includes multi-window support, print and metafile support. No DLLs required.
	ECCO PRO with APL	poa	Leading group and personal information management system with comprehensive customising. Supplied with sample +Win workspace to interface to ECCO databases via DDE.
	NEWTT TCP/IP SDK with APL	poa	Lead TCP/IP SDK with interfaces to all protocols. Supplied on 3 CD ROMS together with a sample +Win workspace.

	DB+	poa	Database interface for APL+DOS under Windows. Allows combining character-based APL applications with ODBC-compliant databases such as Oracle and SQL-server.
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
	FAB	free	Training program for the above.
Weighahead Systems	Weighahead Windows Weighing System (3WS)	poa	Recipe Weighing System for Manufacturing Industries. Pharmaceutical, Cosmetics, Foods etc. Works without keyboard or mouse. Uses Electronic Balances, Laser scanners, bar codes and label printers.
Zark	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY AND DEVELOPMENT

COMPANY	PRODUCT	PRICES(£)	DETAILS
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
Andrews	Consultancy	poa	APL programming and analysis, algorithms, tree processing and design programs for craft work.
APL Borealis Inc.	Support and Development	poa	APL Software Support and Development. Specialists since 1979 in Sharp APL, APL*Plus, APL*Win, Dyalog APL
APL Solutions Inc	Consultancy	poa	APL systems design, development, maintenance, documentation, testing and training. Providing APL solutions since 1969.
APL Systems IDC SL	Consultancy	poa	Consulting and maintenance for APL applications.
AUSCAN Software	Consultancy and Training	poa	APL software development, training
Camacho	Consultancy	poa	Manuals; feasibility reports and estimates; analysis and programming; APL and MS Windows applications; Sharp, ISI APL, APL*PLUS, APL2/PC and other APLs spoken. Fixed price systems a speciality
Ian Clark	Consultancy	poa	Interfacing APL, VB, C/C++, ActiveX/COM. Screen design and documentation. National language porting.
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Causeway	Consultancy and Training	poa	On-site training for Causeway, RainPro and NewLeaf. Customisation and enhancement to meet local needs. Code review and pre-implementation check of Causeway applications.
Paul Chapman	Consultancy	250-500	24-hour programmer: APL, Smalltalk, C; Windows front end design a speciality.
CODEWORK	Consultancy	poa	Development, maintenance, migration, documentation of APL applications. Speciality: info systems for top executives, internet applications.
CoSy	Consultancy	poa	CoSy.com, Coherent Systems, provides rapid development in the K language and associated data base products, with a particular interest in quantitative (financial) modelling.
David Crossley	Consultancy	poa	Experienced in large APL system developments since 1969 for PC or mainframe.
Dinosoft Oy	Consultancy	poa	Specialised in very large databases.

Dittrich & Partner	Consultancy	poa	APL programming and analysis; APL workshops and training on the job
Dyalog	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
DynArray	Consultancy	poa	DynArray offers consulting in the areas of DSS, Y2K and APL programs upgrade/conversion to modern Web enabled platforms.
Evestic AB	Consultancy	poa	Excellent track record from 15+ years of APL applications in banking, insurance, and education services. All dialects, platforms and project phases. SQL expertise.
First Derivative Analytics Ltd.	Consultancy	poa	Analysis, design, prototyping, development & testing of APL (especially financial) applications: Sharp, Dyalog APL/W.
General Software	Consultancy	from 200	Over 20 years experience with every version of APL, large mainframe systems and small PC based programmes.
Godin London Inc	Software Development	poa	We have applications in the food manufacturing field, travel agency and airline bookings field and in product lease management.
HMW	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Hoekstra Systems	Consultancy	poa	APL consultancy, programming, etc. Also UNIX system administration
Michael Hughes	Consultancy	poa	APL consultant with 20 years experience with all versions of APL. I can create your dynamic Web sites using the full power of APL working with Microsoft IIS (Internet Information Service) on Windows NT or 2000. I also undertake System design, Programming and Maintenance on all platforms, particularly MS Windows.
INFOSTROY	Consultancy	poa, competitive	Broad experience in various APL platforms. Special skills and knowledge in developing complex applications for investment, financial and construction markets. Implementation of hybrid solutions based on APL, Delphi, C++, VBA, SQL servers.
Insight Systems	Consultancy	poa	We have experience with just about every APL system and platform in common use during the last 20 years, from SHARP APL under MVS or Linux to APL+Win and in particular Dyalog APL under Windows 9x, NT or 2000. If you have decisions to take about adapting your APL application to take advantage of emerging technologies, or would like your strategy reviewed, give us a call. We have extensive experience in all areas of APL development, from legacy systems, up, down and sideways migrations, to the development and support of shrink wrapped solutions based on APL. Even if we don't have time to do the work ourselves, we will know where to find someone who is an expert in your version of APL and your application area, on your continent.
JAD Software	Consultancy	poa	Systems design and development, project management, technical manuals, financial and actuarial expertise in APL.
KJK	Consultancy and software development	poa	APL-based data management: conversions, ad hoc-analyzing tools, well-interfaced methods for defining, processing and browsing of multi-dimensional reports. Rapid custom software development based on proven modular toolset approach.
Lambent Technolgy	Consultancy	poa	APL programming, consulting & training; web design and construction.
Phil Last	Consultancy	poa	APL consultancy, modelling and programming.
Lescasse Consulting	Consultancy	poa	A range of consultants, experts in Windows programming, with APL+Win and Dyalog APL/W. More than 100 major APL applications already developed. We all have additional expertise in Formula One and Delphi.
Lingo Allegro	Consultancy	poa	General APL consulting, Internet website development, migration and downsizing, performance tuning, education and training.
Lucas Solutions	Consultancy	poa	Rates depend on task and location.

Mackay Kinloch Ltd	Consultancy	poa	Design, analysis and programming for banking, insurance and pensions, financial planning and modelling, corporate performance and legal reporting
MasterWork Software	Consultancy	poa	Consulting and J programming for econometrics and statistics in public policy, health and food industries.
Milinta Inc	Consultancy	poa	Design, development, maintenance, conversion, documentation in all APLs, most APs and some specific Sharp products (LOGOS, ViewPoint, Retrieve). Experience in multi-user, multi-task systems, databases, Windows programming.
Ellis Morgan	Consultancy	poa	Business Forecasting & APL Systems.
Oasis	Consultancy	poa	Expertise in APL system design, Project management, conversion, migration, tuning, for all APL versions (10+ years experience)
Omega Computing	Consultancy	poa	APL consultancy, programming, etc.
Optima	Consultancy, Training & Development	poa	We have been in business since 1990 and have a team of APL professionals with many years experience in pharmaceutical, industrial and financial systems on both PC and Mainframe platforms. In addition to pure APL we can also offer assistance with network integration, ACT! customisation and issues concerning the internet and/or web design.
RadSys Technologies	Consultancy	poa	Areas of expertise: financial systems, risk analysis systems, healthcare systems.
Resources & Results	Consultancy	poa	Knowledge management company builds decision support, data warehouse, data mining and strategic planning systems for CFO's, CEO's, and senior management, using our Rapid Application Development (RAD) methods and tools. Extensive experience in large-scale system development and ad hoc executive support for Fortune 500 clients.
RE Time Tracker Oy	Consultancy	poa	APL application conversions, APL Windows interfaces, APL to API-level interfacing to any system under Windows, TCP/IP network and database connectivity.
Graeme Robertson	Consultancy & Training	250-500	Design and write custom software. Maintain and upgrade APL systems. Deliver customized APL training courses
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Shepp & Associates	Consultancy	poa	APL applications development and consulting, especially in the travel industry, especially on small computers. 25 years experience in APL programming.
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL parallel compiler APEX is giving very good initial performance tests with convolution somewhat faster than FORTRAN.
SovAPL	Consultancy	poa	Offshore APL development service.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface Implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.
Rex Swain	Consultancy	poa	Independent consultant, 25 years experience. Custom software development, PC and/or mainframe.
Sykes Systems Inc	Consultancy	poa	Complete APL services specialising in audit, optimisation and conversion of APL systems. Excellent design skills. All dialects and platforms. 17-23 years experience.
		MACfns	MACfns has been released. A suite of over 100 high-accuracy, high-speed assembler code utility functions for APL+Win and APL+DOS. MACfns has been thoughtfully designed and implemented for APL programmers. It includes functions for segmented string manipulation, data restructuring, translation and datatype adjustment, and many numerical computations. Typical speedups range from 4-10 times, with reduced workspace usage, datatype conservation, identity detection, and greater numerical

Weighahead Systems Consultancy	poa	precision amongst the other benefits. Documentation of MACfns is extremely detailed and thorough.
Stephen Wynn Consultancy	poa	Specialising in industrial systems. Links to PLCs, laser scanners, bar codes, weigh scales, label printers etc. Also programmable hand held scanners.
	poa	Most experience of financial planning, and mathematical areas: operational research, quality control, experimental design.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL-385	Typefaces	poa	Variants of the APL2741 typeface available to specification.
APL Borealis Inc.	APL Training	poa	Hands-on courses in Introductory, Intermediate, Advanced and Windows APL. Courses are customized and flexible, and may be delivered on-site, with strong emphasis on methods for efficient and maintainable APL systems development.
Atomic Vector	RELCELL	poa	Details: Report Writer, Data Mining, Database Visualization, and Export to Excel from relational databases. Offered in English, Spanish, Italian, Portuguese, French and German. With this product, you don't have to know English in order to use SQL. Point and click with easy to use Wizards.
	APL Games Volume 1	\$15	Details: Volume 1 - A collection of four APL games on cd, saved from past yearly issues of the APL Planet. Great way to demonstrate the capabilities of APL, and to entice the younger generations into APL. Written entirely in Dyalog APL.
ComLog	Comic-Logger	\$25.95+p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.
HMW	Employment	poa	Contractors and permanent employees placed.
I-APL Ltd	Books	poa	I-APL stocks books written to go with the I-APL interpreter and some APL Press books. For a list write to 11 Auburn Road, Bristol BS6 6LS, ring 0117 973 0036 or email acam@blueyonder.co.uk.
Oasis	Training	poa	Introductory courses in APL. Advanced courses for different APL versions
Optima	IBM Compatible	poa	Complete business solutions including hardware, software and support plus additional network installation and maintenance if required.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Right Seat Software	Vox Proxy	\$199 (comm)	Vox Proxy is authorware for PowerPoint(r) 2000 or 2002 which allows the use of Microsoft Agent Technology (3D talking animated characters) within slide shows. VP appears on PowerPoint's main menu and provides editing slide-by-slide with slides. Automated script-writing provides control of PowerPoint, allowing the use of characters for live presentations or fully-automated tutorials, demos, or training programs. Optional CD Prep program allows the user to create auto-starting CD's that will play on any version of PowerPoint or without PowerPoint.
		\$69.95 (edu)	

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM SigAPL	International	APL QuoteQuad	Conferences; APL white pages; web site	\$30
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$20
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Germany e.V.	Germany	APL Journal	Semi-annual meetings	DM60

Ass. Francophone pour la promotion d'APL France		Les Nouvelles d'APL		FF350 (private) FF2800 (Company)
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
Danish SIG	Denmark			
Dutch APL Assoc.	Holland	Vector provided	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
Japan APL Assoc	Tokyo	APL Journal	Monthly meetings (4th Sat)	10,000yen to join Material fee 5,000yen a year
NY SigAPL	New York, USA	Big Apple APL	Monthly meetings	\$35/\$25(ACM)
Rome/Italy SIG	Roma, Italy			
SE APL Users Grp	Atlanta, Georgia	The APL Planet	Semi Annual meetings	\$13 (US), \$25 (non-US)
SovAPL	Moscow, Russia	-	Seminars and Annual Meeting	
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG) Bern		Part of Qtly SH-Info		SF60 (SI) + SF20 (SAUG)
Toronto SIG	Toronto, Canada		Occasional meetings, APL Skills Database, Toronto Toolkit	

ADDRESSES

ORGANISATION	CONTACT	ADDRESS, TELEPHONE, FAX, EMAIL etc.
ACM SigAPL	David Siegel	ACM, 1515 Broadway, 17th Floor, New York, NY 10036, USA (Subs only)
ADAPTA Software GmbH	Michael Baas	Stellinger Weg 19, 20255 Hamburg, Germany. Tel: +49 40 40170951 Fax: +49 40 40170952. Email: info@adapta.de
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 9589 5578 Fax: +61 3 9589 3220 Email: hillrj@meibpc.org.au
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342 Email: adfee@concepts.nl
Andrews	Dr Anne D Wilson	12 Thorny Hills, Kendal, Cumbria LA9 7AL, UK. Tel: 01539-731205 Email: ADWilson@kencomp.net
APL-385	Adrian Smith	Brook House, Gilling East, York YO62 4JJ, UK. Tel: 01439-788385 Email: apl385@compuserve.com
APL2000 (Europe)	Fred Honea	see APL Systems IDC SL.
APL Bay Area APLBUG	Curtis Jones (Sec)	228 South 15th Street, San Jose, CA 95112-2150, USA Tel: +1 (408) 292-4060 Email: jonesca@vnet.ibm.com
APL Borealis Inc.	Richard Procter	381 Manor Road East, Toronto, Ontario M4S 1S7, Canada. Tel: (416) 457-7828. Fax: (416) 482-6582 Email: info@aplborealis.com
APL Club Austria	Harald F. Nelson	c/o N-TECH, Siebenbrunnengeldg. 4-6, A-1050 Wien, Austria. Tel: +43 1 5458063 Fax: +43 1 5458063-17
APL Germany e.V.	Dieter Lattermann	Rheinstraße 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469. Email: Dieter_Lattermann@t-online.de
APL Software/Services	Dick Holt	31570 Heather Field Lane, Earlysville VA 22936 USA Tel: +1 (434) 973-4255; Fax: +1 (703) 528-7617; Email: dick_holt@email.com
APL Solutions Inc	Eric Landau	1107 Dale Drive, Silver Spring, MD 20910-1607 USA Tel: +1 (301) 589-4621 Fax: +1 (301) 589-4618 Email: apisi@starpower.net
APL Systems IDC SL	Fred Honea	Alfredo Marquerie, 12 - 2 F, 28034 Madrid, Spain. Tel: +34 91 730 7008 (Office) +34 60 680 5949 (Mobile). Fax: +1 775 743 6131. Email: uksales@apl2000.net
Association Francophone pour la promotion d'APL	Ludmila Lemagnen	174 Boulevard de Charonne, F-75020 Paris, FRANCE Email: lemagnen@aol.com
AtomicVector Productions Inc.	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA. Tel: +1 (770) 972-3755 Email: E-mail: mailto:seapdoc@aol.com

AUSCAN Software Ltd	Richard Procter	PO Box 39, Mansfield, Ontario L0N 1M0 Canada Tel: +1-705-434-1239 Email: rjp@ca.inter.net
BACUS	Joseph De Kerf	Rocinberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 5, Jenkintown, PA 19046, USA Tel: +1 (215) 886-2636; Fax: +1 (215) 886-4888 Email: BeautifulSystems@goffs.net
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. email: acam@blueyonder.co.uk
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS, UK. Tel: 01252-874697 Email: ray_cannon@compuserve.com
Causeway Graphical Systems Ltd	Adrian Smith	The Maltings, Castlegate, MALTON, North Yorks YO17 7DP, UK. Tel: 01653-696760 Fax: 01653-697719 Email: adrian@causeway.co.uk
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB, UK. Tel: 020 7404 5401, Compuserve: 100343.3210
Cinerea AB	Rolf Komemark	Box 61, S-193 00 Sigtuna, Sweden. Tel/Fax: +46 859 255 421 Email: rolf@cinerea.se
Ian Clark	Ian Clark	1205 Deer Creek Drive, Plainsboro, NJ 08536, USA. Tel: +1 609 716 5832 Email: earthspot2000@hotmail.com
CODEWORK	Mauro Guazzo	Corso Lanza 105, 10133 Torino, Italy. Tel: +39 11 6601560 Fax: +39 11 6601560 Email: codework@codework- it.com
CornLog Software	Jeff Pedneau	18728 Bloomfield Road, Olney, MD 20832 USA Tel: +1 (301) 260-1435 Email: jeff@softmed.com
CoSy.com	Bob Armstrong	42 Peck Slip #4B, New York, NY 10038-1725, USA. Tel: +1 212-285-1864 Fax: +1 212-285-1864. E-mail: bob@CoSy.com.
David Crossley	David Crossley	187 Le Tour du Pont, 84210 ST DIDIER, France. Tel: +33.4.90.66.08.87 Email: crossley@au-village.com
Danish User Group	Helene Boesen	c/o Insight Systems ApS, Nordre Strandvej 119G, Hellebæk, Denmark
Dinosoft Oy	Pertti Kalliojärvi	Lönnrotinkatu 21C, 00120 Helsinki, FINLAND. Tel: +358 9 70028820 Fax: +358 9 70028824 Email: dinosoft@dinosoft.fi
Dittrich & Partner Consulting GmbH	Axel Holzmüller	Kieler Strasse 17, D-42697 Solingen, Germany. Tel: +49 212-260 660 Fax: +49 212-260 6666; Email: info@dpc.de
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 347 342 337 Fax: +31 347 342 342
Dyalog Ltd.	Peter Donnelly	Grove House, Lutylens Close, Chineham Court, Basingstoke, Hampshire RG24 8AG, UK. Tel: 01256-338461 Fax: 01256-316559 Email: sales@dyalog.com; support@dyalog.com
DynArray Corporation	Dr James Brown	16360 Monterey Rd. Suite 260, Morgan Hill, CA 95037, USA Tel: +1 (408)-782-6648 Fax: +1 (408)-782-6627 Email: info@DynArray.com
Entropy Software Limited	George MacLeod	Calle Viena 13, Residencia Larrosa, Guardamar del Segura, 03140 Alicante, Spain. Tel: +34 61 00 41 940 Email: george.macleod2003@yahoo.co.uk
Evestic AB	Olle Evero	Bertellusvagen 12A, S-146 38 Tullinge, Sweden Tel & Fax: +46 778 4410 Email: olle.evero@mail.com
FinnAPL	Olli Paavola	Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland Email: olli.paavola@pyr.fi
First Derivative Analytics Ltd.	Ken Chakahwata	114 Lemsford Lane, Welwyn Garden City, Herts AL8 6YP, UK Tel/Fax: 01707-339620. Email: KenChakahwata@compuserve.com
General Software Ltd	M.E. Martin	Little Wester House, Westerhill Road, LINTON, Kent ME17 4BS Tel: 01622 832463 E-mail: martin@gssoft.plus.com
Godin London Incorporated	Gaëtan Godin	12 Gerrard St., London, Ontario, Canada N6C 4C5 Tel: +1 (519) 679-8290 Fax: +1 (519) 438-6381 Email: Info@godin.on.ca
HMW Computing	Chris Hogan	Hamilton House, 1 Temple Avenue, London EC4Y 0HA, UK. Tel: 0870-1010-469; Email: HMW@4xtra.com
Hoekstra Systems Ltd	Bob Hoekstra	Dominique, Salisbury Road, Woking, Surrey, GU22 7UR, UK. Tel: 01483-771028. Fax: 01483-837324 Email: Bob.Hoekstra@HoekstraSystems.ltd.uk
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Leics. LE16 8QL, UK. Tel: 01536-770998 Email: Michael@Hughes.uk.com
I-APL Ltd	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS, UK. Tel: 0117-973 0036. Email: acam@blueyonder.co.uk

IBM APL Products	Nancy Wheeler	APL Products, IBM Silicon Valley Lab, Dept H36/F40, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 [+1 (408) 463-2752] Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com
INFOSTROY	Alexey Miroshnikov	3 S. Tulen Lane, St. Petersburg 191186 Russia. Tel: +7 812 325-9797 Fax: +7 812 311-2184 Email: aim@infostroy.ru
Insight Systems ApS	Helene Boesen	Nordre Strandvej 119G, DK-3150 Hellebæk, Denmark Tel: +45 70 26 13 26 Fax: +45 70 26 13 25 Email: info@insight.dk
JAD Software	David Crossley	175 East 98th St., Apt. 17G, New York, NY 10128 Country: USA Tel: +1 (212) 369-6713 Fax: +1 (212) 761-0124 Email: jadsms@usa.net
Japan APL Assoc	Juichiro Takeuchi	Keio Univ. Faculty of Sci.&Tech. 3-14-1 Hiyoshi, Kohoku-ku Yokohama Kanagawa, Japan 223-8522. Tel: +81(045)563-1411 Fax: +81(045)566-1617 E-mail: takeuchi@ae.keio.ac.jp
J Austria	Joachim Hoffmann	Hohenitz Nord 104, 8182 Puch bei Weiz, Austria. Tel: +43 (0)699 191 42 512 Mobile: 0043 (0)699 1 91 42 51 2. Email: joachim.hoffmann@gmx.at
JSoftware Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9. Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@jsoftware.com
KJK-tieto Oy	Kimmo Kekäläinen	Merikasarminkatu 10 B 56, 00160 Helsinki, Finland. Tel: +358 50 55 27 207; Email: Kimmo.Kekalainen@pp.hti.fi
Kx Systems	Simon Garland [Europe]	555 Bryant St., No. 375, Palo Alto CA 94301 USA. Tel: +1 866 kdb fast email: info@kx.com (in Europe simon@kx.com)
Lambent Technology Ltd	Stephen Taylor	81 South Hill Park, London NW3 2SS, UK. Tel: +44(0)20 7813 3786. Email: stj@lambenttechnology.com
Phil Last Ltd	Phil Last	146 Crossbrook Street, Cheshunt, Herts, EN8 6JY, UK. Tel: 01992-633807 Fax: 0121-359 0375 Email: phil.last@net.nfl.com
Lescasse Consulting	Eric Lescasse	18 rue de la Belle Feuille, 92100 Boulogne, France. Tel: +33 1.46.05.10.76 Fax: +33.1.46.04.60.23 Email: eric@lescasse.com
Lingo Allegro USA, Inc	Walter G. Fil	203 N. LaSalle Street, Suite 2100, Chicago, Illinois 60601, USA. Tel: +1 (800) 546 4621 E-mail: lingo-allegro@visto.com
Lucas Solutions	Jim Lucas	Stubbedamsvæj 9C, 3.tv., 3000 Helsingør, Denmark Tel: +45 49 26 52 42. Email: jel@danbbs.dk
Mackay Kinloch Ltd	Alastair Kinloch	519 Webster's Land, Edinburgh EH1 2RX, Scotland, UK. Tel: +44 (0)7802 430 202 Email: alastair.kinloch@btinternet.com
MasterWork Software Ltd	Fraser Jackson	PO Box 56-036, Tawa, Wellington, New Zealand. Tel: +64 (4) 232-4440 Fax: +64 (4) 232-4452 Email: fraser.jackson@xtra.co.nz
Mercia Software Ltd.	Gareth Brentnall	Mercia House, Ashted Lock, Aston Science Park, Birmingham, B7 4AZ, UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	Richard Nabavi	The Roller Mill, Mill Lane, Uckfield, E Sussex TN22 5AA Tel: 01825 768050. Fax: 01825 749472 Email: MicroAPL@microapl.demon.co.uk
Milinta Inc.	Dan Baronet	Contact Dan Baronet at dbaronet@milinta.com
Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants GU32 3PE, UK. Tel: 01730-263843 Email: apl@ellismorgan.co.uk
NY Sig	David Siegel	PO Box 2697, New York, NY 10163-2697, USA. Email: NYSIGAPL@ACM.ORG
Oasis b.v.	Louis Rijkse	Lekstraat 4, 3433 ZB Nieuwegein, Holland. Tel: +31 30 60 66 336 Fax: +31 30 60 65 844 Email: rijkse@oasis.nl
Omega Computing Inc	Alan Graham, Andrew Chou	3 Columbus Avenue, Edison, NJ 08817, USA. Tel: +1 (732) 985 9519 Email: alangraham@mindspring.com
Optima Systems Ltd	Paul Grosvenor	Optima House, Mill Court, Spindle Way, Crawley, West Sussex, RH10 1TT, UK. Tel: 01293 562700 Fax: 01293 562699 Email: paul@optima-systems.co.uk
Qualedi Inc.	Nicole Schless, Georges Brigham	121 West Main Street, Milford, CT 06460, USA. Tel: +1 (203) 874-4334 Fax: +1 (203) 876-9083. Email: sales@qualedi.com; info@qualedi.com
RadSys Technologies AB	Randolph Schrab	Lovsångarv. 18, S-756 52 Uppsala, Sweden. Tel: +46 18 32 41 53 Fax: +46 708 1996 11 Email: randolph.schrab@radsys.se
Renaissance Data Systems Ed Shaw		P.O. Box 511, Botsford, CT 06404, USA. Tel: +1 (203) 270-9729 Email: aplbooks@earthlink.com
Resources & Results	Frank Rhodes	342 Glen Oaks Blvd. Dallas, TX 75232, USA. Tel: +1 (817) 501-9825 Email: frank21@decision-support.net
RE Time Tracker Oy	Richard Eiler	Mikonkatu 8 A, 2 krs, PL 363, 00101 Helsinki, Finland. Tel: +358 9-621 3300 Fax: +358 9-621 3378 Email: re@rett.fi

Right Seat Software, Inc.	Tom Atkins	1110 12th Street, Golden, CO 80401, USA. Tel: +1 303 278 2244. Fax: +1 303 278 6967. Email: info@voxyproxy.com
Graeme Robertson	Dr. Graeme D. Robertson	15 Little Basing, Basingstoke, Hants RG24 8AX, UK. Tel: +44 (0) 1256-364071 Email: GraemeDR@nildram.co.uk
The Rochester Group Inc.	Robert Miller	600 Park Avenue, Rochester, NY 14607-2926, USA. Tel: +1 (716) 271-1110. Fax: +1 (716) 271-1230
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00149-Roma Trullo, Italy Email: mario.sacco@tin.it
SE APL Users Group	John Manges	413 Comanche Trail, Lawrenceville, GA 30044, USA Tel: +1 (770) 972-3755 Email: seapldoc@aol.com
Shepp & Associates LLC	Andrew Shepp	1312 Washington Avenue, 6th Floor St. Louis MO 63103, USA Tel: +1 (314) 621-3272 Fax: +1 (314) 621-4267 Email: ashepp@compuserve.com
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@interlog.com
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA. Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Sollton Associates		Sollton Inc, 44 Victoria Street, Suite 2100, Toronto, ON M5C 1Y2 Tel: +1 (416) 364 9355 Email: sales@sollton.com
SovAPL Russian Chapter of SIGAPL	Alexander Skomorokhov	PO Box 5061, Oboinsk-5, Kaluga Region 249020, Russia Tel: +7(08439)47109 Fax: +1 (530) 6885510 Email: askom@obninsk.com
Strand Software Inc	Anne Faust	P.O. Box 330, Excelsior, MN 55331 USA Tel: +1 (952) 470-7345 Fax: +1 (952) 470-9202 Email: info@strandsoft.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (860) 868-0131 Fax: +1 (860) 868-9970 Email: rex@rexswain.com
SwedAPL	Christer Ulfhielm	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd Sweden. Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CService: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: si@ifi.unizh.ch
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1-818-347-5779 Email: roysykes@earthlink.net
Toronto SIG	Richard Procter	PO Box 55, Adelaide St. Post Office, Toronto Ontario M5C 2H8, Canada Email: info@torontoapl.org
Weighahead Systems	Philip Bulmer	Camberley House, 1 Portesbery Road, Camberley, GU15 3RB, UK. Tel +44 1276 20789 Email: sales@weighahead.com
Stephen Wynn		8 Clarence Gardens, Brighton, Sussex BN1 2EG, UK. Tel: 01273-327238 Email: centre@bolblue.com
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (860) 872-7806

FTP SITES

IBM APL2	ftp.software.ibm.com/ps/products/apl2
Waterloo Archive	archive.uwaterloo.ca/ftparch/languages/apl
APL-to-ASCII	archive.uwaterloo.ca/languages/apl/workspaces/aplascii

WORLD WIDE WEB SITES

ACM SigAPL	www.acm.org/sigapl/
Adapta Software	www.adapta.de/
Adaptable Systems	www.assuredsystems.com.au/
AFAPL	www.afapl.asso.fr/ (Journal available on line)
APL2000	www.APL2000.com/
APL-385	www.demon.co.uk/apl385/
APL Forum	www.aplforum.com
APL Journal, Germany	www.rhombos.de/apljour.htm
APL Borealis Inc.	www.aplborealis.com
APL Systems IDC SL	www.apl2000.net

AUSCAN	http://home.ca.inter.net/rjp/auscan/
Eke van Batenburg	www.bio.leidenuniv.nl/~Batenburg/index.html
Carlisle Group	http://carlislegroup.com/
Causeway	www.causeway.co.uk/
CODEWORK	www.codework-it.com/tangram/eng/
CoSy (Bob Armstrong)	CoSy.com/
Dinosoft Oy	www.dinosoft.fi/
Dittrich & Partner	www.dpc.de/ ; www.apl-online.de
DMOZ - Open Directory	http://dmoz.org/Computers/Programming/Languages/APL/
Dyalog Ltd	www.dyalog.com/
DynArray	www.dynarray.com/
FinnAPL	www.pyr.fi/apl/
Godin London Inc	www.godin.com/
Houben	http://home.wanadoo.nl/houb/IDA.htm
Hoekstra Systems	www.HoekstraSystems.ltd.uk/
IBM APL2	www.ibm.com/software/awdtools/apl
Infostroy	www.infostroy.ru
Insight Systems ApS	www.insight.dk/
Japan APL Association	www.ae.keio.ac.jp/lab/soc/takeuchi/japla/
JSoftware Inc	www.jsoftware.com/
KJK-tieto Oy	www.kjk-tieto.com
kx systems	www.kx.com
Lambent Technology	www.lambenttechnology.com
Lescasse Consulting	www.lescasse.com/
Lingo Allegro USA Inc	www.lingo.com/
Mackay Kinloch	http://mackaykinloch.ltd.uk/
MicroAPL Ltd	www.microapl.co.uk/apl
Milinta Inc	www.milinta.com
Oasis b.v.	www.oasis.nl/
Optima Systems Ltd	www.optima-systems.co.uk
Qualedi, Inc.	www.qualified.com
Renaissance Data	www.aplbooks.com/
Resources & Results	www.decision-support.net
RE Time Tracker Oy	www.rett.fi/
Right Seat Software, Inc.	www.voxproxy.com
The Rochester Group Inc.	www.rochgrp.com/
Shepp & Associates	www.digitravel.com/
SigAPL	www.acm.org/sigapl/
Soliton	www.soliton.com/
Snake Island Research Inc.	www.snakeisland.com
Strand Software Inc.	www.strandsoft.com/
Rex Swain	www.rexswain.com/
Toronto SIG (for Toolkit)	www.torontoapl.org/
Jim Weigang	www.chilton.com/~jimw/
Weighahead Systems	www.weighahead.com

Zark Newsletter Extracts

by Gary Bergquist, reprinted by permission

Utility Corner: Word Wrap

(The purpose of this column is to make you more productive by introducing you to utility functions. Think of utility functions as APL functions that have names instead of symbols. By expanding your function vocabulary, you'll be able to write APL code that's more concise, more efficient, and more readable.)

In the last *Limbering Up* column (re-printed in *Vector* 20:3), you were asked to define a dyadic utility function `WRAP` that does word wrap. Its right argument is the character vector to be wrapped. Its left argument is the width within which the text is to be wrapped. Its result is a character vector with new-line (carriage return) characters inserted in such a way that the text wraps appropriately when displayed. For example:

```
17 WRAP 'This is a sample. The last nonblank character in
each line fits within the specified width.'
```

```
This is a sample.
The last nonblank
character in each
line fits within
the specified
width.
```

The piece of text is broken into subsets, without breaking any words, so that each subset is no longer than the specified width, in this case 17.

The result of `WRAP` has the same length as the right argument. The newline character (`␣TCNL` or `␣TC[2]`) replaces the last blank after the last word in each line.

If the right argument already contains newline characters, they are not ignored or removed. Each one begins a new line. In a sense, existing newlines are "paragraph" delimiters.

Each group of contiguous nonblank characters is considered a word. The aim is to wrap the text without breaking any words. However, if any "word" is longer than the specified width, `WRAP` inserts one or more newline characters within it to

break it into pieces whose length equals the width. If this is the case, the result will naturally be longer than the right argument.

Your primary objective was to write a function that worked. You were offered extra credit (a pat on the back) if your function was efficient, clever, simple, elegant, or could leap tall buildings in a single bound.

In the "leap tall buildings in a single bound category" was the following entry:

```

      ▽ R←WID WRAP CVEC;C;I;J;M;P;TCNL;⌈IO
[1]  A Wrap text CVEC into lines of width
[2]  A WID or less by inserting newline
[3]  A Characters.
[4]  ⌈IO+1 A Origin 1
[5]  TCNL←⌈TCNL A APL*PLUS newline
[6]  A TCNL←⌈TC[2] A APL2
[7]  A Algorithm needs start/end blanks:
[8]  CVEC←' ',CVEC,' '
[9]  A Indices of blanks
[10] J←(CVEC=' ')/⌈pCVEC
[11] A 1s mark blanks past the cutoff:
[12] P←(J+WID+1)∘.<J
[13] A Mark last blank that fits on line
[14] M←P<1⌈P
[15] I←(pM)p1,(1+pM)p0 A Identity matrix
[16] C←⌈I-M A Transitive closure of M
[17] A Insert the line breaks:
[18] CVEC[C[1;]/J]←TCNL
[19] A Drop the blanks we added
[20] R←1+~1+CVEC
      ▽

```

Since the algorithm employed in the `WRAP` function is intuitively obvious (especially line [16]), we won't waste space or time explaining it.

Well... maybe we'll say a few words. This algorithm was originally presented in the mid-1970s by Bob Smith of Scientific Time Sharing Corporation in response to a challenge like this one in the Sharp APL newsletter. The solution is non-looping by virtue of its use of `⌈` to compute the transitive closure of a Boolean matrix.

The algorithm was resurrected for this Limbering Up exercise by Jim Weigang and Bob Smith. Bob Smith even offered a page of explanation, which appeared to be in English. Copies are available for the masochistic. Write to us.

Rex Swain recognized that the Bob Smith solution, as expressed above, does not solve the Limbering Up task. Specifically, it does not handle multiple contiguous

blanks or words that are longer than the specified width. So he fixed it, without sacrificing its non-looping nature.

```

▽ R←WID WRAP CVEC;A;B;C;D;E;I;L;P;TCNL;Z;IO
[1]  A Wrap text CVEC into lines of width
[2]  A WID or less by inserting newline
[3]  A characters.
[4]  IO←1 A Origin 1
[5]  TCNL←TCNL A APL*PLUS newline
[6]  A TCNL←TC[2] A APL2
[7]  B←CVEC=' ' A Flag blanks
[8]  A Flag all but the last letter in
[9]  A each word:
[10] A←B÷1+B,1
[11] A Partition words with more than one
[12] A letter
[13] P←A/1+B,1
[14] A Point to the left of those "long"
[15] A words
[16] C←1+P/1pP
[17] A Lengths of those words:
[18] C←C-1÷0,C
[19] A Treat breakpoints in long words as
[20] A blanks
[21] B←B∧A\0=WID|+1-P\C
[22] A Right-hand blanks in sequences of
[23] A blanks:
[24] Z←B>1+B,0
[25] A Left-hand blanks in sequences of
[26] A blanks:
[27] L←B>1÷0,B
[28] A For each A, the corresponding
[29] A value in C is the index in A of
[30] A the right-most blank within WID+1
[31] A characters
[32] C←+/(WID+1)+0,Z/1pZ)°.≥0,(L/1pL),1+pL
[33] A Ye olde identity matrix:
[34] I←(2ppC)p(1+pC)†1
[35] A Key is transitive closure:
[36] A←(Z\1=1†((pC)†1)⊗I-(1pC)°. =C)/1pCVEC
[37] A Now A points to blanks that should
[38] A be replaced with newlines, except
[39] A in too-long words, where we want
[40] A to insert the newlines after the
[41] A character:
[42] I←CVEC[A]≠' ' A Need to insert?
[43] A Where NLs go (after expansion)
[44] A←A++\I
[45] A Expansion vector
[46] E←((pCVEC)++/I)p1

```

```

[47] A Need extras here for inserted NLs:
[48] E[I/A]+0
[49] R+E\CVEC A Do the expansion
[50] A Replace and/or insert NLs
[51] R[A]+TCNL

```

▽

Unfortunately, this function still does not properly handle the existence of embedded newlines. But hey! Who cares? These functions can't be used for practical problems anyhow. They're too dang slow. The first function can be likened to driving in a thumb tack with a nuclear warhead. The second function embellishes upon that by requiring that a lead vest be worn during the process.

Jim Weigang, recognizing that a practical solution was also needed, offered one:

```

▽ R+WID WRAP CVEC;B;C;E;I;J;N;S;T;TCNL;□IO
[1] A Wrap text CVEC into lines of width
[2] A WID or less by inserting newline
[3] A characters.
[4] □IO+1 A Origin 1
[5] TCNL+□TCNL A APL*PLUS newline
[6] A TCNL+□TC[2] A APL2
[7] A Simplify edge conditions
[8] CVEC←' ',CVEC,' '
[9] A Mark blanks and newlines:
[10] B+CVEC←' ',TCNL
[11] A Last of each group of blanks
[12] C←(B>1+B,0)/1pB
[13] A Nonblank before each grp of blks:
[14] E←~1+(B>~1+B,0)/1pB
[15] A Indices of newlines:
[16] N←(CVEC=TCNL)/1pCVEC
[17] I←10 A Where to insert add'l TCNL's
[18] A Index of character before start of
[19] A current line:
[20] S←1
[21] A Loop for each line. Find word that
[22] A Extends past line end, & back up:
[23] L1:J←~1+(E>S+WID);1
[24] →(J=pE)/L4 A Quit if at end
[25] A Newlines within this line:
[26] →(0=pT+((S<N)^N≤C[J])/N)/L2
[27] A Use last one as new base; repeat:
[28] S←~1+T
[29] →L1
[30] L2:T←S A Save old base
[31] S←C[J] A New base (break point)
[32] →(S>T)/L3 A Branch if okay
[33] A Else word is too long; remember to

```

```

[34] A insert a newline past S:
[35] I←I,S←T+WID
[36] →L1
[37] A Else, replace blank w/ newline:
[38] L3:CVEC[S]+TCNL
[39] →L1
[40] A Branch unless some newlines need
[41] A inserting (long words)
[42] L4:→(0=ρI)/L5
[43] E←((ρCVEC)+ρI)ρ1 A Form expand mask
[44] E[I+I+ρI]+0 A ...where newlines go
[45] CVEC+E\CVEC A Insert spaces
[46] CVEC[I]+TCNL A Put in newlines
[47] L5:R←I+1↓CVEC A Drop blanks added

```

▽

A loopier. Jim's approach is to find the indices of the starts (C) and ends (E) of words and of the embedded newlines (N). He then loops, doing one line at a time.

On line [23], he looks ahead WID characters to find the end of the current line. If there's a newline within the next WID characters, he starts from that newline to look ahead again (line [28]). If there are no newlines, he finds the end of the last word and replaces the (last) blank after that word with a newline character (line [38]). If there is no last word because the line is just one *long* word, he makes a note to insert a newline character in the middle of the word (lines [35, 42-46]) and proceeds to the next line.

Bill Chelew also offered a looping solution:

```

▽ R←WID WRAP CVEC;N;TCNL;V;□IO
[1] A Wrap text CVEC into lines of width
[2] A WID or less by inserting newline
[3] A characters.
[4] □IO←1 A Origin 1
[5] TCNL←□TCNL A APL*PLUS newline
[6] A TCNL←□TC[2] A APL2
[7] R←''
[8] A Exit if nothing left:
[9] LOOP:→(N+ρCVEC)↓0
[10] A Grab up to WID+1 chars:
[11] V←(N+N[WID+1])↑CVEC
[12] A Done if no more than WID:
[13] →(N>WID)/L1
[14] R←R,V
[15] →0
[16] A Any newlines in this string?
[17] L1:→(TCNL≠V)↓L2
[18] A If so, keep up to it and go back:
[19] R←R,(N-V)↑TCNL)↑V

```

```

[20]  CVEC+N+CVEC
[21]  →LOOP
[22]  A Branch if last char is blank:
[23]  L2:→(' '=⌈1↑V)/L4
[24]  A Index of last blank in V
[25]  N←(WID+2)-(ϕV)⌈' '
[26]  →(N=0)/L3 A Branch if no blanks
[27]  V[N]←TCNL
[28]  R←R,N↑V
[29]  CVEC+N+CVEC
[30]  →LOOP
[31]  A Use full WID if no blanks
[32]  L3:R←R,(WID↑V),TCNL
[33]  CVEC+WID+CVEC
[34]  →LOOP
[35]  A Branch if any nonblanks
[36]  L4:→(v/V≠' ')/L5
[37]  R←R,TCNL A Newline if all blanks
[38]  →L6
[39]  L5:V[WID+1]←TCNL
[40]  R←R,V
[41]  CVEC←(WID+1)↑CVEC
[42]  A Go back if no leading blanks
[43]  →(' '≠1+CVEC)/LOOP
[44]  A Delete leading blanks
[45]  L6:N←(CVEC≠' ' )⌈1
[46]  CVEC←(N-1)↑CVEC
[47]  →LOOP

```

▽

Bill's approach was to build the result by catenating to it one line at a time. At each iteration, he looks ahead $WID+1$ characters. If there aren't that many left, he's done (lines [12-15]). If the line of characters contains a newline, he grabs all the characters up to and including the newline, catenates them to the result, and goes back for the next line ([16-21]).

If the line contains no newlines, he finds the last blank before the last nonblank and inserts a newline character there ([27-30]), catenating those characters to the result. If there are no blanks in the line, he breaks the *long* word after WID characters, inserting a newline ([31-34]) and catenating it to the result. If the line is all blank, he catenates a newline to the result and discards all subsequent blanks ([36-38] and [44-47]).

If the line ends on a word that is followed by multiple blanks, and if those blanks extend beyond the specified width, the first WID characters are catenated to the result along with a newline, and the remaining blanks are discarded ([39-47]).

These last two functions seem to handle all practical examples you would be likely to throw at them. To find out which was faster, we timed them (as well as the first two functions, for the chuckles).

Here's what we found:

Example	1	2	3	4
35 WRAP 200p...	40.8	17.0	1.47	1.03
(with newlines)	--	--	1.55	1.00
65 WRAP 2000p...	4932	1788	1.73	1.00
(with newlines)	--	--	1.95	1.07
65 WRAP 20000p...	--	--	2.17	1.00
(with newlines)	--	--	2.85	1.09

- 1 = Nonlooping  version
- 2 = Doctored up  version
- 3 = Loop, inserting newlines
- 4 = Loop, catenating to result

For each size of problem (200, 2000, 20000 elements), the timings were converted to multiples of the fastest algorithm. For example, the fastest algorithm for a 200-element character vector was algorithm 4 when the character vector contained occasional newline characters. Algorithm 1 took 40.8 times longer to run.

For a 2000-element character vector, algorithm 4 was faster when the vector did not contain newline characters than when it did.

The fastest algorithm for the 2000-element vector took 6.59 times longer to run than the fastest algorithm for the 200-element vector. The fastest algorithm for the 20000-element vector took 111 times longer to run than the fastest algorithm for the 200-element vector.

Limbering Up: Permutations and Anagrams

(The purpose of this column is to work some flab off your APL midsection. Like muscles, your APL skills can atrophy if not exercised with adequate frequency and variety. This column presents a task for you to perform. Set aside a few minutes from your busy schedule and work the task. Mail in your solution and stay tuned for the results.)

In the 1992 Quarter 2 issue of the Zark APL Tutor News, a highly efficient (though obscure) function for constructing combinations was presented:

```

▽ R←M COMBIN N;D;E;F;G;P
[1]  A Returns a matrix of every possible
[2]  A combination of M elements from the
[3]  A vector ιN. That is, returns a ma-
[4]  A trix with M!N rows and N columns.
[5]  A
[6]  E←(ιP←N-R+M-1)~⊖IO
[7]  D←R+ιP
[8]  R←(P,1)ρD
[9]  P←Pρ1
[10] L1:→(⊖IO>1↑D←D-1)ρ0
[11] P←+\\P
[12] G←+\\~1+0,F←ϕP
[13] E←F/E-G
[14] R←(F/D),R[E+ιρE;]
[15] E←G
[16] →L1
▽

```

```

3 COMBIN 5

```

```

1 2 3
1 2 4
1 2 5
1 3 4
1 3 5
1 4 5
2 3 4
2 3 5
2 4 5
3 4 5

```

Your task is to write a monadic function PERMUTE, whose argument is a positive integer N and whose result is an N-column, (|N)-row matrix, in which each row is a different permutation of ιN.

```
PERMUTE 3
```

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

Try to make the function as efficient as possible. In case you don't find this exercise adequately limbering, your second task is to write a monadic function **ANAGRAM**, whose argument is a character vector, and whose result is a character matrix in which each row is a different arrangement of the argument's letters. (**ANAGRAM** calls **PERMUTE**.)

```
ANAGRAM 'SMSI'
```

```
IMSS
ISMS
ISSM
MISS
MSIS
MSSI
SIMS
SISM
SMIS
SMSI
SSIM
SSMI
```

Send your solution to:

Vector Production
Brook House
Gilling East
York YO62 4JJ
UK email apl385@compuserve.com

The notable functions and their authors' names will be printed in the next issue of *Vector*. Good luck and happy limbering.

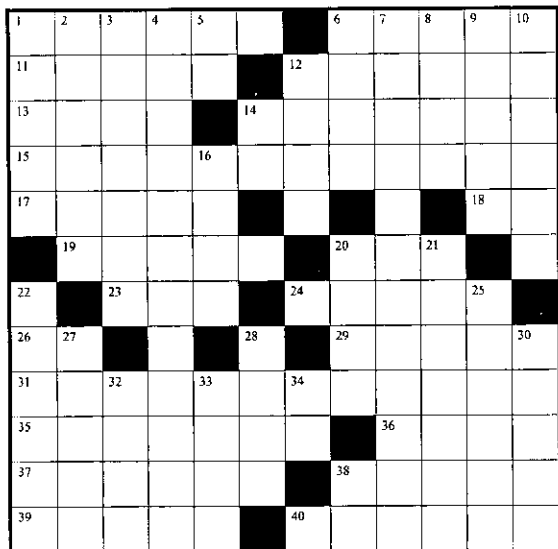
Reprinted with kind permission from Zark APL Tutor News, a quarterly publication of Zark Incorporated, 23 Ketchbrook Lane, Ellington, CT06029, USA

20.4 Solution, Puzzle, and New Crossword

Solution to Crossword in 20.4

A	L	P	H	A	B	E	T	S	O	U	P
N		P	I	F	-	L	P	I	F		E
D	L		T	=	T	L	ϕ	C		C	R
O	D	D		E	R	/	2		1	-	M
N	$\geq$	*	R			V	$\div$	5	$\uparrow$	B	U
E	D	$\div$	L	T	H		K		ρ	ρ	T
F	B	E		+		B	R	$\neq$	M	$\div$	A
O	C	-	B	?	V	C		G	A	$\times$	T
R	τ	1		2	+	,	T		T	$\backslash$	I
A	V		L	0	.	5	+	M		G	O
L		W	/	-	$\times$	/	S	,	Y		N
L	E	A	S	T	S	Q	U	A	R	E	S

New Crossword



Across

1. F random numbers from $(-B+V)$ to $(-B+1)$
6. Number of values of vector DS not in V
11. $UM \times EV$ for numeric scalar UM , bit vector EV , without multiplying
12. $[KBp \tau E]$
13. The largest value in vector RM
14. Is W true and VND identical to C ?
15. The non-function symbol with orders of magnitude significance
17. The number JGB smaller than S
18. $\square$, the first number in τN
19. $\vee / A \wedge \sim D$ for vectors A and D
20. $(\neg 1 \ 0 \ 1) [\square IO + (IN \geq 0) + (IN > 0)]$
23. And so on
24. How many values in vector N are found in V ?
26. The next integer at or below Y
29. Matrix U ravelled, with scalar AJ preceding the values of each row
31. All

35. $\ast (\odot B) + MF \times L$
36. Word prefix meaning 31 across
37. Absolute value of the quotient of BKA and H
38. $(-R) - \tau O - R$ (O not zero) in origin 1
39. τ tells where, ϵ tells whether, a value is ____
40. Responsibility of system architect

Down

1. The lesser of $-E$ or $_{DS}$
2. $(+ / BM) \rho E - 1$ for bit vector BV , scalar E
3. Vector whose first difference is $RJC \epsilon E$
4. Who **DEBUGGER BOY** grows up to be
5. A random number from the set τM
6. Number of dimensions in VP
7. Divisors
8. First 5 elements of vector DI
9. Are arrays K and NI different?
10. Rank 1 array
12. Are both B and LT true?
14. $\square$ __, the number of bytes remaining
16. Is MB larger than C ?
20. Product of all the numbers in array C
21. Is the number N found in vector I beyond its first JIM elements?
22. The remainder of $F \div AB$, but no more than 4
25. V followed by the location of G in vector ON
27. $' \rho \rho, (YN, BO) \rho 0$ for scalars YN and BO
28. Surface area of cylinder with diameter D , height H , excluding ends
30. $\wedge$ is to intersection as $\vee$ is to ____
32. $\times / KU \rho D$ for scalars KU and D
33. The number of D size slices in an EF size pizza
34. $\square$ __, the number of users on your computer
38. $E \times^{-1}$

J-ottings 41: Shortest Paths and Related Algorithms

by Norman Thomson

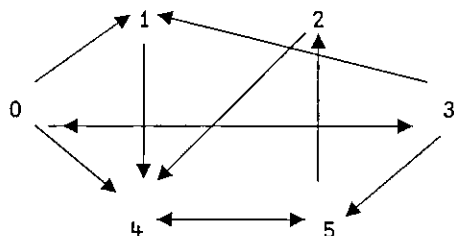
Coincidentally, Eugene MacDonnell and I were simultaneously and independently thinking about shortest path algorithms. Eugene's interest was as a step in finding word ladders; mine in generalised joining verbs such as JE (Join Each) - see J-ottings 40 - which is a key component of the verbs *extend* and *slack* below. JE performs scalarised joins, that is, it is a derivative of *append* which behaves like a scalar verb with a result which is always a boxed list or a list of boxed lists.

```
box=.]`<@.(-:>)    NB. boxes if unboxed, else do nothing
EACH=.&.>          NB. adverb
JE=.,EACH&box      NB. scalarised join, results boxed
```

The following examples demonstrate how JE works:

```
1 2 JE 3 4 5          NB. append and box two lists
+-----+
|1 2 3 4 5|
+-----+
(1 2;4 5 6) JE 7 8 9    NB. cf. 1 2 + 3
+-----+
|1 2 7 8 9|4 5 6 7 8 9|
+-----+
(1 2;2 4 6) JE 4 5;7 8 9 NB. cf. 1 2 + 3 4
+-----+
|1 2 4 5|2 4 6 7 8 9|
+-----+
```

To begin the search for shortest paths, it helps to be specific by using a network in the form of a directed graph such as the following:



There are three reasonably obvious ways in which such a network can be represented. First it can be represented as a list of 2-lists each of which is a directed arc

```
arcs=.0 1;0 3;0 4;1 4;2 4;3 0;3 1;3 5;4 5;5 2;5 4
```

or this can be compacted into a list of n "nearest neighbour" lists where n is the number of nodes:

```
n1=.1 3 4;4;4;0 1 5;5;2 4
```

or thirdly, it can be represented as a matrix in which unconnected nodes are represented by _:

```
]g1=.1(arcs)}6 6$_      NB. amend replacing _ with
weights
```

```
_ 1 _ 1 1 _
_ _ _ 1 _
_ _ _ 1 _
1 1 _ _ 1
_ _ _ 1
_ _ 1 _ 1 _
```

The matrix in which unconnected nodes are represented by 0s is simply %g1, and the nearest neighbour form can be recovered from the matrix form by

```
btoi=.# i.@#      NB. binary list to list of 1-
positions
mton=(btoi@:%)EACH@<"1 NB. matrix to neighbour
representation
mton g1
+-----+-----+-----+
|1 3 4|4|4|0 1 5|5|2 4|
+-----+-----+-----+
```

The reverse transformation to mton is:

```
ntom=.monad : 'i((i.n),EACH EACH y.))(n,n=.#y.)$_'
      NB. neighbour to matrix representation
```

The rationale for using _ to represent unconnected nodes is partly visual, but more importantly, the shortest path matrix (that is, the matrix whose values are the lengths of the shortest paths between all pairs of nodes) can be obtained by using the 'minimum-dot-plus' inner product applied using the power conjunction as many times as there are nodes in the graph.

```

spm=.monad : '<./y.(<./ .+)^:(i.#y.)y.'
spm g1                      NB. all shortest paths (non-
weighted)
2 1 3 1 1 2
_ _ 3 _ 1 2
_ _ 3 _ 1 2
1 1 2 2 2 1
_ _ 2 _ 2 1
_ _ 1 _ 1 2

```

spm has the merit of working equally well for weighted graphs, that is if the non-infinity values in the base matrix are path-lengths rather than just ones.

```

wts=.2 7 6 1 5 7 6 1 4 8 4
]h1=.wts(arcs)}6 6$ _      NB. amend replacing _ with
weights
_ 2 _ 7 6 _
_ _ _ _ 1 _
_ _ _ _ 5 _
7 6 _ _ _ 1
_ _ _ _ _ 4
_ _ 8 _ 4 _

```

```

spm h1                      NB. all shortest paths (weighted)
14 2 15 7 3 7
_ _ 13 _ 1 5
_ _ 17 _ 5 9
7 6 9 14 5 1
_ _ 12 _ 8 4
_ _ 8 _ 4 8

```

For testing purposes, a random network y .square with a density x . is obtainable by:

```
randg=.dyad : '%(100*x.)>?(y.,y.)$100'
```

a random weighted matrix with weights in the range 1 to 100 by

```

randh=.dyad : '(>:(y.,y.)$100)(*&.)x.randg y.'
0.25 randh 8              NB. 25% of paths valid in 8 by 8 matrix
_ _ 1 _ _ 46 _ _
_ _ _ 37 93 96 _ 7
12 _ _ 51 _ _ 42
_ 71 57 _ _ _ 24 8
42 _ _ _ _ 80 52 _
86 _ 71 26 _ 8 _ _

```

To generate a symmetrical matrix using the values above the leading diagonal use

```

mksym=.(+|:.)@(*~/~@(.@#))    NB. make symmetrical
%mkysym%0.6 randh 4            NB. symmetrical weighted matrix
79 23 11
79 25
23 25 68
11 68

```

So far the discussion has focussed on the *lengths* of the shortest paths. SP below returns the actual shortest path between any single pair of vertices of an unweighted graph:

```

SP=:4 : 0
r=.{.y. [ tgt=.:y.
while.(~tgt e.tails r)do. r=.r extend x.addnodes r end.
r=.(tgt=tails r)#r
)
tails=.:{:G>                NB. tails of list of lists
addnodes=.<@[rtoi EACH tails@] NB. new nodes now reachable
rtoi=.btoi@: (%@~)          NB. btoi after making _s into 0s
extend=.:@:(JE"1 0 EACH)    NB. add new nodes to curr. paths

g1 SP 0 2                    NB. shortest path(s) from 0 to 2
0 3 5 2
0 4 5 2

```

Next here is an algorithm which returns all paths between a pair of nodes for an unweighted matrix:

```

AP=:4 : 0                    NB. All Paths
p=.i.0 [ r=.{.y. [ 'src tgt'=.y. NB. initialise loop
while.(0~:#r)do.            NB. r = incomplete paths
r=.r~.p [ p=.p,((tails r)e.tgt)#r NB. p = complete paths
r=.r extend x.addnodes r      NB. add one node to each r path
r=.{#~((=&#~.)&>))r          NB. remove loops
end. p
)

g1 AP 0 2                    NB. all path(s) from 0 to 2
+-----+-----+-----+-----+
|0 3 5 2|0 4 5 2|0 1 4 5 2|0 3 1 4 5 2|
+-----+-----+-----+-----+

```

In the case of a weighted matrix it may not be the case that the path with the smallest number of steps is the one with the lowest weighted value. To calculate the weighted value of a path use

```

steps=<"(1)@(2&{,/\\)) NB. boxed list of node-pairs
values=.,+/@:([~steps@]) NB. sum of step values from wtd matrix
h1 values 0 3 5 2 NB. sum of (0 3;3 5;5 2){h1

```

16

To find the critical path in a network, find the minimum value path from all paths:

```

>(<h1) values EACH g1 AP 0 2
16 18 15 26
((=<./)>(<h1) values EACH t)#t=.g1 AP 0 2
+-----+
|0 1 4 5 2|
+-----+

```

To build this into a critical path verb with a single argument `h1`, observe that a weighted matrix is converted into an unweighted (that is 1 or _) form by `unweight=.%<&_`

```

CP=:4 :0 NB. critical path
t=.(unweight x.) AP y. NB. all paths
v=>(<x.) values EACH t NB. all paths evaluated
(((=<./v)#t),<./v NB. minimum value path selected
)
h1 CP 0 2 NB. critical path and value
+-----+
|0 1 4 5 2|15|
+-----+

```

If there is more than one critical path, all are reported, together with their common value. For a list of slack values associated with each path, only a slight modification to `CP` is needed, together with a further application of `JE` :

```

slack=.-~ >./
SLACKS=:4 :0
t=.(%x.<_) AP y.
t JE EACH slack >(<x.) values EACH t
)

```

```

h1 SLACKS 0 2
+-----+
|+-----+|+-----+|+-----+|+-----+| | | | | |
||0 3 5 2 10||0 4 5 2 8||0 1 4 5 2 11||0 3 1 4 5 2 0||
|+-----+|+-----+|+-----+|+-----+|
+-----+

```

Next here is a transliteration of Eugene's shortest path algorithm `FL` adapted to accept a weighted matrix as argument and with the value of the shortest paths attached. I have also lengthened the noun names to help exposition. The algorithm

is virtually self-describing, and it is also quite hard to spot the primitive J verbs, indicating how much of the work has already been embodied in the J interpreter.

```

FL=.4 :0
'src tgt'=.y.
pathsssofar=.,.src [ oldnodes=.newnodes=.,.src
while. -. tgt e. newnodes do.
  newodelist=.btoi EACH<"(1) _~:newnodes{x.}-.EACH<oldnodes
  newnodes=.;newodelist
  if. '' -: newnodes do. _1 return. end.
oldnodes=.,.oldnodes,newnodes
pathsssofar=.(#&>newodelist)#pathsssofar),.newnodes end.
r=.pathsssofar#~tgt=newnodes
r,.;(<x.)value EACH<"1 r
)
h1 FL 0 2          NB. shortest path(s) from 0 to 2 plus lengths
0 3 5 2 16
0 4 5 2 18

```

Eugene's algorithm is a great deal faster than SP, the only disadvantage is that, as far as I can see, it is not easy to adapt it to find all paths and critical paths because the paths are homogeneous at all stages, and the program stops whenever an occurrence of the target node is detected.

Now that shortest paths, all paths and critical paths are available, a great variety of other possible results can be obtained. For example, returning to the unweighted case, the connectivity matrix is simply the shortest path matrix with 1s indicating non-infinity entries. An entry of 1 on the leading diagonal means that there is an outward path from the corresponding node which leads back to it, otherwise the entry is 0.

```

connected=.,:&_@spm
connected h1    NB. connectivity matrix
1 1 1 1 1 1
0 0 1 0 1 1
0 0 1 0 1 1
1 1 1 1 1 1
0 0 1 0 1 1
0 0 1 0 1 1

```

The list of lists of reachable nodes from each node as starting point is given by

```

reachable=.btoi EACH@<"1@connected
]u=.reachable h1
+-----+-----+-----+-----+
|0 1 2 3 4 5|2 4 5|2 4 5|0 1 2 3 4 5|2 4 5|2 4 5|
+-----+-----+-----+-----+

```

and a list of all pairs of nodes between which a path exists is given by

```
arclist=.;@:((i.@#)(,EACH EACH)reachable)
12{.arclist h1      NB. first 12 items in arclist
+-----+-----+-----+-----+-----+-----+
|0 0|0 1|0 2|0 3|0 4|0 5|1 2|1 4|1 5|2 2|2 4|2 5|
+-----+-----+-----+-----+-----+-----+
```

Footnote

It seems that everybody is thinking about graphs these days! Following John Scholes "A Note on Graphs" in Vol. 20 No. 4 here is an equivalent breadth-first search algorithm using the apparatus described above.

```
bf=.4 :0
r=.y.
while.(0 e.(i.#x.)e.~.r)do.r=.~.r; x. addnodes r end.
)
sg      NB. John's first graph in my notation
- 1 1 - -
- 1 - -
- 1 - 1 -
1 - - 1
- 1 - -
sg bf 4
4 2 1 3 0
```

Spanning trees are a little more complicated as John's APL function demonstrates. Also for the general sort of networks which we both allow, it may not always be the case that a spanning tree exists starting from a given root. The principle of the spanning tree algorithm below is to use arrays to advance on a broad front in the while loop, and then prune the paths once all vertices have been visited.

This does not in general produce a *minimum* spanning tree, but neither I think does John's algorithm.

```

st=.4 : 0      NB. syntax is <graph> st <root node>
r=.y. [ v=.i.#x. NB. v = nodes still to visit
while.(0<#v)do.
  v=. (i.#x.)-.;r [ r=.r extend remdn(x. addnodes r) end.
  rems rdup~.EACH y.,EACH r-. EACH <y. NB. prune tree

remdn=.3 :0      NB. remove duplicate newnodes
r=.y. [ s=.~.>i{y. [ i=.0
while.(i<<:#y.)do.
  s=.s,~.>i{r [ r=.((<>i{r)-.s)i}r [ i=.i+1 end. r

iss=. [-: (#@[]){.} NB. is-suffix
rems=.monad : ' (1=+/"1 iss&>/~y.)#y.' NB. remove suffix
rdup=.#~i.~=#i.@# NB. remove duplicate nodes in path

sg st 0
+-----+-----+-----+
|0 1 2 3|0 2 1|0 2 3 4|
+-----+-----+-----+

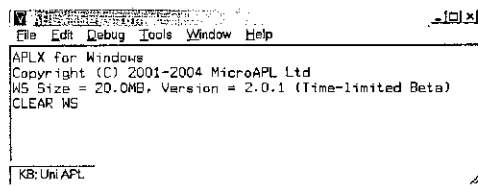
```

All of which goes to prove along with the song that anything APL can do, J ...
oops, can't remember how the words go after that

APLX Beta 2.0.1 for Windows

reviewed by Ajay Askoolum

APLX is a cross platform APL development environment for Windows, MacOS and Linux. Workspaces are interchangeable across platforms. Portability requires that no platform specific features be used except in conditional blocks of code. For instance, a `Ona` function based on a Windows API call will be meaningless on the Mac or Linux platforms.

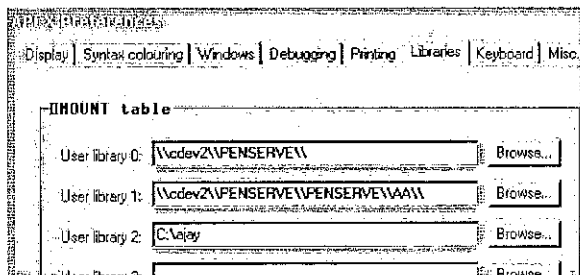


Although APLX is the newest commercial APL interpreter, it is based on APL.68000 and therefore it is not only robust but also from an experienced APL provider, MicroAPL.

This article is based on the Windows personal version (there is a server edition). I have explored APLX on Windows 2000. APLX distinguishes itself with several unique cutting edge features.

Building on the APL.68000 Legacy

APLX still supports the APL.68000 features, including the `del` editor, component file functions (`⎕`, `⎕` etc. see below), library numbers etc. For a cross platform development, the use of library numbers provides a unified method for referring to folders in APL code.



Up to 10 permanent library numbers (0 to 9) can be defined using the **Tools | Preferences + Libraries** tab.

`⎕mount ''` returns current definitions.

Libraries may be defined either using Universal Naming Convention (UNC) or drive letters or both. Libraries created using `⎕mount Variable` are session bound only; permanent libraries need to be defined using the above dialogue.

```

tie1←'0 afile.txt' Oncreate 0 a Library numbers with native files
tie2←'0 penserve\aa\bfile.txt' Oncreate 0 a Path appended to library
tie3←'1 cfile.txt' Oncreate 0
tie4←'2 dfile.txt' Oncreate 0
Onnums
1 2 3 4
Onnames a Refers to UNC names, not libraries
\\cdev2\\PENSERVE\\afile.txt a UNC name returned
\\cdev2\\PENSERVE\\penserve\aa\bfile.txt a Path correctly appended
\\cdev2\\PENSERVE\\PENSERVE\\AA\\cfile.txt
C:\ajay\dfile.txt a Fixed path returned

```

Library numbers and UNC or fixed paths can be used interchangeably:

```

'\\cdev2\\PENSERVE\\afile.txt' Onerase tie1
'0 penserve\aa\bfile.txt' Onerase tie2 a Library + path
'1 cfile.txt' Onerase tie3 a Library instead of path
'C:\ajay\dfile.txt' Onerase tie4 a Path instead of library

```

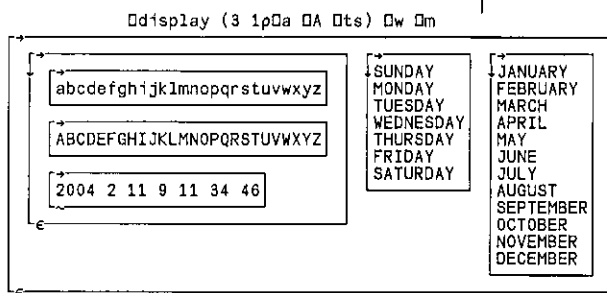
Although APLX provides support for the APL and unified keyboards, it provides a facility for switching from the APL keyboard to the locale keyboard using Ctrl + N. This is useful in that the unified keyboard does not appear to map all keys to the locale keyboard. For instance, # is found on shift + 3 and not where engraved on the keycap. This may quite correctly be perceived as a nuisance as it should not be necessary – this is reminiscent of ALT + BCKSPACE on mainframe APL. Vendors may not be able to ignore internationalisation considerations for much longer. APLX has begun to make inroads into this area: this is such an obvious area for vendors to establish competitive advantage.

The system function `Onconf` allows some configuration of date and time formats as shown on the right. However, this does not extend to the language in which system constants are reported.

```

)save
2004-02-09 18:58:57 aa
6 Onconf 3 1 1
4 0 1
)save
9/02/2004 18:59:08 aa

```



In line with other APL, now APLX has the full list of `On` and `On`* system constants or functions.

For now, APLX returns these constants without any reference to the locale: it needs to return this information in line with it.

It is possible to protect a workspace with a password – this requires library numbers to be used.

Any attempt to load a protected workspace without specifying the password fails with a 'WS LOCKED' error; there is no prompting for the password.

```
)save 0 myws:mypwd
2004-02-08 16.23.21

)load 0 myws
WS LOCKED
)load 0 myws:mypwd
SAVED 2004-02-08 16.23.21
```

APL has the ability to save the contents of the active workspace and re-open it in its entirety or retrieve selected functions and variables from the archive.

APLX can transfer the active workspace to a transfer (*.ATF) file. The name of the transfer file may be a fully qualified name or one specified by reference to a library number – APLX adds the ATF extension.

The contents of a transfer file can be added to the active workspace either in its entirety or selectively.

```
)out 0 myws

If library 0 is defined as c:\, the
above is equivalent to:
```

```
)out c:\myws
)clear
CLEAR WS

)in c:\myws ac

pDn1 2 3
1 2
)in c:\myws
pDn1 2 3
2 2
```

An ATF file is a workspace saved in text format. As with workspaces, the appropriate system commands do not require the file extension to be specified. Although most objects transfer correctly, another APL may not be able to read the entire ATF file because of language and atomic vector differences.

APL system commands can be executed within code rather than in immediate mode only.

```
vExecSysCmd
[1] a←a')symbols'
v 2004-02-08 16.30.32
ExecSysCmd
a
IS 1026, USED 2
```

Cross-platform Development

Platform commands can be executed from the interpreter. For Windows, the syntax is shown on the right. The string `CMD /C` instructs the interpreter to use `COMMAND.COM`.

The host system can be interrogated as shown on the right.

```
a←Dhost 'CMD /C dir c:\aa /s'
p⇒(a#Dtcn1)ca
181 81
pb←⇒(a#Dtcn1)ca
181 81
b[13:]
Volume in drive C is Ajay
Volume Serial Number is 6C34-55CF
Directory of c:\ajay

      Dhost ''
WINDOWS
```

Since the host name can be queried programmatically, it is possible to embed platform specific code within functions: this removes the need for maintaining platform specific workspaces in isolation. NOTE: workspace files are interchangeable across platforms.

```
vZ←PlatformCode;A;B;C
[1] A Illustrates platform dependent coding
[2] A←'Hello! 'A block of common code goes here
[3] :Select Dhost '' A platform specific code in a :select structure
[4] :Case 'AIX' A AIX specific code
[5] :Case 'LINUX' A LINUX specific code
[6] :Case 'MACOS' A MAC specific code
[7] :Case 'WINDOWS' A Windows specific code
[8] B←'This is APLX hosted by Windows.'
[9] :EndSelect
[10] C←'Task Id is ',⌈'#' Dwi 'taskid' A more common code
[11] Z←A B C
v 2004-02-10 21.37.09

⇒PlatformCode
Hello!
This is APLX hosted by Windows.
Task Id is 15166396
```

The APLX interpreter functionality simplifies access to some platform features - very neatly.

```
Dhost 'hostname' A Like the GetComputerName Windows API
ajaylap
'#' Dwi 'text' 'Ajay Askoolum' A Write to clipboard
'#' Dwi 'text' A Retrieve clipboard
Ajay Askoolum
p'#' Dwi 'printers'
4
p'#' Dwi 'fonts'
45
```

Re-inventing APL+

OK, feel free to read APL*PLUS – a quad function for everything. Traditionally, personal computer based APL interpreters have provided a self-sufficient development environment: APL system functions provide access to the operating system facilities.

The requirement of cross-platform portability coerces APLX back into this tradition; however, APLX can also deploy platform specific features such as Windows ActiveX objects.

Black Magic APL

Some aspects of APL work very mysteriously. These work fine with APLX, illustrating its compatibility with other APL interpreters.

```

a←10?1000
((2|a)/a)+0 ⍝ Replace odd numbers by 0
a
132 756 0 0 0 48 0 680 0 384

a←⍲10
11a ⍝ Sum of vector
55
01a ⍝ Last element of vector
10
a←10 2 3p⍲60
11a ⍝ Like +/[⍲io]a
280 290 300
310 320 330
01a ⍝ Like a[(1↑pa)~⍲io;:] BUT no need to know rank of argument
55 56 57
58 59 60

```

Other recent (I am not sure but APLX may well have had these features for a long time) improvements in APL work fine in APLX too.

```

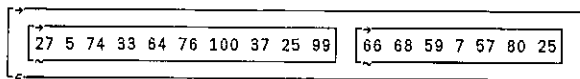
2++⍲10 | N-Wise
3 5 8 12 17 23 30 38 47 57

(2 3p⍲10)×[⍲io]2 3 ⍝ Axis operation
2 4 6
12 15 18
(2 3p⍲10)×[⍲io+1]2 3 4 ⍝ More Axis operation
2 6 12
8 15 24
a←(2?50) (5?100)
(c←⍲a)⍲a ⍝ Sort with thin quad (APL2 compatibility)
1 34 4 6 52 53 84
a
34 1 52 84 4 6 53

```

Consider two vectors created as follows:

```
a←10?100
b←7?90
ⓓdisplay a b
```



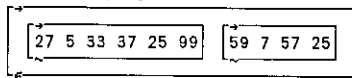
I raised this on the Vector Wiki pages recently. I am quite baffled as to the reason neither of the expressions shown below on the left work (with APL+Win, originally - and now with APLX).

These do not work!

```
(2|'a b')/'a b
DOMAIN ERROR
(2|'a b')/'a b
^
(2|'a b')/a b
DOMAIN ERROR
(2|'a b')/a b
^
```

It is baffling why this works!

```
ⓓZ←L Fn R
[1] a Operator
[2] Z←L/R
v 2004-01-30 11.36.58
ⓓdisplay (2|'a b') Fn 'a b'
```



Note the interpreter level support for ⓓdisplay.

File Functions

APLX version 2.0 adds the full complement of ⓓf and ⓓn functions with some useful improvements. Library numbers and fully qualified path names can be used interchangeably. Library numbers can be used with both types of files: therefore, applications can be based exclusively on library numbers. Practically speaking, the switch between a development and testing environment is achieved as easily as defining a library number. That is, a simple switch of library numbers points the application to one or the other of the two environments.

First, it is possible to use a tie number of 0 for both component and native files and for both create and tie operations: APLX allocates the next available tie number. This promotes sound code as it eliminates hard coded file numbers and the inadvertent references to the wrong numbers.

```
tie←'c:\temp\xx.sf' ⓓfcreate 0
tie
1
ⓓfuntie tie
```

```
tie←'c:\temp\xx.sf' Ⓚtie 0
tie
```

1

Like other APLs component file names do not need an extension: more accurately, any valid extension can be used for component file names. Native files can be tied with positive or negative integers – component files use positive integers only. APLX manages the sequence of native and component tie numbers independently: curiously, a given integer can be a tie number for both a native and a component file.

```
'c:\temp\nativefile.txt' Ⓚcreate 0
1 'c:\temp\compfile.apl' Ⓚcreate 0
1 'nativefile' Ⓚappend 1
  'componentfile' Ⓚappend 1 Ⓚ First component
  Ⓚunread 1 82 100 0
Insufficient data available
FILE I/O ERROR
  Ⓚunread 1 82 100 0
  ^
  Ⓚunread 1 82, (p'nativefile'), 0
nativefile
  Ⓚfread 1 1
componentfile
```

Note that the same integer, 1, is used concurrently as the handle for both a native and a component file. An important difference is that the native file cannot be read past the end of file without generating an error. The inability to read past the end of file may make the migration of an existing APL+Win application to APLX problematic – it depends on whether the code relies on the ability to read past the end of file.

Second, two new functions, Ⓚfdelete and Ⓚfwrite are provided for component files. The new Ⓚfdelete function allows a component to be deleted from anywhere in a file.

```
(110) Ⓚfappend 1 Ⓚ Second component
2 Ⓚfdelete 1 1 Ⓚ Deleting first component
  Ⓚfsize 1 Ⓚ Should have just one component
1 2 2560 0 256
```

Remaining components are automatically renumbered after Ⓚfdelete.

```
Ⓚfread 1,1
1 2 3 4 5 6 7 8 9 10
```

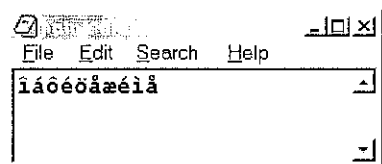
In other words, all hard coded references to component numbers must be changed. Components are also renumbered when `Ofwrite` is used to insert components.

```
(2x10) Ofwrite 1,0.5 # Insert before first
Ofread 1 1
2 4 6 8 10 12 14 16 18 20
Ofread 1,2
1 2 3 4 5 6 7 8 9 10
```

If the second element of the `Ofwrite` function is omitted, it works like the `Ofappend` function. If it is an integer, it works like the `Ofreplace` function: it must be a valid or existing component number. If it is a floating-point number, a component is inserted between `[SecondElement` and `# SecondElement` components.

The renumbering of components may present practical difficulties: it is easily forgotten that hard coded references to component numbers need to be changed. My wish list is twofold: allow components to be written/inserted/read by name rather than by ordinal position, and allow a component file to be sparsely populated – this can be achieved by writing `n` components past the next available component. Currently, any attempt to write past the next available component creates a run time error.

Third, the `Onwrite` function can be used to write Unicode characters to native files. Native files are not written using ASCII and rely on the conversion argument for correct representation. The `Uucs` function provides conversion from `Uav` to Unicode.



The second element of `Onread`, the conversion type, is crucial in reading from native files correctly. The help file documents the possible arguments. The `Onwrite` function takes the same arguments for writing to native files.

Fourth, the new functions `Onerror` and `Oferror` return the last error encountered and can be reported gracefully by an error handler routine. I wish APLX had thought of `OfisFileFree filename`: this could have returned 0 or 1 if the file were free and perhaps `-1` if the file did not exist. This would reduce the need for error handlers. Perhaps it will be available in the next version?

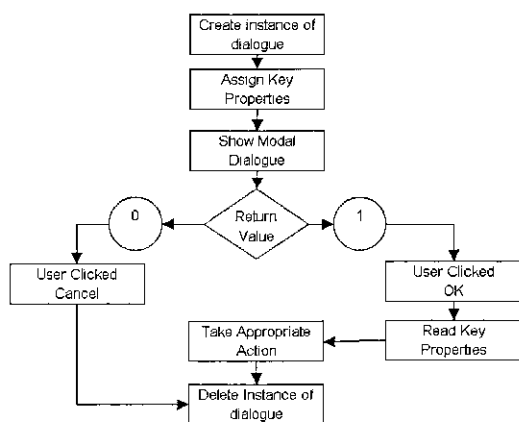
Standard Dialogues

The hallmark of any application boasting a Graphical User Interface (GUI) is the dialogues that allow the user to configure and manage their environment. On the Windows platform, the developer can use either the COMDLG.OCX object, subject to licensing requirements, or API calls. APLX provides native support for such utilities – that is, APLX offers standardisation and, from the developers' point of view, simplicity (This approach liberates the APL developer from platform constraints). This is a valuable and persuasive extension to APL programming: other platform-dependent APL interpreters have failed to provide these facilities, leaving the developer to grapple with platform APIs and data structures that do not exist in APL but must be created therein.

The table below shows the standard dialogues on the first row and the common set of properties in the first column: the grid shows ✓ where the property is available to the dialogue otherwise it is ✕. Key properties are shown in a box.

	ChooseColor	ChooseDir	ChooseFont	MsgBox	OpenFile	Printer
caption	✕	✓	✕	✓	✓	✓
children	✓	✓	✓	✓	✓	✓
class	✓	✓	✓	✓	✓	✓
color	✓	✕	✓	✕	✕	✓
copies	✕	✕	✕	✕	✕	✓
custom	✓	✕	✕	✕	✕	✕
data	✓	✓	✓	✓	✓	✓
default	✕	✕	✕	✓	✓	✕
directory	✕	✓	✕	✕	✕	✕
events	✓	✓	✓	✓	✓	✓
extent	✕	✕	✕	✕	✕	✓
file	✕	✕	✕	✕	✓	✕
filter	✕	✕	✕	✕	✓	✕
filterindex	✕	✕	✕	✕	✓	✕
font	✕	✕	✓	✕	✕	✓
fonts	✕	✕	✕	✕	✕	✓
handle	✕	✕	✕	✕	✕	✓
icon	✕	✕	✕	✓	✕	✕
margin	✕	✕	✕	✕	✕	✓
methods	✓	✓	✓	✓	✓	✓

	ChooseColor	ChooseDir	ChooseFont	MsgBox	OpenFile	Printer
name	✓	✓	✓	✓	✓	✓
opened	✓	✓	✓	✓	✓	✓
orientation	✗	✗	✗	✗	✗	✓
page	✗	✗	✗	✗	✗	✓
position	✗	✗	✗	✗	✗	✓
printername	✗	✗	✗	✗	✗	✓
printers	✗	✗	✗	✗	✗	✓
properties	✓	✓	✓	✓	✓	✓
range	✗	✗	✓	✗	✗	✗
scale	✗	✗	✓	✗	✗	✓
self	✓	✓	✓	✓	✓	✓
size	✗	✗	✗	✗	✗	✓
style	✓	✗	✓	✓	✓	✗
text	✗	✗	✗	✓	✗	✗
tie	✓	✓	✓	✓	✓	✓
units	✗	✗	✗	✗	✗	✓
winptr	✗	✗	✗	✗	✗	✓



The procedure for showing the built in dialogues is illustrated on the left. All the dialogues, except Printer, are made visible for user interaction using the Show method. The Printer dialogue is made visible using either the Job or Setup methods.

Interpreter level support for these dialogues provides a very neat and efficient way of eliciting user input and

```

vPrinter
[1] a Built in Printer dialogue
[2] 'pr' Dwi 'Create' 'Printer'
v 2004-02-08 11.29.13

```

avoids having to grapple with data structures when using API calls for the same purpose.

By using the Create instead of the New method, there is no need to delete a previous instance.

After running the Printer function, the available printers can be enumerated.

```

>'pr' Dwi 'printers'
PS/PDF on FILE:
HP DeskJet 970C Series on USB/DeskJet 970C/ES9921310ZJQ
HP LaserJet 4L on LPT1:

```

Similarly, the available printer fonts can be enumerated.

```

>'pr' Dwi 'fonts'
Courier
CG Times
Univers
..

```

The Print method of the Printer object prints RichEdit text without any fuss. **File + Print Window** from the Editor window prints functions as shown if a colour printer is used and colour printing is enabled.

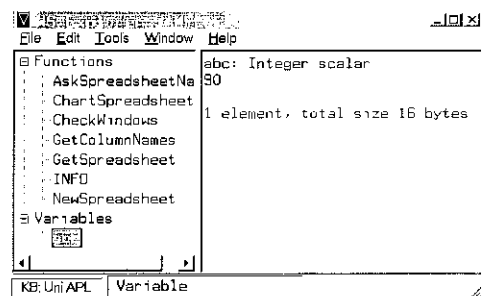
The help file provides working examples of each dialogue; this is a good place to start learning about APLX.

The APLX IDE

Somehow, it is difficult to talk about an Integrated Development Environment (IDE) if it does not offer IntelliSense as well. APLX does not offer IntelliSense. The editor is like any other APL editor but has some significant facilities.

- **Edit | Clean up indentation** (Ctrl + I) inserts a wave pattern indentation to functions. APLX has control structures, adapted from APL+Win: proper indentation is valuable in that it increases the readability of long functions.
- **Copy + Paste** operations support atomic vector or Unicode character sets.
- Functions can be exported to and imported from text files.

The usual facilities for search and replace etc. are available but simple numeric arrays cannot be edited. Like other editors, this one cannot edit nested variables either.



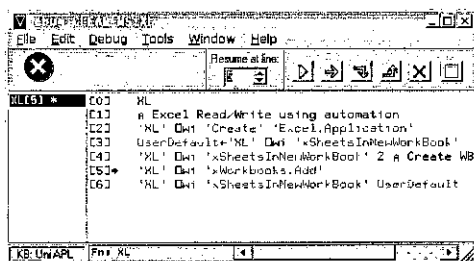
Tools | Windows Explorer displays the dialogue shown on the left. It shows all the functions and variables. Functions may be edited in the right pane: it would be difficult to edit variables, especially nested variables.

For the future, it would be very useful for the workspace explorer to show the dependency of functions graphically.

There is a debug option, with the usual facilities.

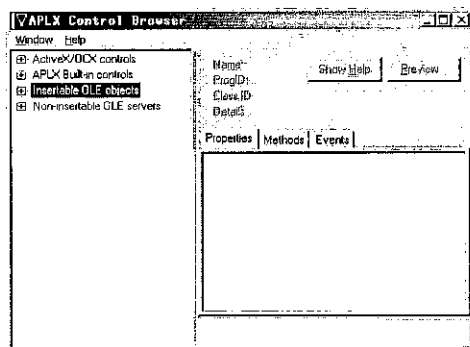
I still tend to use `⌈stop` and `⌈trace` for debugging. The ability to program function keys makes this easier.

(`⌈→01c'`, `⌈tnc1`) `⌈pfkey 12`



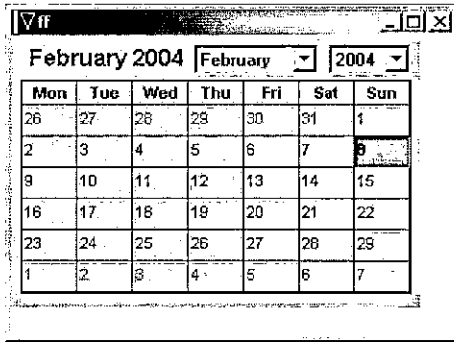
APLX has another dialogue that allows available GUI elements to be identified graphically.

Edit | Control Browser shows the dialogue to the right. The properties, methods and events of controls that can be put on APLX GUI forms can be examined. Note the Show Help and Preview buttons. Together, the controls of this dialogue accelerate the process of acquiring information on GUI elements that are available. A useful extension would be the ability to show the hierarchy of an active APLX form.



The process for deploying controls is quite simple and minimalist: controls must be children of APLX GUI forms or windows.

The example below shows the deployment of the calendar control.



```

vCalendar
[1]  R Using the Calendar Control
[2]  'ff' Qwi 'Create' 'Form'
[3]  'ff.cc' Qwi
      'Create' 'Calendar Control 9.0'
[4]  'ff' Qwi 'Show'
      v 2004-02-08 19.34.20
  
```

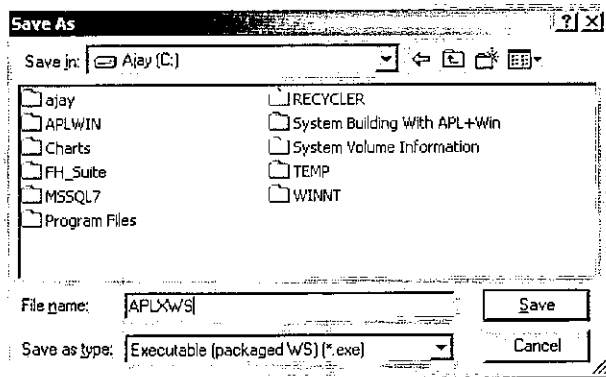
The **File | Print | Workspace Listing** option sends the functions within the current workspace to the printer. Although `Qvr` is missing in APLX, the listing is as though it were produced by `Qvr`.

The **File | Session History** option can clear the current session or save it to a file that can subsequently be retrieved. Essentially, the session window is a record of keyboard activity and system results in the active workspace. All other things being equal, should the workspace and session be saved, the state of play may be recreated on another computer using the two files.

The IDE does not have a GUI form designer or editor. Depending on your experience, this may or may not be a handicap. I have used the APL+Win designer: although useful sometimes, I can live without it.

Application Deployment

The distribution of the packaged exe requires APLXSupport.dll – subject to licensing requirements – and any used APLX fonts. On Windows, distribution of APLX applications could not be simpler! The EXE needs to be in the same location as the DLL – no registration is required.



File | SaveAs can package a workspace into an EXE file, ready for distribution: note the Save as type option. The workspace must have its `ⓘx` set or it will do nothing!

Language Enhancements

APLX has the `ⓘsv` and `ⓘna` function for defining external language functions. On the Windows platform, API calls can be defined using the `ⓘna` function. The FindWindow API call translates as follows:

```
Declare Function FindWindow Lib "user32" Alias "FindWindowA" (ByVal lpClassName As String, ByVal lpWindowName As String) As Long
    'FindWindow' ⓘna 'U USER32|FindWindowA <CT[256] <CT[256]'
    FindWindow 'TAPLSessionWin' ''
1114352
```

The class name of APLX is TAPLSessionWin. FindWindow returns the windows handle of a class or zero. Another example is the MessageBox API call.

```
Declare Function MessageBox Lib "user32" Alias "MessageBoxA" (ByVal hwnd As Long, ByVal lpText As String, ByVal lpCaption As String, ByVal wType As Long) As Long
```

It translates as follows:

```
'MessageBox' ⓘna 'U USER32|MessageBoxA U <CT[256] <CT[256] U'
```

The message to the right is shown by the following line:



```
MessageBox 0 'Message' 'APLX Caption' 16
```

The functions defined using `ⓘna` appear as normal locked functions and can be saved in the workspace.

```
ⓘnc ⚡'FindWindow' 'MessageBox'
3 3
```

The process of defining `ⓘna` functions for Windows APIs is rather tenuous: APLX does not devote a great deal of space to their definition. Perhaps this is to be expected from a cross platform development tool.

Another method permits true language extensions on the Windows platform. The Microsoft Script Control allows APLX to execute VBScript or JavaScript functions to be executed from the APL environment.

```
vSC
[1] 'sc' ⓓwi 'Create' 'MSScriptControl.ScriptControl'
[2] 'sc' ⓓwi 'xLanguage' 'JavaScript'
[3] 'sc' ⓓwi 'XExecuteStatement' 'var myString="Monday/Tuesday";'
v 2004-02-08 20.42.15
```

Using the formatted numeric LPDISPATCH handle, I can execute JavaScript functions.

```
xx←⚡'sc' ⓓwi 'XEval' 'myString.split("/")'
'sc' ⓓwi xx, '.properties'
x0 x1
'sc' ⓓwi xx, '.x0'
Monday
'sc' ⓓwi xx, '.x1'
Tuesday
```

APLX does not yet provide XML facilities: the developer has to write APL functions to produce XML trees. On Windows, the MSXML parser SDK can be used. This omission is surprising given that XML is currently fashionable, platform independent and APLX is a cross platform tool.

APLX and GUI Elements

Like APL+Win, the APLX GUI interface is based on `ⓓwi`; this creates a sense of familiarity for APL+Win users. However, there are important and significant differences; some of these, based on APL+Win version 4.0, are

- APLX can create a form using three classes, Form, Dialog, and Window; APL+Win has MDIForm and Form.
- APLX has OLE containers; APL+Win does not.

- APLX can resize its objects, parent and child, using its 4-element *anchor* property; APL+Win does not have this property.
- Both have a Grid and RichEdit controls. The APL+Win Grid object supports locale; the APLX Grid object does not. The size of both Grid objects exceeds the dimensions of an Excel worksheet and neither has methods for controlling pagination and printing. The APL+Win Grid can be saved as XML; APLX has no method for saving its Grid content.

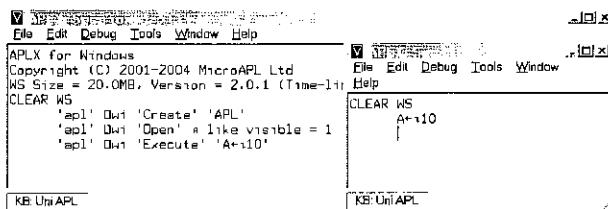
Similarly, the control structure implementations in the two interpreters are very close but there are important differences.

From a developer's point of view, it is difficult to understand the divergence of the GUI interface among APL interpreters. APL+Win, Dyalog, IBM and APLX have all diverged on GUI, control structures and language extension implementations while at the same time APL+Win, Dyalog and APLX have sought IBM core APL language compatibility. The scenario is reminiscent of SQL; in spite of an international standard, vendors have indulged in customised extensions in order to establish competitive advantage.

Multi-session APLX

A parent APLX session may spawn one or more child sessions; child sessions can be controlled individually. MicroAPL describes this as "APLX runs as a single *process*, with multiple *threads*. One thread controls the user-interface, and each APL task has its own thread." The initial APLX session is automatically the parent session: it can create another parent session using **File | New Session**. A child session is created as follows.

```
'apl' ⌐wi 'Create' 'APL'
'apl' ⌐wi 'Open' a Like visible = 1
```



At this point, two sessions exist. The parent session creates a variable in the child session.

Like other objects, the properties, methods, and events of the child session may be enumerated.

```
'apl' ⌐wi 'properties'
background children class data events extent methods name opened properties self size
```

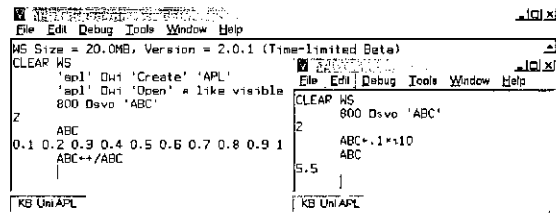
status tasked text tie visible where wssize
'apl' Dwi 'methods'
Close Create Delete Execute Hide New Open Send Set Show Signal Trigger
'apl' Dwi 'events'
onClose onDestroy onError onExecute onOpen onReady onSend onSignal

Unfortunately, the properties and methods do not include the variables and functions, respectively, of the child session. That is a pity.

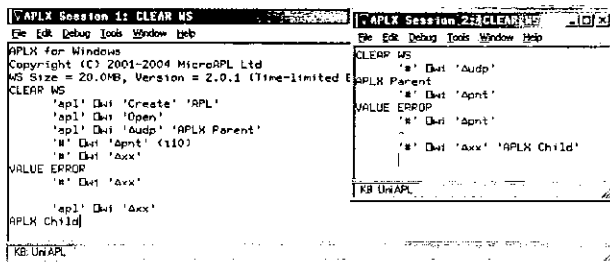
MicroAPL have chosen auxiliary processor 800 as the mechanism for sharing variable. On the plus side, using the Dsvo route, all sessions can share a set of variables.

The sessions can only share the variables via auxiliary processor 800.

As shown on the right, either session can query or change a shared variable.



An alternative method for exchanging data is via user defined properties: MicroAPL calls these delta properties.



An example of using delta properties is shown on the left.

Note the pitfalls too!

The delta properties of the system objects of either the parent or child are not shareable. Any reference to a non-existent property generates an error. This can be a nuisance especially since there is no convenient way to test for the presence of a delta property. The documentation explains the various methods for using delta properties.

Of the methods of child sessions, Execute provides the gateway to some very powerful facilities.

In the example on the right, the parent executes these commands.

```

A Load workspace
'apl' Dwi 'Execute' 'load Child'
A Run function
'apl' Dwi 'Execute' 'ChildAPLX'

```

The child session, shown on the right, confirms the loading of the workspace and the running of the function – listed manually in the child session.

```

APLX Session 2: Child
File Edit Debug Tools Window Help
CLEAR /S
)load Child
SAVED 2004-02-12 19.50.12
ChildAPLX
A
1 2 3 4 5 6 7 8 9 10
vChildAPLX[0]v
vChildAPLX
[1] A=110
v 2004-02-12 19.50.04
KB: UniAPL

```

The crucial difference with APLX's implementation is that the Execute method is asynchronous: control returns to the parent immediately. The parent may query the status property of the child in order to establish the state of the child. A status code of 1 indicates that the child is ready; the APLX documentation elicits all 6 codes. This means that an APLX application can create child tasks, and launch asynchronous tasks concurrently and monitor the status of the child tasks via events that they raise or their respective status property. Depending on system resources, a partitioned APLX application is likely to be very fast.

APLX and COM Automation

The Windows platform offers Component Object Model automation. The classes that are available can be listed using '#' Dwi 'xclasses'. This has the potential for speeding up application development in so far as some aspects can be readily off loaded to automation servers.

ActiveX Data Object (ADO) provides easy access to databases. This example illustrates how easily APLX can add a data tier: that is, use databases.

```

vAPLXwithADO:Cnn;Sql
[1] Cnn+ 'Provider=MSDASQL;Driver={Microsoft Access Driver (*.mdb)};DBQ=c:\nwind.mdb;'
[2] Sql+ "SELECT LASTNAME,FIRSTNAME,TITLE,BIRTHDATE FROM EMPLOYEES WHERE FIRSTNAME
LIKE 'M%';"
[3] 'RS' Dwi 'Create' 'ADODB.Recordset'
[4] 'RS' Dwi 'XOpen' Sql Cnn
v 2004-02-12 8.52.32

APLXwithADO
v 'RS' Dwi 'XGetRows' A Note the v
Peacock Margaret Sales Representative 13777
Suyama Michael Sales Representative 23194
'RS' Dwi 'XMoveFirst'

'RS' Dwi 'XGetString' A Note v is not necessary

```

Peacock Margaret Sales Representative 19/09/1937
 Suyama Michael Sales Representative 02/07/1963

The above are examples of how the record set object can be handled: they highlight two problems, first the need for `q` and second the inability of APL to represent dates. APLX is row major whereas the wider world seem to be column major – hence the need to transpose incoming data. APL does not have a date data type: incoming dates are integers representing *n* days from a reference date.

In practice, each record is processed in turn and transposition does not matter. Individual fields from the record set can be queried as follows:

```
'RS' Qwi 'XMoveFirst'
'RS' Qwi 'Fields().Value' 'FirstName'
Margaret
'RS' Qwi 'Fields().Value' 'LastName'
Peacock

'RS' Qwi 'XMoveNext' A Go to next record
'RS' Qwi 'XClose' A Close record set
'RS' Qwi 'Delete' A Tidy up
```

APLX and Automation Servers

APLX can use automation servers such as Word and Excel. The process is quite simple, as illustrated below using Excel.

- APLX can use Excel as a data source.
- APLX can use Excel to present data from the workspace.

```
vXL
[1] A Excel Read/Write using automation
[2] 'XL' Qwi 'Create' 'Excel.Application'
[3] UserDefault+'XL' Qwi 'xSheetsInNewWorkbook'
[4] 'XL' Qwi 'xSheetsInNewWorkbook' 2 A Create WB with 2 sheets
[5] 'XL' Qwi 'xWorkbooks.Add'
[6] 'XL' Qwi 'xSheetsInNewWorkbook' UserDefault
v 2004-02-12 10.52.37
```

After running this function, the Excel object is available for use.

```
A Insert 20.4 into A1
'XL' Qwi 'xWorkbooks().Sheets().Range().Value' 1 1 "A1" 20.4
A Read the value back
'XL' Qwi 'xWorkbooks().Sheets().Range().Value' 1 1 "A1"
20.4
A Read back the value - alternative syntax
'XL' Qwi 'xWorkbooks().Sheets().Cells().Value' 1 1 (1 1)
20.4
```

The latter syntax is highly suitable for APL. Consider the Excel object as a multi-dimensional array. The first dimension is the workbook. The second dimension is the sheet in the workbook. The third and fourth dimensions are the rows and columns in the worksheet. Thus APLX can use Excel using numbers to index this multi-dimensional array. This applies to scalar operations.

For array operations, it is still easier to think in terms of rows and columns rather than in terms of Excel ranges. Consider an APLX array.

```

      A
11718.59 67316.22 40860.74
47463.77 19506.89 4191.17
      pA
2 3

```

This will use 2 rows and 3 columns in Excel. Next, decide which row and column the first element of A will appear in. Say, this is row 5, column 15 of the worksheet. The address to be occupied by A can be calculated as follows.

```

      (€'R' 'C','¥'5 15 ),':',(€'R' 'C','¥'1+5 15+pA)
R5C15:R6C17

```

This produces an xlR1C1 address but automation requires xlA1 addresses: it requires conversion. The Excel formula is:

```

Application.ConvertFormula("R1C1:R10C5",xlR1C1,xlA1)

```

In APLX, this translates as follows.

```

Range←(€'R' 'C','¥'5 15 ),':',(€'R' 'C','¥'1+5 15+pA)
FromStyle←'XL' Dwi 'ValueOf' 'xlR1C1'
ToStyle←'XL' Dwi 'ValueOf' 'xlR1C1'
'XL' Dwi 'XConvertFormula' Range FromStyle ToStyle
Address←'XL' Dwi 'XConvertFormula' Range FromStyle ToStyle
Address
$0$5:$0$6

A Put values
'XL' Dwi 'xWorkbooks().Sheets().Range().Value' 1 1 Address A
A Get values
'XL' Dwi 'xWorkbooks().Sheets().Range().Value' 1 1 Address
11718.59 67316.22 40860.74
47463.77 19506.89 4191.17

```

Ranges that contain numeric data only transfer as simple arrays – missing values are filled with 0 – and arrays containing mixed data transfer as nested arrays. This subtlety may cause inadvertent run time problems: the content of a range cannot be predictable and therefore APLX code cannot anticipate the depth of values returned. This could have been avoided by using nested arrays all the time. APLX

provides a sample workspace to illustrate the basics of Excel automation: SAMPLEEXCEL.AWS.

Conclusion

APLX is appealing for several reasons.

- At roughly half the price of competing interpreters, APLX ought to be the outright choice for cross platform development. This advantage applies to platform-specific development too, especially to Windows, the largest platform: the developer has a tool chest of readily available APLX functions for interaction with the operating system.
- Having used and witnessed the evolution of other APL interpreters, I know that the rate of progress in full platform compliance can be rapid: APLX, at just version 2.0, already provides core cross platform compliance on key aspects and has thereby liberated the APL developer. For example, a single syntax is used for core dialogues, and for native and component file handling.
- Being the newest provider, having re-entered the APL market, I expect MicroAPL to be energetic in developing its offering. Its core APL, based on APL.68000, already has a pedigree that can be trusted.
- APLX can deliver any business logic tier, much as any other interpreter. Its GUI is capable of delivering a standard – as expected by users on each respective platform – presentation tier. As demonstrated, its ability to use ADO and ODBC, on the Windows platform, empowers it to deliver a data tier. I assume that corresponding technology can be deployed on the other platforms. Therefore, APLX is a tool for n-tier application design.
- APLX is a cross platform development tool: this implies that it needs to provide core extensions within the interpreter. This means that a single APL syntax needs to be learnt for such things as browser and XML functionality: surely, these must be high on MicroAPL's to-do list. Unicode support is already at the top of the list. Look out for version 2.0 and, hopefully, a series of quick releases offering these facilities!

For further details of APLX, trial versions, documentation etc., refer to:

<http://www.microapl.co.uk/APL/index.html>.

RECENT MEETINGS

In this issue, we cover the joint APL2000 and Dyalog User's Meetings at Naples Beach, Florida and the APL Germany autumn meeting in Ingolstadt.



Photographer's note

Anyone with an interest in projective geometry must (and I mean *must*) get themselves to Ingolstadt some day. The picture is one small corner of the most fantastic Trompe d'oeil ceiling I have ever seen. It all worked perfectly from the X just inside the west door, but several of the panels worked just as well (but giving a totally different landscape) from other points all around the body of the building. All quite remarkable, and for me it made the entire expedition well worth while.

Naples Beach 2004

APL2000 and Dyalog APL Conferences

notes by Adrian Smith

Background

This was meant to be the last time APL2000 came to the Naples Beach, but the weather was great and the company was good, so they may well be back again next year. I assume Dyalog will be returning to Europe next time around, but as a one-off, the back-to-back meetings worked quite well. Maybe more of the APL2000 users would have stuck around for the extra days if they had realised in time what was planned, but the Dyalog numbers were well up to expectations so in the end this wasn't a problem.

One of the great things about Florida is that all the beach is public land, so a morning walk (in my case) or run (Reima and Jouko) was always a welcome start to the day.



Beach with Running Finns

Hurricane Alex had made landfall about 50 miles north of here, where the destruction was clear to see. All Naples lost was about 50 yards of sand, and I guess the winter storms will rapidly pile it back. So if you can talk the company into paying your way back here next year, it will probably be your last chance.

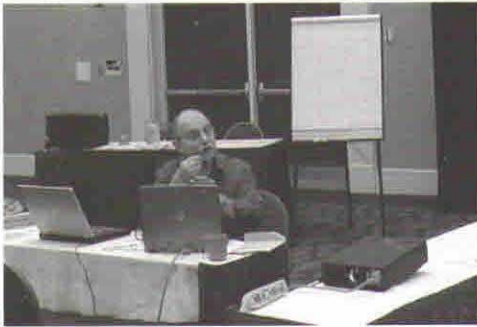
APL 2000: Conference Report

This is a hard one for me to report, as the emphasis was heavily on all the amazing things you can do with APL2000 WebServices (which I have never used, so know little about) and on the fun Fred and Jairo have been having with something called *Santa Fé* (which was remarkably hard to pin down). This leaves some really solid user-presentations and Bill Rutiser's slow and steady approach to 'objects' as the only things I am really able to report properly. If someone else wants to set the record straight, please do.

What is this thing called *Santa Fé*?

I *think* it is a lightweight array language wrapped on top of a C# library of APL-like primitive functions. The snippets of syntax we saw look more like VB.net than APL, and I have no real idea where its limitations are. I suspect it is a quick rework of the JScript engine into C# with an interface thrown over it so you can type expressions and assign variables. It is obviously not APL, and I don't believe anything except the most basic code snippets would be automatically translatable into it (see my notes on Joe Blaze's talk for the limitations of JScript). Is it the interpreter of the future, or it is just a quick Fred Waid lash-up, hastily polished as a conference demo? We will have to wait another year to know.

Larry Mysz and his Immunology Machine



I greatly enjoyed this talk, for all the wrong reasons! I suppose the point was to tell us about the APL content, but it really turned out to be an excellent immunology primer, and a nice summary of a typical system history. To summarize: testing blood samples for immune responses is a mindlessly tedious job involving matrices of sample plates; APLer hacks out a handy analysis tool

which talks down the serial cable to the machine that tests the responses; 10 years later the same APLer has a handy little business working with the machine manufacturers supplying 'embedded' software.

Maybe one day someone will rewrite the whole thing in a 'real' computer language, but in the meantime it ain't broke so why fix it?

Joe Blaze on Year-3 of APL web-based systems

To summarize Joe's talk from 2003: IIS errors cost BlazeSSI money. Up to 20% of the 10,000 hits per day were giving 404 responses, triggering clawback credit clauses from web-based clients. They can fix the response problem (and cut the number of servers) by handing off from IIS to an APL2000 WebService. This year's talk was a progress update, and a summary of the issues encountered.

First, some advice on an approach to the project:

- divide applications into pure UI and application logic. Some things absolutely have to be run server-side.
- hit the systems with the highest user-count first (10,000 users of a relatively small system). In Joe's case the analysis took around 2 months.
- give each 'core service' (such as arithmetic calculations) a workspace and a port number so any other service can call it.
- keep the user interface to a thin client (IE 5.5 or higher and Acrobat Reader 5).

Joe followed the approach of using ASP developed initially in JWI format and exporting it via JSave to make runnable forms. However he quickly found that you couldn't convert directly because of the JScript restrictions, so was back to line-by-line reworking of forms.

However, given the choice between writing APL that writes JScript (however flawed) and respecifying the application in JScript, he still preferred the option of sticking with the APL and having the programmers learn to code 'in the box'.

"You have to read every detail that Fred and Jairo tell you. Appreciate the restrictions of JScript".

The convertor works great for 'Fred' code, and the generated JScript assumes VisualStudio coding habits. Eventually the APL team learns the APL habits which make it work smoothly "... over time it has improved". Some other design advice:

- RunAtServer - don't build a large data array and send it all at once. Do it field by field with little packets of homogeneous data. This is then very reliable.
- just RunAtServer 'foo' arg returning a simple code which the JScript interprets and displays. Complex results add a large overhead in time (to wrap and unwrap the data) and in transmission cost.

- use overlaid frames with 'breadcrumbs' to show users where they are in the system. This deprives the user of the temptation to use 'Back' to revert to a prior step. Instead they have a clear 'Step1>>Step2>>This panel' chain to guide them through the system and make sure they back up in a controlled way.
- output formatting and content has to be consistent, so PDF is used everywhere to show completed forms. This also means there are no concerns about how the results will print. The server uses Causeway *NewLeaf* to generate the PDF and return it to be displayed directly in the browser (Acrobat Viewer works fine as a browser plug-in). This eliminated all printing hassles. Causeway helped by adding code to the NewLeaf PDF engine to take a collection of small reports and merge these up into a single consolidated document for the user to see in the browser.
- it is good to save common APL routines out to JScript as stand-alone .js files, which are referenced from the pages. This way they get cached by the browser, and the loading speed improves greatly after the first hit.

Joe's final advice was to include a 'Sunset' date to ensure obsolescence of old versions. When an application is split up and distributed in this way, it is crucial to have everyone using the same version of all the components.

Eric Lescasse on a pioneer WebServices application

Actually, this came before Joe's talk in the conference, but it makes much more sense to report it afterwards as it illustrated perfectly the perils of leaping into a big webservice conversion of a complex application.

Eric has been helping with the conversion of a questionnaire-analysis application for Renault, which must handle 90,000 responses to help analyse the effectiveness of training sessions. He had begun the project on the assumption that the JScript engine would be able to convert the vast majority of the existing code, but had quickly found himself on the phone to Fred or Jairo to help him discover ways around its limitations. Eric drew a decent veil over exactly what his over-run had been, but I think he had underestimated the work by at least a factor of 2, and for a careful professional like Eric, this is most unusual. Enough said.

Bill Rutiser on new core language features

Bill is a cautious engineer – he starts by looking at the technical implementation, and then adds new language features being careful all the way not to break existing code. This paper was a set of design ideas based around $\square$ MOM which is a proposed new system function to allow objects to be defined as recognised data types in the workspace. This is designed to be very non-specific, and will allow

entirely new language capabilities as well as access to external libraries such as the .Net framework.

“Remember, this is experimental work in progress. Everything is subject to change or abandonment.”

So you know where you stand with Bill! The main proposed addition to the interpreter is support for a new data-type, the *object reference*. This can be assigned to a variable, passed to a function, or returned like any other scalar value. It is possible to have several references to the same object (see *namespace references* in Dyalog) using ordinary assignment. The only structural primitive available to date is *match* – you cannot yet have arrays of references. References have no implicit output format (yet).

This leads to the obvious required syntax extension – the use of a dot to separate a reference from some content within it (again as for Dyalog APL and most object-based languages). The only way to create a reference (for now) is with `⊞MOM` which takes the specification of the object content, and returns a reference to it.

So far, the only object we have is a kind of package called a ‘sack’ which is really just a demonstration of the technology. You can put a bunch of names into a sack with:

```
myref←⊞MOM mynamelist
mylist←myref.⊞NL 3
```

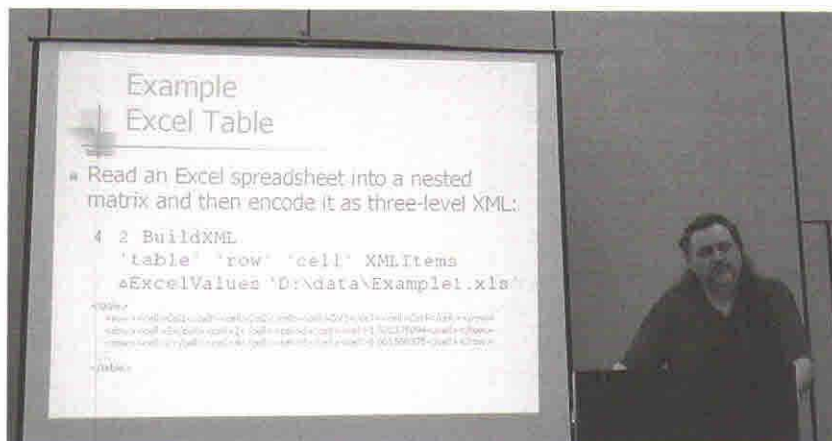
... and any of these internal names can use `⊞MSELF` to call other functions (or reference variables) in the same object.

That is about all it can do for now, but you can see the potential. Let me state a personal interest -- having finished re-implementing *RainPro* as a .Net library I would love to stop shipping 3 slightly different versions. If I could let all my +Win customers update to the .Net version, they would have a faster, lighter engine, no extraneous code in their workspace, and I could permanently archive one of the 3 sets of APL source. Wonderful.

Davin Church with some array-friendly XML tools

Davin is one of those useful guys who keeps generating well-structured utility libraries which he tests thoroughly and even takes the time to document. As XML gets to be the pervasive language of the internet, applications will increasingly need to use it for data exchange. This means that you must be able to take data and wrap it with valid XML tags (simple but mighty tedious), or alternatively take some valid XML from elsewhere and dig out the data from it.

Davin's paper deserves a full run in a future Vector (editor please note), but I was very impressed by his approach and I can already see where I might be able to use his toolkit.



I know there are Microsoft tools out there, but they are totally scalar oriented and hard to use effectively from APL. I looked at the .Net XMLFormatter object when we were first scoping the conversion of the RainPro SVG engine (SVG is a pure XML format) and in the end it was just going to pervert the application logic to the point where I would no longer recognise my own code. We have already defined an XML format for a RainPro 'chart specification' and I have a very basic toolkit which will read and interpret this to rebuild the data and call the appropriate functions. Davin's code may well be a lot solidier, and might just come in really useful here.

Summary (and sunset)



Yes, of course it was worth going, and no it wasn't just a nice chance to loaf on the beach. Meeting customers is always a necessity, and trying to get a feel for where the major vendors are headed is a crucial part of planning my own future.

On this last point, I have to report that I really don't know. You can see where Bill is headed, but whether objects will actually make it into the mainstream I don't know. Does anyone apart from Fred know where Fred and Jairo are headed? If so, I would love to hear.

Dyalog APL: Conference Report

My notes are limited to the first day, as I had already agreed to talk to the German APL group in Ingolstadt on the following Monday, which left me running for home on the Friday afternoon. Never mind, I picked up John Daintree on the latest enhancements to Dyalog 10.2 and John Scholes on "how to write computer programs" so there was enough to enjoy, even without the duck-in-the-mug being offered on the registration desk.



I suppose the Dyalog interpreter is a bit like my hard-drive. I don't think I have ever deleted anything except temporary 'gash' versions of things. The drive gradually fills up, and then I buy a new computer with a drive 10 times bigger. Invariably, this ends up with a folder called 'oldDdrive' containing the entire data-drive of the previous machine. Somewhere in here is a folder called oldDdrive, containing the entire data drive of the previous machine.

Somewhere in here and so on. So maybe Dyalog can just keep adding stuff, and the hardware will keep leaping ahead to the point where having a 3Mb interpreter is actually less overhead on your application than having a 620K interpreter (which is about the size of Dyalog 6.1, the first Windows build) was in 1993.

Anyway, I can hardly complain, as most of the new stuff is exactly what I felt was most needed to finish the job on object-orientation that they began last year.

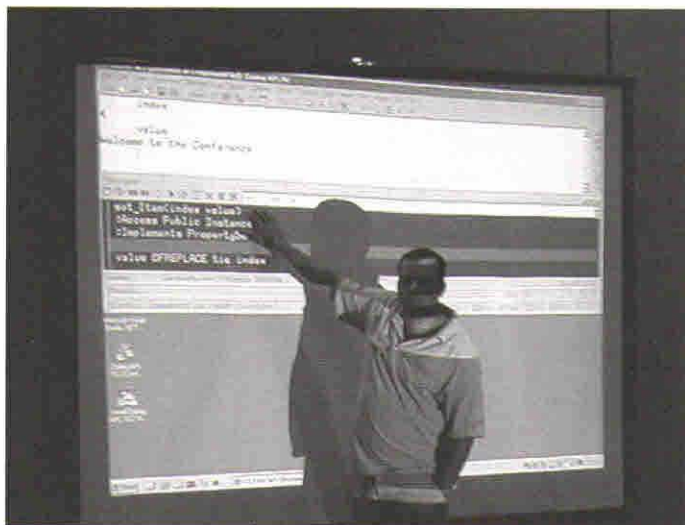
John Daintree on Dyalog 10.2

I don't know if this got the biggest cheer of the entire conference, but it certainly deserved it. John is a truly 'modern' programmer who actually lives and breathes this object-oriented C# stuff, so when he adds these capabilities into the interpreter, he does it how a C# programmer would do it.

Of course, you can fake your way around the limitations of namespaces with your own implementation of `new` (mine is so ugly I really can't face quoting it here) and lots of careful naming conventions and special comments. The problem with anything like this is that it is like `□FHOLD` – it only works if all the calling code knows the rules and abides by them. You can insist that the only functions available outside your namespace are the ones with `⌞Public` in the leading

comments, but if it is late on a Friday and you have a customer yelling for a fix, you will soon choose to leap the barrier and go calling things you really shouldn't.

Which is where John Daintree comes in ...



John Daintree with Dyalog 10.2 in action

What a lot of new control words we have today! Just in case you can't read the screenshot, the important bit is:

```
:Access Public Instance  
:Implements PropertySet Item
```

We have some new system functions too:

```
myduck+NEW Duck
```

At last! This creates a proper lightweight copy of a namespace, leaving the code where it was, and noting the 'Get' and 'Set' functions for properties. This allows the calling application to refer to properties as if they were variables:

```
myduck.species←'Mallard'
```

... but have the interpreter look up the appropriate *PropertySet* function and automatically call it, passing the assigned value as argument. The same applies in

reverse if you reference a property, and of course only functions declared to be public show up in the instance.

There is still some stuff for John to do here (autocomplete in the editor does not yet notice instances) but the basic idea looks completely sound, and gets APL a lot closer to the kind of modern environment that many of Dyalog's users are shouting for.

I think there was only one thing bothering me as I came away from the talk. In the current implementation, instances have the same nameclass as namespaces (being basically references with some stuff added). Surely, they are more special than that? If I have been testing stuff, I am likely to have a load of gash instances lying around in my workspace – it would be so nice to clean them up with a swift `⊖EX ⊖NL 11` or whatever. I can't see that you would ever want a saved workspace with any instances in it, apart from the usual `)continue lunch` scenario.

John Scholes on the art of Izzy programming

I hope John's deadpan humour works as well in Florida as it does back home – I think our US colleagues picked up the jokes and asides, but one can never be quite sure. This is another paper that deserves to be published in full, as much of the material was by no means as lightweight as John made out.

Basically, it was a plea for a more declarative approach to program design, typified by the phrase:

"Describe the result in terms of the arguments,
using only the present tense of the verb 'to be'."

So rather than telling the computer how to make the result, we just tell it what the result is. John gave the simple example of the definition of mean:

$$\{ (+/\omega) \div \rho\omega \}$$

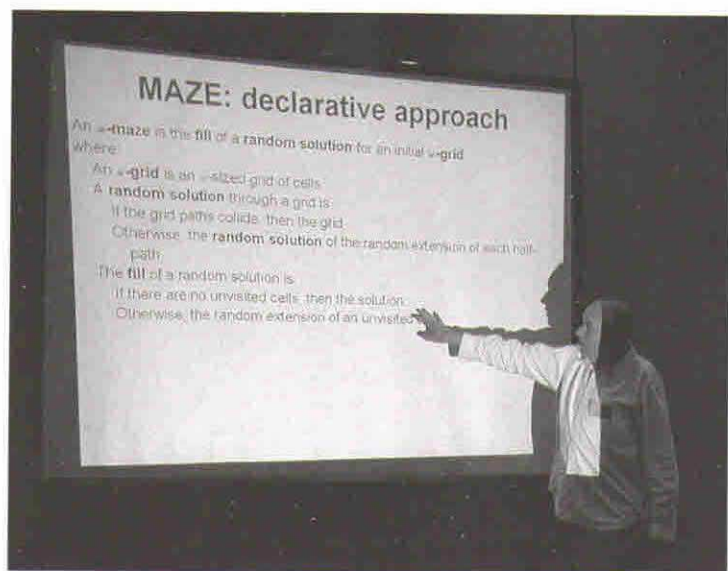
"The mean is the quotient of the sum and the number of items"

Which leads to the second nostrum ...

"You may use conditional if clauses and
supplementary where clauses to define terms"

In this example, the supplementary clauses are too obvious to need further elucidation, but in John's final example (a very procedural maze-solving algorithm) you quickly get to see that this really is a new way of thinking.

Here he is, with the algorithm specified in a pure declarative style. Suddenly, coding is the easy part.



John Scholes with an example of Izzy programming

And finally ...

How many of you knew that this works (and has worked for ages) ...

```
)ed ~ a    A a is a new character matrix
)ed € a    A a is a new vector of text vectors
)ed ∇ a    A a is a new function (default setting)
)ed → a    A a is a new character vector

)ed ° anyclass A will make a new class definition in 10.2
```

Maybe this is documented somewhere deep in the Dyalog User's Guide, but I never knew it before John Daintree's talk. I told you it was worth the air fare!

APL Germany in Ingolstadt

16th November 2004

reported by Adrian Smith

Getting There



Ingolstadt is a classic medieval town, still with many of its walls and gateways, nestling comfortably on the bank of the Danube, about an hour by train north of Munich. The Technical University is just outside this splendid 13th-century gateway, and the majority of the visitors stayed at a typically functional modern hotel a few streets away in the heart of the Altstadt.

Our host Prof Kuesters brought along several of his Economics students, who had been exposed only briefly to APL, but were keen and interested, and asked me plenty of searching questions. Several of the talks went clear over my head, and I am not entirely sure where the APL content lay in two of them, but I can report on Dave Liebttag's review of recent APL2 enhancements, and on

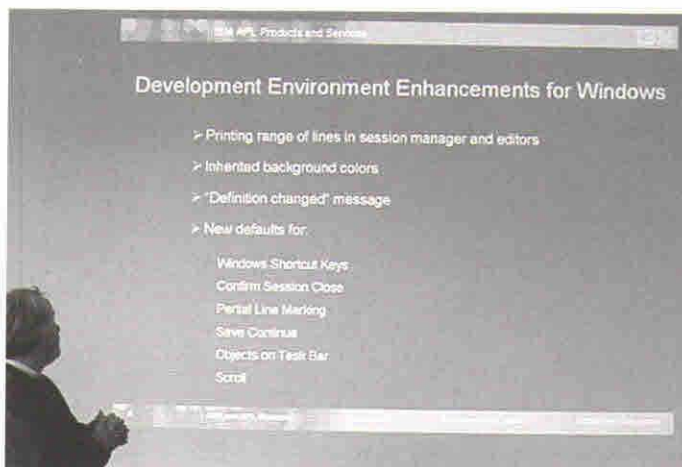
Romilly Cocking's work with BNP Paribas doing reconciliation work with APL2 and SmartArrays. My own talk was mostly a rehash of John Scholes' excellent summary of the art of Izzy programming, with a few asides on C# conversion thrown in. It probably went straight past the (mostly APL2) audience, but I was very encouraged at the keen interest I got from the students. I still think that if someone out there makes a lightweight, affordable D engine, it will get APL back into education faster than you can say 'Gimme Arrays'.

What's New in APL2

Dave Liebttag (IBM)

Dave was his usual avuncular self, and worked through a number of enhancements to the APL2 engine, mostly to do with interface and DB2 access. He very proudly showed us the PDF of an 80-page book "APL2 Programming with

Websphere" which he suggested was at least a 2-day read, and contains everything Dave knows about the Java interface.



As you can see, he has been quietly fixing up the APL2 Windows offering, with better Unicode support as well as lots of small interface improvements.

Using SmartArrays in a Banking Application

Romilly Cocking

Reconciliation of (say) 2 million records in your accounting system versus another 2 million entries on your bank statement takes a lot of processor time (typically 4 hours on an RS6000 at BNP Paribas). Romilly has been working on a flexible reconciliation engine in APL2, then compiling it down to C++ using Jim Brown's SmartArrays technology. The benefits are flexibility, reliability and (once compiled) performance.

I can certainly relate to the problem – even with the tiny number of transactions Causeway has to cope with, matching an entry on the bank statement (suppose we sell a GraPL for \$72 and a RainPro for €400 on the same day – the VISA people convert the currency and group the transactions) with the automated emails, is no fun at all.

So I enjoyed this talk, as well as meeting Romilly again for the first time in years. He also gives me hope that one day I might do something similar with our C# conversion kit, but that can wait for another day.

ERRATUM

VECTOR 20/4 p62: TABULA: A scientific calculator for the 21st century

I thought I knew the difference between gram-calories and kilogram-calories, but the scenario on how long a pound of fat would light a 100W bulb (converted at 100% efficiency) somehow managed to employ a bad definition of kcal in the constant definition tables. Once this is corrected the calculator shows the following figures:

1 second	170804.6 s
2 watt	100 W
3 Product of lines 1 2	4082.328 kcal
4 energy in fat	9 kcal/g
5 Dividend of lines 3 4	1 lb

170804.6s is approx: 47.5h, rather more than the published 170.8s of light-bulb burn.

Apologies to anyone who went out and bought an exercise bike and a generator on the strength of the published figures. You'll only have achieved a thousandth of the anticipated weight-loss. The method is guaranteed to yield significant results only when employed in conjunction with a calorie-reduced diet :-)

GENERAL ARTICLES

The Tower of Hanoi	Roger Hui	78
Further Surprises Involving a Man and a Goat The "Monty Hall" Problem Solved ...	Devon McCormick	85
Celtic Knotwork in J	Michael Horton	90
ACROSS and DOWN	Brian Schott	99

The Tower of Hanoi

by Roger K.W. Hui

The Problem

The Tower of Hanoi problem is to move a set of n different-sized disks from one peg to another, moving one disk at a time, using an intermediate peg if necessary. At all times no larger disk may sit on top of a smaller disk.

For example, moving 3 disks from peg 0 to peg 1 can be done as follows:

```
move disk 0 from peg 0 to peg 1
move disk 1 from peg 0 to peg 2
move disk 0 from peg 1 to peg 2
move disk 2 from peg 0 to peg 1
move disk 0 from peg 2 to peg 0
move disk 1 from peg 2 to peg 1
move disk 0 from peg 0 to peg 1
```

The description of the moves can be shortened if we observed that in moving a disk from peg A to peg B, it is always the top disk on peg A that is moved. Thus the 3-disk problem can be solved as follows:

```
0 1
0 2
1 2
0 1
2 0
2 1
0 1
```

Legend has it that a group of priests has been solving the 64-disk problem since the beginning of time, and when they finish, the world will come to an end.

An Initial Solution

There is a simple recursive solution to the n -disk problem: First, move disks 0 to $n-2$ from peg 0 to peg 2, then move disk $n-1$ from peg 0 to peg 1, then move disks 0 to $n-2$ from peg 2 to peg 1. If there is just one disk, the one disk can be moved from peg 0 to peg 1 straightaway.

This can be implemented as a dyadic verb: the right argument is the number of disks; the left argument are 3 integers of the pegs (*from, to, via*).

```
H=: 4 : 0
  if. 1>y. do.
    (y.,2)$x.
  else.
    ((0 2 1{x.} H y.-1), (2{x.}), ((2 1 0{x.} H y.-1)
  end.
)
```

```
0 1 2 H 3
0 1
0 2
1 2
0 1
2 0
2 1
0 1
```

From the definition of H , it is easy to see that the n -disk problem requires $<:2^n$ moves. If $n=1$, there is one move ($1=<:2^1$). If $n=k-1$ required $<:2^{k-1}$ moves, then $n=k$ requires the following numbers of moves:

```
<:2^k-1    ((0 2 1{x.} H y.-1)
1           (2{x.})
<:2^k-1    ((2 1 0{x.} H y.-1)
```

for a total of $<:2^k$ moves. A similar argument shows that the n -disk problem requires $<:2^n$ calls of the verb H .

```
0 1 2 #@H"1 0 i.10
0 1 3 7 15 31 63 127 255 511
```

```
<: 2 ^ i.10
0 1 3 7 15 31 63 127 255 511
```

Singly-Recursive Solutions

The two recursive steps in H differ only in the labelling of the pegs. Therefore we can replace them with a single recursive call, and do two relabellings of the pegs to get the effect of two recursions. That is verb $H1$ below.

Moreover, in $H1$ the left argument is unchanged in the recursion; that is, the left argument is always $0 1 2$, and can be elided.

We saw previously that on the n -disk problem the doubly recursive H required $<:2^n$ calls. The singly recursive $H1$ and $H2$ require n calls. Benchmarks on the 10-disk problem demonstrate the difference this makes.

```

H1=: 4 : 0
  if. 1>y. do.
    (y.,2)$0 1{x.
  else.
    ({&0 2 1 , 0 1"_ , {&2 1 0) x. H1 y.-1
  end.
)

H2=: 3 : 0
  if. 1>y. do.
    i.y.,2
  else.
    ({&0 2 1 , 0 1"_ , {&2 1 0) H2 y.-1
  end.
)

    0 1 2 (H -: H1)"1 0 i.10
1 1 1 1 1 1 1 1 1 1

    (0 1 2&H -: H2)"0 i.10
1 1 1 1 1 1 1 1 1 1

    ts=: 6!:2 , 7!:2@]    NB. time and space

    ts '0 1 2 H 10'
0.0496812 55360

    ts '0 1 2 H1 10'
0.000907099 69184

    ts 'H2 10'
0.000878045 68736

```

Non-Recursive Solutions

We now proceed to derive non-recursive solutions to the problem.

The results of verbs H , $H1$, and $H2$ are rows of the 6-row matrix A (the 6 different ways of moving from peg i to peg j where i and j can be 0, 1, or 2). Thus a more efficient representation for the solutions is to encode the moves as row indices of A , the integers from 0 to 5.

A=: 6 2 \$ 0 1 0 2 1 0 1 2 2 0 2 1

A
0 1
0 2
1 0
1 2
2 0
2 1

A i. H2 1
0

A i. H2 2
1 0 5

A i. H2 3
0 1 3 0 4 5 0

A i. H2 4
1 0 5 1 2 3 1 0 5 4 2 5 1 0 5

A i. H2 5

0 1 3 0 4 5 0 1 3 2 4 3 0 1 3 0 4 5 0 4 3 2 4 5 0 1 3

Judicious alignment of the indices reveals a pattern:

1: 0
2: 1 0 5

2: 1 0 5
3: 0 1 3 0 4 5 0

3: 0 1 3 0 4 5 0
4: 1 0 5 1 2 3 1 0 5 4 2 5 1 0 5

4: 1 0 5 1 2 3 1 0 5 4 2 5 1
5: 0 1 3 0 4 5 0 1 3 2 4 3 0 1 3 0 4 5 0 4 3 2 4 5 0 1 3

To get from A i. H2 $n-1$ to A i. H2 n , merge (intersperse) the result for $n-1$ with 1 5 2 if n is even and with 0 3 4 if n is odd. Thus:

```

H3=: 3 : 0
  if. 1>y. do.
    y.$0
  else.
    }: , ((2^y.-1)$ (2|y.)){1 5 2,:0 3 4) ,. 0,~H3 y.-1
  end.
)

```

```

      H3 3
0 1 3 0 4 5 0

```

```

      (A i. H2 3) -: H3 3
1

```

```

      (A&i.@H2 -: H3)"0 i.10
1 1 1 1 1 1 1 1 1 1

```

H3 n works by appending an atom to the result of H3 n-1; then stitching a list, repetitions of 1 5 2 or 0 3 4; then raveling; then dropping the last (previously appended) atom.

The list of numbers to be stitched (repetitions of 0 3 4 or 1 5 2) depends only on n, and on H3 n-1 not at all. This suggests a different method of directing the computation: Compute the lists x_i of numbers to be stitched for 1, 2, 3, ..., up to n, and then apply an appropriate verb f between them:

```

xn f      f x3 f x2 f x1
> f&.> / xn ;      ; x3 ; x2 ; x1

```

Moreover, we can avoid incorporating into f the appending and dropping of atoms, if we start out with an "extra" atom and drop it only at the end:

```

> f&.> / xn ;      ; x3 ; x2 ; x1
}: > g&.> / xn ;      ; x3 ; x2 ; x1 ; atom

```

In H4 below, the verb g is ,@,. (ravel stitch).

```

H4=: } : @ > @ ( ,@,.&.>/ ) @ ( < ,~ 2&^@i. $&.>&|. $&(0 3 4;1 5
2))

```

```

      (H3 -: H4)"0 i.10
1 1 1 1 1 1 1 1 1 1

```

```

arg=: < ,~ 2&^@i. $&.>&|. $&(0 3 4;1 5 2)

```

arg 1

0	1
---	---

arg 2

1	5	0	2
---	---	---	---

arg 3

0	3	4	0	1	5	0	3
---	---	---	---	---	---	---	---

arg 4

1	5	2	1	5	2	1	5	0	3	4	0	1	5	0	4
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

There is another possibility. The pattern in H4 is:

```
} : > g&.> / xn ; ... ; x3 ; x2 ; x1 ; atom
```

In an intermediate stage, when g is being applied,

```
xi g yi
```

g "knows" from yi alone what xi has to be: the length of yi determines the length of xi, and the leading atom of yi (0 or 1) determines whether 1 5 2 or 0 3 4 should be ravel-stitched. This suggests another solution: a monad is applied n times to the atom 1.

```
H5=: [: } : ( , @ , . ~ # $ { . { ( 1 5 2 , : 0 3 4 ) " _ ) ^ : ( ) ` 1 : )
```

```
(H4 -: H5)"0 i.10
1 1 1 1 1 1 1 1 1 1
```

```
rsa=: , @ , . ~ # $ { . { ( 1 5 2 , : 0 3 4 ) " _
```

```
rsa 1
0 1
```

```
rsa rsa 1
1 0 5 1
```

```
rsa rsa rsa 1
0 1 3 0 4 5 0 1
```

```
rsa^(3) 1
0 1 3 0 4 5 0 1
```

```
H5 3
0 1 3 0 4 5 0
```

Conclusion

The Tower of Hanoi problem can be solved in a variety of ways, with a wide variation in efficiency.

```
H    double recursion
H1   single recursion
H2   single recursion, monad
H3   single recursion, atomic representation
H4   non-recursive, insert
H5   non-recursive, power
```

```
t=. ' /0 1 2 H 10/0 1 2 H1 10/H2 10/H3 10/H4 10/H5 10'
Oj8 0 ": ts;._ 1 t
0.05121852 55488
0.00089900 69184
0.00087777 68864
0.00089564 40832
0.00036457 30720
```

The material in this article is available in the J 5.03 distribution as the "Tower of Hanoi" lab.

Further Surprises Involving a Man and a Goat

by Devon McCormick (devonmcc@yahoo.com)

The "Monty Hall" Problem Solved both by Simulation and by Bayesian Analysis

In a Parade magazine column by Marilyn Vos Savant¹, renowned by the Guinness² book of world records as the person with the highest IQ, was posed the following problem:

Suppose you're on the television program "Let's Make A Deal" and are asked to choose one of three doors for a prize. The prize behind two of the doors is a goat; the third door conceals a car. You choose one of the doors. However, instead of letting you know what you've won, Monty Hall opens one of the doors you didn't choose to reveal a goat. He then asks if you'd like to switch to the other, unopened door or remain with your initial selection.

The questions are: does it make any difference whether or not you switch? If it does make a difference, should you switch or not?

Ms. Vos Savant answered that it *does* make a difference and you should always switch; this assumes the host always opens a door to reveal a goat and always gives you the choice of switching. This column produced more mail than any she had ever written, much of it from mathematics professors, many of whom disagreed with her answer.

I know that when I read about this problem and its solution in a mathematical journal years ago, I didn't understand the solution given there and it seemed incorrect. However, it was but a matter of a few minutes (well, maybe 45) to write a simulation of the problem in APL. Here it is:

```
[0] CARS←N GOATDOOR C;[]IO;DOORS;PICK;GOAT;NG
[1] A Let's make a deal! We can choose 1 of 3 doors: 1 hides a car,
[2] A 2 hide goats. We pick one but Monty gives us a chance to re-choose
```

¹ Parade Magazine: September 9, 1990, February 17, 1991, and July 7, 1991.

² The Guinness Brewing Company has an important place in the history of statistics, having employed William Sealy Gosset to apply statistical methods of random sampling to their brewing process in order to brew a consistently good beer. Mr. Gosset discovered that staple of modern analysis, the (Student's) t-test.

```

[3] A after he opens one of the remaining 2 doors to reveal a goat.
[4] A How many cars do we get by
[5] A C: 0→keeping 1st choice, 1→changing to other door?
[6] A N: number of trials.
[7] QIO←1 ◊ CARS←0
[8]
[9] L_MAKEADEAL:DOORS←(?)#0 0 1      A Randomize choices: 1=car
[10] PICK←(?)#0 0 1                  A We pick a door: 1 is choice
[11] NG←0+. =GOAT←DOORS≠PICK        A Number of unpicked goat doors
[12] GOAT[(←GOAT)/1 2 3][?NG]]+NG#1 A Show a goat at random (=0)
[13] DOORS←GOAT/DOORS ◊ PICK←GOAT/PICK A Eliminate the shown door
[14] PICK←C≠PICK                    A Change pick or don't
[15] CARS←CARS+PICK/DOORS           A Add car if we got it
[16] →(×N←N-1)ρL_MAKEADEAL

```

After running this for half a million iterations – which took all night on my “powerful” 20 MHz 286 PC – I had an answer that matched the one given in the mathematical journal to 3 or 4 places.

My simulation was short enough and sufficiently simple that I was reasonably confident of its correctness. It was, however, a brute-force solution. Years later, I started learning about Bayesian methods in statistics and discovered a simple analytical technique for solving the same problem.

To see why Ms. Vos Savant was correct, at least for a common interpretation of the problem*, consider the version of Bayes’s Law which considers not just the probability of a single event but of the exhaustive series of events:

$$P(P_i|D) = \frac{P(P_i) \times P(D|P_i)}{(P(D|P_1) \times P(P_1)) + \dots (P(D|P_k) \times P(P_k))}$$

Or, in APL:

```

[0] R←BayesPP PROBS
[1] A Calc Bayesian posterior probability given PROBS: 2 row mat:
[2] A [0:] P(P1), P(P2)....; [1:] P(P1|D), P(P2|D)....; i.e. [0:] isolated
[3] A event probability, [1:] conditional probability given data D.
[4] R←R÷+/R÷×/PROBS

```

Our initial probabilities $P(P_i)$, for each of the three doors concealing a car, are $1/3$. This assumes a “flat” prior, that is, in the absence of any other knowledge, we assign each door an equal probability at the outset. That is, our initial probabilities are as follows:

$P(C) = 1/3$ probability of choosing the car

$P(G) = 2/3$ probability of choosing one of the 2 goats

Once Monty has opened one of the doors, we now have new data "D" to incorporate into our estimation. The probabilities are now

$$P(C|D) = 1/2 \quad \text{probability of choosing the car}$$

$$P(G|D) = 1/2 \quad \text{probability of choosing the goat}$$

because there are only 2 doors. Thus, our *posterior* probability of having chosen the car (arbitrarily calling our door #1) is

$$P(P_1|D) = \frac{1/3 \times 1/2}{(1/2 \times 1/3) + (1/2 \times 2/3)} = 1/3$$

That is, our posterior probabilities for choosing a door hiding a car and a goat, respectively, are:

$$\text{BayesPP } 2 \quad 2p(1 \quad 2/3), 1 \quad 1/2 \\ 0.33333333 \quad 0.66666667$$

Thus, we should always switch.

The following table illustrates why this is so.

Initial Choice	After Monty Opens Door Revealing Goat	
	Switch	Don't Switch
Car	Goat	Car
Goat 1	Car	Goat
Goat 2	Car	Goat

This enumerates all the possible combinations. As you can see, switching will more often give us a car than a goat.

* From The New York Times, July 21, 1991, Sunday, Late Edition – Final:

Her answer – that the contestant should switch doors – has been debated in the halls of the Central Intelligence Agency and the barracks of fighter pilots in the Persian Gulf. It has been analyzed by mathematicians at the Massachusetts Institute of Technology and computer programmers at Los Alamos National Laboratory in New Mexico. It has been tested in classes from second grade to graduate level at more than 1,000 schools across the country.

But it was not until Thursday afternoon that a truly realistic simulation of the problem was conducted. The experiment took place at the Beverly Hills home of Monty Hall, the host of 4,500 programs of "Let's Make A Deal" from 1963 to 1990. In his dining room Mr. Hall put three miniature cardboard doors on a table and represented the car with an ignition key. The goats were played by a package of raisins and a roll of Life Savers.

After Mr. Hall allowed this contestant 30 independent attempts to win the car, two conclusions seemed clear:

Ms. vos Savant's vitriolic critics, including the mathematics professors, are dead wrong. But Ms. vos Savant is not entirely correct either, because there is a small flaw in her wording of the problem that was detected not only by Mr. Hall but also by some of the experts. Despite her impressive analysis and 228-point I.Q., she was not as quick as Mr. Hall in understanding the psychological dimensions of the problem.

...

After the 20 trials at the dining room table, the problem also captured Mr. Hall's imagination. He picked up a copy of Ms. vos Savant's original column, read it carefully, saw a loophole and then suggested more trials.

On the first, the contestant picked Door 1.

"That's too bad," Mr. Hall said, opening Door 1. "You've won a goat."

"But you didn't open another door yet or give me a chance to switch."

"Where does it say I have to let you switch every time? I'm the master of the show. Here, try it again."

On the second trial, the contestant again picked Door 1. Mr. Hall opened Door 3, revealing a goat. The contestant was about to switch to Door 2 when Mr. Hall pulled out a roll of bills.

"You're sure you want Door No. 2?" he asked. "Before I show you what's behind that door, I will give you \$3,000 in cash not to switch to it."

"I'll switch to it."

"Three thousand dollars," Mr. Hall repeated, shifting into his famous cadence. "Cash. Cash money. It could be a car, but it could be a goat. Four thousand."

"I'll try the door."

"Forty-five hundred. Forty-seven. Forty-eight. My last offer: Five thousand dollars."

"Let's open the door."

"You just ended up with a goat," he said, opening the door.

The Problem With the Problem

Mr. Hall continued: "Now do you see what happened there? The higher I got, the more you thought the car was behind Door 2. I wanted to con you into switching there, because I knew the car was behind 1. That's the kind of thing I can do when I'm in

control of the game. You may think you have probability going for you when you follow the answer in her column, but there's the psychological factor to consider."

He proceeded to prove his case by winning the next eight rounds. Whenever the contestant began with the wrong door, Mr. Hall promptly opened it and awarded the goat; whenever the contestant started out with the right door, Mr. Hall allowed him to switch doors and get another goat. The only way to win a car would have been to disregard Ms. vos Savant's advice and stick with the original door.

Was Mr. Hall cheating? Not according to the rules of the show, because he did have the option of not offering the switch, and he usually did not offer it.

And although Mr. Hall might have been violating the spirit of Ms. vos Savant's problem, he was not violating its letter. Dr. Diaconis and Mr. Gardner both noticed the same loophole when they compared Ms. vos Savant's wording of the problem with the versions they had analyzed in their articles.

"The problem is not well-formed," Mr. Gardner said, "unless it makes clear that the host must always open an empty door and offer the switch. Otherwise, if the host is malevolent, he may open another door only when it's to his advantage to let the player switch, and the probability of being right by switching could be as low as zero." Mr. Gardner said the ambiguity could be eliminated if the host promised ahead of time to open another door and then offer a switch.

Ms. vos Savant acknowledged that the ambiguity did exist in her original statement. She said it was a minor assumption that should have been made obvious by her subsequent analyses, and that did not excuse her professorial critics "I wouldn't have minded if they had raised that objection," she said Friday, "because it would mean they really understood the problem. But they never got beyond their first mistaken impression. That's what dismayed me."

Still, because of the ambiguity in the wording, it is impossible to solve the problem as stated through mathematical reasoning. "The strict argument," Dr. Diaconis said, "would be that the question cannot be answered without knowing the motivation of the host."

Which means, of course, that the only person who can answer this version of the Monty Hall Problem is Monty Hall himself. Here is what should be the last word on the subject:

"If the host is required to open a door all the time and offer you a switch, then you should take the switch," he said. "But if he has the choice whether to allow a switch or not, beware. Caveat emptor. It all depends on his mood.

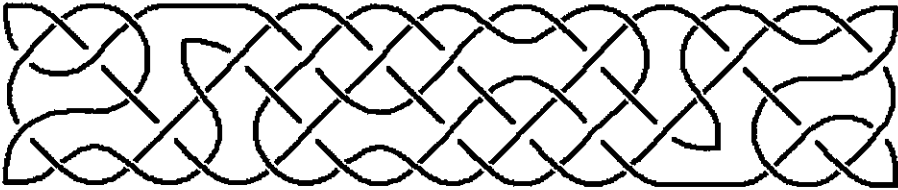
"My only advice is, if you can get me to offer you \$5,000 not to open the door, take the money and go home."

Celtic Knotwork in J

by Michael Horton (mhorton@utas.edu.au)

Introduction

Celtic knots are a simple but very decorative form of geometric art made to give the appearance of a thick line twisting around and crossing itself.

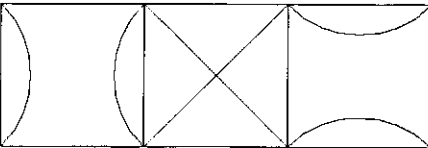


A systematic method of drawing knots using square grid was found by Andy Sloss [1]. A J code adaption of this method follows. It generates a graphical knot from a grid of pseudorandom integers. The picture is one example.

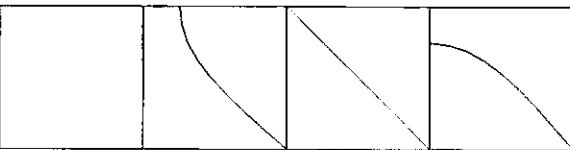
The Squares

Each square in the knot can be defined by five properties: how lines enter from each of the four corners, and how the lines cross the square. The J program encodes these in a list of five integers.

Crossings may be 0 (vertical), 1 (diagonal) or 2 (horizontal).

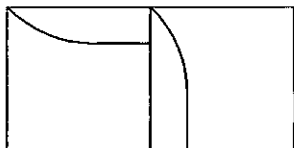


Corner entries may be 0 (no line), 1 (vertical), 2 (diagonal) or 3 (horizontal). Horizontal and vertical entries are one quarter of the side.

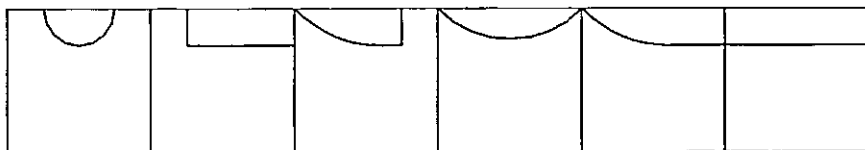


Line Segments

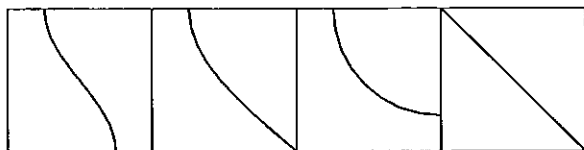
Since segments may start in one of 4 corners, enter 3 ways (not including lines that don't exist), cross in 3 ways and leave in 3 ways, there appear to be 324 possible segments. However, many of them are rotations and reflections of each other. For example, a segment entering diagonally, crossing horizontally and leaving horizontally is a rotation and reflection of a segment entering diagonally, crossing vertically and leaving vertically.



In total, there are 10 distinct sections of lines crossing squares. These can be divided into 6 segments crossing horizontally (rotated to get lines crossing vertically) and 4 segments crossing diagonally.



Horizontal segments (h11, h13, h21, h22, h23 and h33)



Diagonal segments (d11, d12, d13 and d22)

For best aesthetics, the knotwork should mix curved and straight lines. This is why h13 and h21 use sharp angles while the other segments use curves.

Once the curves are constructed, they can be rotated and reflected to create all possible lines, and combined to create all possible squares. Geometry was generated with complex coordinates, as these are much easier to manipulate. In the end, the complex geometry needs to go into 'gdlines', which takes lists of real numbers.

The 'graph' script is needed for drawing and the 'trig' script for the curves.

```
Load 'graph trig'
```

```
outerLine:=gdpen@2:,WHITE&(gdcolor@[]
innerLine:=gdpen@1:,BLACK&(gdcolor@[]
drawComplex:=$@0:`gdlines@:,@:+.@.(*@#)
knotComplex:=(drawComplex@,innerLine),(drawComplex@.@):@,outerLine)
```

The drawComplex function will only call gdlines if it receives at least one number - some parts of the knot generate no geometry at all. The knotComplex function draws a thick white line followed by a thin black line. This deals with overlapping lines in the middle of a square.

The next problem is to generate the individual line segments. Each of the 10 basic line types is created once. They are then rotated and flipped to create the knot components.

NB. Geometry modification functions

```
flipVert:=+
flipHoriz:=+@-
rotLeft:=0j1&*
flipBoth:=--
```

NB. Horizontal line geometry, top left - top right

```
h11:=(0j1-0.5*(cos j. sin) 0.8%~i.9) "_
h12:=flipHoriz@h21
h13:=_0.5j1 _0.5j0.5 1j0.5 "_
```

```
h21:=(0.5j1,0.5j0.5,((0.5--%:2) j. 0.5)+-(1+_1%:2)%~0j1-(sin
j. cos) 0.32%~i.9) "_
h22:=(0j2-(%:2)*(sin j. cos) 0.16%~i.4) "_
h23:=(1j0.5,((0.5--%:2) j. 0.5)+-(1+_1%:2)%~0j1-(sin j. cos)
0.32%~i.9) "_
```

```
h31:=flipHoriz@h13
h32:=flipHoriz@h23
h33:=_1j0.5 0j0.5 1j0.5 "_
```

NB. Diagonal line geometry, top left - bottom right

```
d11:=((-:@sin@o.@%&16 j. %&8) "0 i:8) "_
d12:=((-@<:@(3r2&*@cos@-:@o.j.<:@+:@)) 8%~i.9) "_
d13:=(1j1-1.5*(cos j. sin) 0.16%~i.9) "_
```

```
d21=:flipBoth@d12
d22=:_1j1 _0.75j0.75 _0.5j0.5 0.5j_0.5 0.75j_0.75 1j_1 "_
d23=:flipHoriz@rotLeft@d12
```

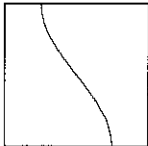
```
d31=:flipBoth@d13
d32=:flipBoth@d23
d33=:flipVert@rotLeft@d11
```

The overlapping segment code requires that every diagonal segment have internal coordinates, which is why the 'd22' function returns a straight line with 5 points in it.

These functions can be tested with commands like

```
>(gdshow ''), (knotComplex d11 0), (gdopen '')
```

Which fills the graph window with the appropriate segment.



Choosing Segments

Segments are selected from a gerund of all the possible line drawing functions. The 'cut' versions of the lists have diagonal endpoints removed to allow line overlaps at the corners.

The topSeg, bottomSeg, leftSeg, rightSeg, sloshSeg and slashSeg functions take two input numbers – the entry and exit directions – and return the appropriate geometry function. Any segment with a zero at either end returns the function 0: . This is the easiest way to generate no geometry for a segment.

```
hList=:0:~0:~0:~0:~0:~0:~h11`h12`h13`0:~0:~h21`h22`h23`0:~0:~h31`
h32`h33`0:~0:~0:~0:~0:~0:~
hListCut1=:0:~0:~0:~0:~0:~0:~h11`}:@}:@h12`h13`0:~0:~h21`},@h22`h
23`0:~0:~h31`}:@}:@h32`h33`0:~0:~0:~0:~0:~
hListCut2=:0:~0:~0:~0:~0:~0:~h11`h12`h13`0:~0:~}:@}:@h21`}:@h22`}
:@}:@h23`0:~0:~h31`h32`h33`0:~0:~0:~0:~0:~

dList=:0:~0:~0:~0:~0:~0:~d11`d12`d13`0:~0:~d21`d22`d23`0:~0:~d31`
d32`d33`0:~0:~0:~0:~0:~0:~
```

```
dListCut=:0:~0:~0:~0:~0:~0:~d11'}.@d12'd13'0:~0:~'.@d21'}.@.:@d22  
~'.@d23'0:~0:~d31'}.@d32'd33'0:~0:~0:~0:~0:  
seqIndex=:(5 5)&#. @]
```

```
topSeg=hListCut2@.segIndex
bottomSeg=:flipVert@hListCut1@.segIndex
leftSeg=:rotLeft@hListCut1@.segIndex@(4:-])
rightSeg=:rotLeft@flipVert@hListCut2@.segIndex@(4:-])
```

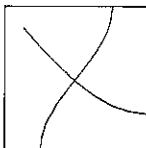
```
sloshSeg:=dListCut@.segIndex
slashSeq:=rotLeft@dList@.segIndex@(4:-1)
```

To build the geometry for a square, call the `selectAllGeometry` function. This takes a list of 5 integers: the square crossing type followed by the top left, top right, bottom left and bottom right corners. It separates out the appropriate corners and passes them to the segment functions.

```
selectAllGeometry=((leftSeg@{3 1&});rightSeg@{4
2&})`(sloshSeg@{1 4&};slashSeg@{3 2&})`(topSeg@{1
2&};bottomSeg@{3 4&}))@.}.
```

The output is two boxed lists, which must be unboxed before sending to `knotComplex`.

```
>(gdshow ''),((knotComplex@>@{.,knotComplex@>@{:})
selectAllGeometry 1 2 1 1 3),(qdopen '')
```



Putting the Squares Together

Since the squares fill the graph window, multiple squares must be moved into place and scaled down. Each corner must be used correctly by the four surrounding squares if the geometry is to match up.

```
getCornerIndices:=<@({0 1}&+),<@({1 1}&+),<@({0 0}&+),<@({1  
0}&+)  
getCombinedList:=([<@[{]}>@{. @}]),([<(getCornerIndices@[{])>@{. @}  
1])
```

```

getSquareGeometry=:selectAllGeometry@{)getCombinedList[)
drawPositioned=:knotComplex@(({.%[%~{.%}.@[-+:@{:@[+j1&+>@]))
drawSquare=:(>./@>@{.%[.j./@>@{.%[.j./@])
(([[drawPositioned{.%]),([drawPositioned{.%})) getSquareGeometry

```

The input to `getCombinedList` is a boxed list of crossovers and corners as its right argument, and the coordinates within them which it is to look up as its left argument. It calls `getCornerIndices` to turn its coordinates into the four corner coordinates within the corners list. The output is the list of five integers that describes the requested square.

The `drawPositioned` function takes a list of three numbers as its left hand argument:

- [1] **Scaling factor:** all coordinates are divided by this amount. It's the greater of the side lengths of the input rectangle.
- [2] **Complex size:** a complex form of the side lengths of the input rectangle. This is subtracted from the coordinates to centre the knot.
- [3] **Square coordinates:** where, in the original rectangle coordinates this segment will be drawn.

The right hand argument of `drawSquare` is the complex list of geometry to be manipulated.

Input for `drawPositioned` is generated by `drawSquare`. The left argument of `drawSquare` should be a boxed list, with the first item being the crossovers and the second being the corners. The right argument should be the coordinates of the square to draw.

```

allIndices=:({.(|,.<.%~)i.@*/)@>@{.
drawKnots=:]drawSquare"1 allIndices
showKnots=:gdshow,drawKnots,gdopen@$@0:

```

To draw all the squares at once, a list of all the square coordinates is needed. This is returned by `allIndices`, which reads what it needs out of the crossovers list. This is used by `drawKnots` to draw every square.

Since a graphics window must be open, the `showKnots` function provides a simple way to open a window, draw a knot and show it. All that's missing is some boxed knot information to feed into it.

Generating the Tables

The simplest way to create the knotwork tables is with random numbers. This is easy, but doesn't generate very nice knots.

```
>corners=:>: ?. 11 5 $ 3
>crossovers=:?. 10 4 $ 3
>showKnots crossovers;corners
```



Grids with rotational symmetry work a lot better. The easiest way to generate them is to generate a section, flip it over and copy it. One of the grids should only have its central row used once. In this case, the 'corners' array is chosen; its central row is replaced with twos, for diagonal crossing.

```
flipAndDuplicate=: (|. "1@|. )
flipAndDuplicateRemove=: (|. "1@|. @):)

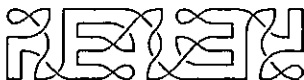
>crossovers=:flipAndDuplicate ?. 6 3 $ 3
>corners=:flipAndDuplicateRemove (>: ?. 6 4 $ 3),2
>showKnots crossovers;corners
```



This is better, but the edges are untidy. These can be corrected by making two changes:

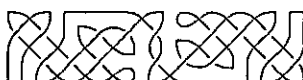
- [1] The crossings along the top and bottom must be horizontal and the crossings to the left and right must be vertical.
- [2] Either the top and bottom or the left and right corners must be zero, indicating no line segment at all.

```
>crossovers=:flipAndDuplicate 0,(?. 5 3 $ 3)
>corners=:flipAndDuplicateRemove (0,(3,.(>: ?. 5 2 $ 3),2),.3)
>showKnots crossovers;corners
```



The knots generally look better if they contain more crossover segments. This can be achieved by taking elements from a list that favours crossovers.

```
>crossovers=:flipAndDuplicate 0,(?. 5 3 $ 5) { 0 1 1 1 2
>corners=:flipAndDuplicateRemove 0,(3,.(?. 5 2 $ 5) { 1 2 2 2
3),2),.3)
>showKnots crossovers;corners
```



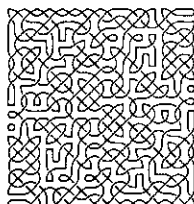
The entertaining part of Celtic knotwork is creating new designs, so the next step is to change from the ? to the ? pseudorandom function.

```
>crossovers=:flipAndDuplicate 0,(? 5 3 $ 5) { 0 1 1 1 2
>corners=:flipAndDuplicateRemove 0,(3,.(? 5 2 $ 5) { 1 2 2 2
3),2),.3)
>showKnots crossovers;corners
```



It is also possible to request a knot of any size, although the overlapping lines will generate gaps on large knots. Always make sure that the corners table is one unit larger in both dimensions than the crossovers table.

```
>crossovers=:flipAndDuplicate 0,(? 7 16 $ 5) { 0 1 1 1 2
>corners=:flipAndDuplicateRemove 0,(3,.(? 7 15 $ 5) { 1 2 2 2
3),2),.3)
>showKnots crossovers;corners
```



Another option is to generate repeating borders.

```
>crossovers=:([,],,],) (? 4 4 $ 5) { 0 1 1 1 2)
>corners=:([,],,],,.) {3,.(? 4 3 $ 5) { 1 2 2 2 3),.3))
>showKnots crossovers;corners
```



Programmers are welcome to invent their own crossover and corner tables.

Discussion

My first attempt at drawing Celtic knots used pairs of integers for geometry and was terribly cumbersome. The complex number coordinates made every operation much simpler.

Graph theorists may wish to see if they can rewrite the construction functions to guarantee a single continuous loop. Small independent circles are not a common part of traditional Celtic knots; this program generates them with regrettable frequency. An alternative would be to create user interface code so the knot can be adjusted after it is drawn.

Reference

- [1] A Sloss, *How to Draw Celtic Knotwork: A Practical Handbook*, Blandford Press, 1997
- [2] Wilson, Anne, *Over and Under: Celtic Knotwork Patterns*, Vector Vol 17.4, p.101

ACROSS and DOWN

by Brian Schott

All of us have pondered automating the craft of authoring crossword puzzles. My daydreaming drifted toward a grid of select words — words of a hobby, locale, or clan, for example. Using the short scripts developed so far, I have produced word crossings for birthday party attendees, contra dance lovers, common sayings, and J primitives; to me its all for fun.

`across` and `down` are my workhorse verbs. The dyadic forms of `across` and `down` take as a left argument the incumbent grid of words crossed and its key features (boxed in three parts) and as a right argument a new candidate word. `across` and `down` return a grid of words crossed and its key features (boxed in three parts). If a place has been found for the new candidate word in the incumbent grid, then the output result is revised appropriately; if not, the output result is unchanged from the input left argument. In either case, the output result can be used as a left argument for a subsequent `across` or “down.” Typically, in my usage of `across` and `down` the two verbs are applied alternately with a new word or an unsuccessful word from a list of words I have created.

The monadic forms of the verbs `across` and `down` were created to seed a word grid. For example, see the excerpted session below exercising monadic `down`.

```

      down 'contra'
+---+-----+
| 3 1 | 3 3 1 1 | % |
|     | 1 6 1 1 | c |
|     |         | o |
|     |         | n |
|     |         | t |
|     |         | r |
|     |         | a |
|     |         | % |
+---+-----+

```

Boxed output/input of `across` and `down`

The three boxed results of this example contain the following information. The third box contains an array, named `AD`, of shape `8 3`. The `%` character — called `Dot` in my script — was arbitrarily chosen to demarcate the beginning and end of each word added to the grid. In addition to being place-holders, `Dots` are treated as special characters which enable the program code to prohibit any words from

touching. (After a final grid is achieved I have typically replaced all Dots with spaces, but other alternatives are possible.)

The first box contains the row-column indices of the center of the grid, named **Center0** in the code (all indexing is 0-origin). In this case the index 3 1 corresponds to the row-column position of the letter "n" in the grid.

The centre box contains AD grid indices; the 4 columns correspond to NSWE (north-most, south-most, west-most, east-most) and the 2 rows correspond to the location in the grid of the across words (first row) and the down words (second row) which are thought to frame the current grid. The indices are given the following 8 names in the script.

```

              NSWE
      +---+---+---+
across |Na|Sa|Wa|Ea|
      +---+---+---+
down   |Nd|Sd|Wd|Ed|
      +---+---+---+

```

NB.	north-most letter of a horizontal word	(Na)
NB.	south-most letter of a horizontal word	(Sa)
NB.	west-most letter of a horizontal word	(Wa)
NB.	east-most letter of a horizontal word	(Ea)
NB.	north-most letter of a vertical word	(Nd)
NB.	south-most letter of a vertical word	(Sd)
NB.	west-most letter of a vertical word	(Wd)
NB.	east-most letter of a vertical word	(Ed)

In the example Nd is 1 and Sd is 6 because the letters *c* and *a* are the north-most and south-most letters of the (only) down word, *contra*. Wd and Ed are both 1 because the word *contra* is in column 1, making the west- and east-most letters in a down word 1. Na and Sa are both 3 and are somewhat arbitrarily set, because there is no across word in this grid. The value 3 was chosen because it is near the middle of the down word, *contra*.

Dyadic across and down overview

The dyadic forms of **across** and **down** take as left arguments the boxed output of **across** and **down** – either monadic or dyadic output – and as right arguments, a word to be added to the word crossing grid. A simple example, building on the previous example is shown next. *dance* is added to the grid as a horizontal word. (Notice that the J primitive adverb “Passive” is employed to finesse the need for

parentheses.) *across* and *down* are case-sensitive to the input words, so *Dance* is different from *dance*, for example.

'dance' across~					down 'contra'	
3	3	3	3	1	5	%
		1	1	3	8	c
						o
						%dance%
						t
						r
						a
						%

dance has been crossed with *contra* at the letter "n" because the result is closer to the center of *contra* than if *dance* had intersected at "c" or "a". Other such word placement protocols are described later.

Initial processing in dyadic *across*: expanding AD (called AD0) into ADI0

To understand the processing in *across* and *down*, only examples from *across* will be given; the *down* processing parallels that of *across* and can be seen in the scripts.

Before the string *dance* can be placed in the original down, *contra*-only string array grid AD, the grid of shape 8 3 is expanded to shape 8 13. A boxed, expanded, and enhanced grid ADI0 from the execution of *across*, is shown below as it appears during a J debug session. The asterisks are placed in the augmented version of ADI0 to show all of the potential positions of the Dot preceding *dance*.

```

      $ADI0
8 13
    <'*(TryNdx)}ADI0
+-----+
|      %      |
| *****c    |
| *****o    |
| *****n    |
| *****t    |
| *****r    |
| *****a    |
|      %      |
+-----+

```

Next I describe how two key arrays, illustrated in the previous example, are constructed in *across*: ADI0 and TryNdx. ADI0 is the expanded grid described

above. TryNdx is an array of the left-most Dot positions of each potential Word placement in ADIO.

Inputs to across are read as follows. Their content was explained earlier.

```
'Center0 NSWE0val AD0' =. x.  
Word =. y.
```

SHAPEIO, the shape of ADIO, is computed in terms of the "core" and "shoulders" of the input to across — AD and Word — as described next. The "core" of the array ADIO is defined as a rectangular region. The height of the rectangle is determined by the north- and south-most existing horizontal words; the width of the rectangle is determined by the west- and east-most existing vertical words. The size of the core of AD is CoreIO and is computed as follows. The regions outside the core, but inside the array AD, is called MinShoulder. It is a minimum shoulder in the sense that the shoulder may have to be extended if the new candidate Word (with one of its Dots added) is larger than the western or eastern minimum shoulder.

```
(,NSWE0) =. ,NSWE0val  
CoreIO =. >: (0&>.) (Sa0-Na0), (Ed0-Wd0)  
WordLengthX =. 2+WordLength =. #Word  
MinShoulder =. >: |-/NSWE0val  
Shoulders =. MinShoulder >.&(_2:]]) 2#0,>: WordLength  
ShapeIO =. CoreIO + /rows Shoulders
```

IndicesInIO, an array of boxed row-column pairs, is computed to properly place the original array AD0 into an expanded (i.e., with the shape of ADIO), empty (i.e., space-filled) array. The placement is accomplished using J's } primitive conjunction, "Amend".

```
NB. OriginIO is an offset needed if the western MinShoulder is  
< #Word  
OriginIO =. 0, (0&>.) WordLength- 2{MinShoulder  
Shape0 =. $AD0  
InIO =. OriginIO + each i. each Shape0  
IndicesInIO =. {InIO  
ADIO =. AD0 IndicesInIO}ShapeIO$' '
```

The corners of the array of valid placement positions, TryCnr, are used to compute the full array of row-column positions, TryNdx, as shown below.

```
TryCnr =. 2 2&$ 1, 0, _2+ ({.Shape0), ({:ShapeIO-WordLengthX)  
ifc =: {:@(({})+each i. each @:>:@:(--)) / NB. indices from corners  
TryNdx =. ifc TryCnr
```

Some constants and utility verbs

Dot and other constants are supplied as follows.

```
Dot =: a.{~ 37
Qu  =: ' '
Sp  =: ' '
```

Some more utility verbs are listed next. Verbs whose names are suffixed with "Q" produce binary results.

ppA	=: [,],	NB. pre- and post-append x. to y.
ppADot	=: Dot&ppA	NB. pre- and post-append Dot to y.
wordDot	=: [: ppADot word	NB. word surrounded by Dot character
unemptyQ	=: [:~.(a:,a:)&~:"1	NB. mask for not empty boxed pairs from List
noEmpty	=: #~unemptyQ	NB. remove empty boxes
neqDotQ	=: ~:&Dot	NB. produces mask for ~: Dot
nonDot	=: #~neqDotQ	NB. selects Letters ~: Dot
nnDot	=: [:#[:nonDot]	NB. count of Letters w/o Dots
neqSpQ	=: Sp&~:	NB. produces mask for ~: ' '
nonSp	=: #~neqSpQ	NB. select non-spaces
allIneqQ	=: [: +./@:(0&~:) ,	NB. are all values ~: 0?
low0	=: 0&>.	NB. at least 0
firstlast	=: {., {:	
butfrstlst	=: }:@:}.	NB. remove Dots

Main across processing

The main processing in **across** involves supplying to the dyadic verbs the array **ADIO** as left argument and as right argument supplying the list of boxed pairs of valid starting positions boxed and the **Word** boxed. The right argument initially is **Tries** and is computed as follows.

```
Tries =. (,TryNdx),.<Word
```

An example of **Tries** is given next. The example is taken from a J debug session of adding *dance* across, as in the example above.

```
3{.Tries
+---+-----+
|1 0|dance|
+---+-----+
|1 1|dance|
+---+-----+
|1 2|dance|
+---+-----+
```

Once the initial list *Tries* is created, a major aspect of the processing in *across* involves culling and arranging this list of candidate starting positions for the present *Word*. The following 4 input utility verbs deconstruct many such verbs' inputs. The left argument to all these verbs is a string array like *ADIO* and the right argument of these verbs is list of boxed items like *Tries*. As such, in ultimate use in the *across* script, these verbs are assigned left and right ranks 2 1. An adverb rank is defined to simplify such rank assignments.

```
rank =: "2 1
```

By "in ultimate use" I mean that because when a verb is constructed in J using mathematical composition, then the rank assigned to the composed verb applies inwardly to the composing verbs, that the explicit rank assignment is only required for the composed verb. To simplify the appearance of verb definitions of verbs which use these target input utility, and on other higher order verb definitions, the rank is not given explicitly.

By "in ultimate use" I am distinguishing between a verb's definition and its application. Verb rank can be defined at either stage; to simplify in *across* a verb's rank is applied when the verb is applied. When a verb is applied in J using mathematical composition, then the rank assigned to the composed verb applies inwardly to the composing verbs; the explicit rank assignment is only required for the composed verb.

```
row      =: {.>@{.@] NB. (row,col) is the first boxed item in y.
col      =: {:>@{.@] NB. (row,col) is the first boxed item in y.
word     =:      >@{::@] NB. word is the last boxed item in y.
target   =:      [
```

After the inputs are read using the above 4 input utility verbs, the next sequence of dyadic verbs are used to extract information from *ADIO* pertinent to each item in the list argument.

```
atoms    =: col + (i.@#)@wordDot NB. column indices of Word
letters  =: atoms {row {target    NB. Letters and Dots of Word
intersect =: [:nnDot[:nonSp letters NB. count of real Letters crossed
rowba    =: ([[:<:row ),([[:>:row ) NB. row Before/After target row
lettersba =: [: butfstlst rows atoms {rows rowba {target
along    =: [:+/:[: +. /[: neqSpQ@nonDot rows lettersba NB. count of Lettersba
```

The following group of verbs are used to check if various requirements are satisfied for the given *Word* to be placed at the given position in *ADIO*.

```

validQ      =: matchQ *. intersectQ *. unequalQ *. roomQ
matchQ      =: (wordDot #-[: neqSpQ letters ) -: [: nonSp letters
intersectQ  =: 1:<. intersect
unequalQ    =: [: -. wordDot -. letters
roomQ       =: [: *./ rows along <:"1 0 intersect

```

`validQ` combines all the others.

`matchQ` checks whether all the letters in the grid match the corresponding letters in `Word`, excluding any spaces. Unfortunately this exception makes the result of `matchQ` true even for any completely empty areas of `ADIO`.

`intersectQ` overcomes the problem with `matchQ` because the two verbs are always used together and `intersectQ` is always 0 in such problem situations.

`unequalQ` is designed to insure that no `Word` is superimposed over itself in the grid, if that `Word` is submitted again.

`roomQ` insures that there are no letters running horizontally parallel immediately adjacent to where `Word` will be placed; but letters of intersecting vertical words are acceptable.

Depending on the binary value of `validQ`, `valid` either returns unaltered the right argument or an empty item.

```

valid      =: (i.0 )" _`]@.validQ

```

With these workhorse verbs defined, the list of valid candidate placement positions are found with the following two lines of `across`. `CandAll` contains results for each item in `Tries`, many of which are empty indicating they are not viable.

```

CandAll =. ADIO valid rank Tries
Cand    =. noEmpty CandAll

```

To clarify, consider the following excerpt from a J debug session extending the simple example above.

```

    $CandAll
30 2
    3{.CandAll
+---+-----+
|   |       |
+---+-----+
|1 1|dance|
+---+-----+
|   |       |
+---+-----+
                noEmpty CandAll
+---+-----+
|1 1|dance|
+---+-----+
|3 2|dance|
+---+-----+
|6 3|dance|
+---+-----+

```

That completes the discussion of the global workhorse verbs in *across*. Next I discuss some of the details of *across*.

More details of *across* processing

Although arguably the definition of the verb *valid* is quite logical, the resulting calculation of *CandAll* described above is very inefficient because the verb train which defines *valid* repeatedly processes very similar arguments. To overcome this inefficiency, the following verb definitions were constructed. The efficiency is gained by first capturing all three repeated calculations into *wrldetal* and then using at most two of the three at a time for the calculation of the four main condition-checking verbs. This efficiency can best be seen in the verb *validEQ* below.

NB. "E" stands for Efficient

```

nSp      =: nonSp@]          NB. simplifies Efficient definitions
pk1      =: 1&{              NB. simplifies Efficient definitions

matchEQ  =: (([ #-[: neqSpQ ])&>/@}: ) -: (nSp &>@pk1)
intersectEQ =: (1: <.: [nnDot nSp]&>@pk1
unequalEQ =: ([: -. [ -: ]&>/@}:
roomEQ   =: ([: *./ rows ] <: "1 0[: nnDot nonSp@[]&>/@}.

wrldetal =: wordDot; letters; along NB. simplifies definitions
validEQ  =: [: (matchEQ *. intersectEQ *. unequalEQ *. roomEQ) wrldetal
valid    =: (i.0 )" _]@. validEQ

```

valid has been given two different and valid definitions in this description. As in J when this is done, only the last definition is actually remembered by J.

Some programming and style conventions follow.

NB. In general, processing in the verbs across and down
 NB. follows a cycle suggested by the following sequence
 NB. of grid name suffixes.
 NB. AD name of the external grid which seeds the system
 NB. AD0 name of the internal grid - Originates the system
 NB. ADIO Intermediate expanded grid of Original
 NB. ADI Intermediate grid containing new word
 NB. ADtmp retrimmed grid;
 NB. really the next AD with an unnecessary new name

NB. Non-noun names begin with a lower case letter.
 NB. noun names begin with an upper case letter.

NB. Several verb pairs are defined for across and down
 NB. processing, respectively. The suffix 'D' denotes
 NB. the down version of the pair.

NB. Verbs which produce boolean results have the letter 'Q'
 NB. as a suffix.

NB. Word and word refer to candidate new words.
 NB. Letters refer to existing words in the grid.

NB. The nouns NSW, NSWI and NSWIO contain noun names,
 NB. which are used for indirect naming.

The naming pattern for each of the name groups - NSW, NSWI and NSWIO - follow the pattern described above for NSW.

The utility verb vw is used frequently to implement indirect assignments by extracting information from the NSW... series of values into corresponding names.

vw =: ".&> NB. for viewing NSW's values

The sorting rules in across determine the placement of each word in the existing grid. First, the rules favor placing each word in positions for which the word intersects the most existing words. Next the rules favor placing each word in positions for which the central "core" of words is least changed. The "core" is defined to be words inside a border defined by the following four values Na, Sa, Wd, and Ed. Finally, sortwise, preference is given to word placement which is closest to the original center of the word-crossing array.

To anticipate sorting criteria needs, once the list Cand of valid locations has been computed, across computes potential updates for the NSW values for each valid

Cand item. Values stored in nouns for which the NSW E name has the suffix "IO" are updated into parallel nouns for which the name has the suffix "I". The single actual update cannot be determined until after the list Cand is sorted in order to choose the best of its items, though. Notice that {. is used extensively and that only "a" named nouns, not "d", are employed because across values are manipulated here.

```
NB. Anticipate the potential change in NSW E
NSWEIOt =. {.NSWEIO
(NSWEIOt) =. ({.NSWEIOval) + 2#OriginIO
NSWEIt =. {.NSWEI
(NSWEIt) =. vw {.NSWEIO
Row =. ADIO row rank Cand
NaI =. NaIO<. Row
SaI =. SaIO>. Row
Atoms =. ADIO atoms rank Cand
WaI =. WaIO<. >:@:{.rows Atoms
EaI =. EaIO>. <:@:{:rows Atoms
```

The sorting section is a 3-level sort performed in reverse order of the importance of the sort criterion. Each sort level is done in 4 steps.

- 0 Calculate or determine the sort variable value.
- 1 Perform the "GradeUp" or "GradeDown".
- 2 Bond the "Grade" indices into a "Pick".
- 3 Incorporate the previous "Bond" into an verb which combines the "Pick" with the with the current one. For the first level this last step is unnecessary.

The least important criterion is the "city street" Distance between each valid placement position and CenterIO. Next important are the Changes the values of NSW E values which will result from each Cand value. The calculation of Changes employs the following global verbs.

```
signcap =: 0&<.&.:maskTimes NB. disallows irrelevant changes
maskTimes =: Mask &* NB. Mask multiplied by ...
Mask =: 1 _1 1 _1
```

The most important criterion is the number of words already in the ADIO grid which will be crossed ("Xrossed") by the new Word.

NB. sorting section follows

```
Distance =. +/|(vw NSWEIt) -" _ 2#CenterIO
DG =. /: Distance NB. DistanceGrade
dG =. DG&{

Changes =. +/|signcap (vw NSWEIt) -" _ (vw NSWEIOt)
CbDG =. /: dG Changes NB. ChangesbyDistanceGrade
cbDG =. CbDG&{
cbDg =. dG (cbDG@)

Xrosses =. ADIO intersect rank Cand
XbCbDG =. \: cbDG Xrosses NB.
XrossesbyChangebyDistanceGrade
xbCbDG =. XbCbDG&{
xbCbDg =. cbDG (xbCbDG@)
```

Once xbCbDg is determined, the overall sort order selection verb has been computed, it can be applied to Cand, NSWEIt, and Atoms.

```
Cand =. xbCbDg Cand
(NSWEIt) =. xbCbDg rows vw NSWEIt
Atoms =. xbCbDg Atoms
```

Once the best placement for the new Word has been identified, replc amends ADIO to ADI. Because the values of Cand have been sorted, the best one is the first one, so { . "Takes" it.

```
replc =: wordDot `(row <@;atoms)`target } NB. amend ADIO
ADI =. ADIO replc rank { . Cand
```

Completing across processing

Finally across must produce as outputs, the left argument input required for the subsequent implementation of down or across. Although ADI contains the correct number of rows for the updated array AD, the number of columns is unnecessarily many. "Center" and NSWE must be correspondingly revised.

The two boxed items in InIO contain the row and column indices of ADIO in ADIO, respectively. But we need to extract from ADIO the expanded row and column indices of ADI for the output ADtmp. So InIO is extended to InI which contains the required extended indices. The extension is done in four steps.

0 The extreme values of the two boxed items in InIO are extracted.

```
firstlast =: {. , {:  
InIOextrm =. ,@:(firstlast@>) InIO
```

1 The selected across values of "NSWEI" are taken from all such values.

```
NSWEaI =. {. rows vw NSWEIt NB. NSWEaI contains the selected values
```

2 Corners, the new extreme row and column values, are computed using "extend".

```
Mask =: 1 _1 1 _1  
minusMask =: -&Mask  
maskTimes =: Mask &*<  
extnd =: [: 0&<. &.maskTimes minusMask@[ - ] NB. define  
sequence extensions  
Corners =. _2<\NSWEaI extnd InIOextrm
```

3 eSeqXbyY computes InI by extending the index sequence in InIO using the Corners.

```
NB. extend Sequence in X using Y  
eSeqXbyY0 =: (*@)+each([{._1:**@])>each *each @>each i. each@ ])"1 0  
eSeqXbyY =: ([:(, @(>@{. @))) eSeqXbyY0&{.}, [:(, @(>@{:@)) eSeqXbyY0&{:}  
InI =. InIO eSeqXbyY each Corners
```

The required output array ADtmp is computed by picking values from ADI.

```
ADtmp =. ADI{~{InI
```

Next, default NSWE0val values are altered as needed. Mostly values with names containing "a" are altered, but potentially two "d"s are too.

```
temp =. low0 WaIO -&{. WaI  
if. temp  
do.  
Wd =. Wd + temp  
Ed =. Ed + temp  
end.  
Na =. Na - low0 NaIO -&{. NaI  
Sa =. Sa + low0 SaI -&{. SaIO  
Ea =. +/ low0 Ea, temp, EaI -&{. EaIO
```

Finally "Center" is updated.

```
RowChange =. 0
ColChange =. +/WaIO >: >:}.best Atoms
Center =. Center0 + RowChange , ColChange
```

Some applications

The script `tests` has been developed for experimenting with grid creation. Although there is no looping in the workhorse verbs `across` and `down`, this verb loops through a word list. The global nouns `Last`, `LastS`, and `LastW`, contain details of the final result of `tests`, and were used for debugging and continue to be used in experimentation.

Because at any point of grid development, it may be impossible to place individual items in the supplied list of words, a strategy for dealing with such cases is required. In this version of `tests`, such words are recycled by being placed at the end of the yet-unprocessed list of words.

Another input to `tests` is a list named `Sequence` containing only the two boxed words "across" and "down", which correspond to the two verbs `across` and `down`, and control the sequence in which the two verbs are applied. In this version of `tests`, when a word is unusable, as discussed just above, the list `Sequence` is rotated one place, also.

`History` is maintained in order to determine whether the processing loop in `tests` needs to be halted due to a lack of change in the developing grid.

The global `CountMax` allows multiple cycles of the inputs to be processed. However, with the setting shown, only one cycle will be processed.

```
CountStart =: 0
CountMax   =: 1 NB. maximum number of iterations of the word list
sm         =: ] NB. can be changed to sm =: smoutput to show intermediate output
```

```

tests =: dyad define
  Start =. x.
  'Words Sequence' =. y.
  Vocab =. Words
  Incumbent =. empty 0
  New =. Start
  Count =. CountStart
  History =. ;/i. Length =. 2>.Words *6# Sequence
  while. (Count<CountMax)*.-. -:/firstlast History
  do.
    Incumbent =. New
    History =. (<New) ,}: History
    Inc =. Lr 'Incumbent'
    LastS =: FSeq =. >@{.Sequence
    LastW =: FWd =. >@{.Vocab
    sm FSeq;FWd
    Tmp =. '(',Inc,')',Sp,FSeq,Qu,FWd,Qu
    Last =: New =. ". Tmp
    sm New
    if. Incumbent -: New
    do.
      Vocab =. Vocab,{.Vocab
    end.
    Vocab =. }. Vocab
    Sequence =. 1&|. Sequence
    if. 0=#Vocab
    do.
      Vocab =. Words
      Sequence =. }. Sequence
      LastC =: Count =. >:Count
    end.
    if. 0&=25|#Vocab do. smoutput Count,6!:0' end.
  end.
  New
)

```

The seven-word phrase "I sing of Olaf glad and big," appears in early J documentation. Coincidentally it is difficult to fit all of the words in this phrase into a single word-crossing grid. After some experimentation with a verb named "tests" I found the index permutation 4 2 1 6 3 5 0 of the seven words "i sing of olaf glad and big" which omits only one word from the final grid (can you tell the missing word?).

```

      (across 'tables') tests (<4 2 1 6 3 5 0{:i sing of olaf
glad and big'),<::'down across'
+-----+
| 3 4 | 1 5 1 7 | %%
|     | 1 6 2 7 | %big%%
|     |     | %l o %
|     |     | %tables%
|     |     | d a i
|     |     | %of%n
|     |     | % g
|     |     | %
+-----+

```

The verb `tests` creates a global noun `Last` which contains the final result, the familiar list of three boxed values. The final grid can be more attractively viewed with the following command.

```

      <(Dot;' ') rplc"i~ toupper >{:Last
+-----+
| BIG
| L O
| TABLES
| D A I
| OF N
| G
+-----+

```

Below are two experimental runs of `tests` with the eleven words of "The Golden Rule." In the first run, two words - "others" and "as" - are unused in the grid. The randomly generated permutation of the word sequence is found to be 5 0 4 0 4 3 6 1 7 1 2 . (The permutation does not really contain repeat indices, as it appears.) In the second example, all eleven words are used. I have enjoyed contemplating the variety of grids produced by changing only the list order.

```

      (K -.:;(:,,@|:)>{:Last)[(across 'the-golden-rule') tests
      (<a=:({~#?#)K=:;'do unto others as you would have them do unto
      you'),<::'down across'
+-----+
| others | as |
+-----+

```

Last										
5	7	2	5	1	15				%	
		1	8	2	15				w	
									%do% % %	
									%unto%y t	
									% y l o h	
									%the-golden-rule%	
									a u % % m	
									v % %	
									e %	
									%	
K i. a										
5	0	4	0	4	3	6	1	7	1	2

(K -.;:(,,@|:)>{:Last)[(across 'the-golden-rule') tests
(<a={~#?#)K=:::'do unto others as you would have them do unto
you'),<::'down across'

Last										
10	9	1	15	1	17				%	
		1	16	2	16				%you%	
									n %	
									t % a	
									%others%	
									% a %	
									w v a	
									%others%	
									% %you%% %	
									t l u %	
									%the-golden-rule%	
									% e %t n	
									%them% %do%t	
									a % %unto%%you%	
									v % o%	
									e %would%	
									% % o	
									%	
K i. a										
5	4	0	2	3	1	1	6	0	7	4

Helpful hints from various J programmers via the JForum have been invaluable to me in code development. Oleg Kobchenko was especially helpful in explaining how display of the final grids can be controlled for clearer web viewing using html.

The scripts can be found at <http://www.gsu.edu/~dscbms/ad/ad.ijs> .

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

At Play with J: The Google Test	Gene McDonnell	116
Tools, Part 3: Object Comparison	Dan Baronet	123
Launching a Workspace in Windows	David Crossley	131
Using J in Excel	Richard Hill	133
More on Execute Inverse	David Crossley	140

At Play with J: The Google Test

by Eugene McDonnell

Highway 101 in the San Francisco bay area is a busy commuter highway, with employees commuting to work at the headquarters of such companies as Adobe, Apple, Applied Materials, Cisco, eBay, Genentech, Google, Hewlett Packard, Informatics, Intuit, Oracle, Silicon Graphics, Sun Microsystems, Yahoo, and hundreds more high-tech companies. These commuters recently drove past a large poster paid for by Google, reading:

{first 10-digit prime found in consecutive digits of e }.com

Google apparently trusted that some among those passing the poster would understand it, and of these some might be intrigued enough by it to see if they could find that prime, and perhaps some of them might use it to go to the resulting web address. Those who did go the whole route would then find themselves with this message:

Congratulations. You've made it to level 2. Go to www.linux.org and enter *Bobsyouruncle* as the login and the answer to this equation as the password.

$F(1) = 7182818284$

$F(2) = 8182845094$

$F(3) = 8747135266$

$F(4) = 7427466391$

$F(5) = \underline{\hspace{2cm}}$

Those who find the value of $F(5)$, and go to the site shown, would get this message from Google Labs:

Congratulations.

Nice work. Well done. Mazel tov. You've made it to Google Labs and we're glad you're here.

One thing we learned while building Google is that it's easier to find what you're looking for if it comes looking for you. What we're looking for are the best engineers in the world. And here you are.

As you can imagine, we get many, many resumes every day, so we developed this little process to increase the signal to noise ratio. We apologize for taking so much of your time just to ask you to consider working with us. We hope you'll feel it was worthwhile when you look at some of the interesting projects we're developing right now. You'll find links to more information about our

efforts below, but before you get immersed in machine learning and genetic algorithms, please send your resume to us at problem-solver@google.com.

We're tackling a lot of engineering challenges that may not actually be solvable. If they are, they'll change a lot of things. If they're not, well, it will be fun to try anyway. We could use your big, magnificent brain to help us find out.

You now have all you need to know to dazzle Google with your magnificent brain. I haven't spoiled it for you, so you can legitimately do it on your own. I will, however, give you a similar puzzle, in two parts, and will solve it for you. It uses the digits of pi.

Problem 1: Finding 10-digit primes

The first problem is to find among the digits of pi a ten-digit sequence that, when evaluated in base ten, is a prime number, and is the eighth such. Your first problem, then, is to obtain the first few hundred or so digits of pi. We're in luck! The great Indian mathematician Ramanujan used the theory of complex multiplication of elliptic curves to give a number of beautiful formulas for pi's digits, and a variation of this technique was discovered by the ingenious Chudnovsky brothers, from New York City by way of Kiev. A J function for their method is:

```
Bigpi =: 3 : 0
a=. 545140134x
b=. 640320x ^ 3
c=. 13591409x
d=. 6541681608x
n=. 1. >: x: y.
s=. (! 6 * n) * c + a * n
e=. (! 3 * n) * ((! n) ^ 3) * b^n
m=. {: e
f=. d * - / (s * m) % e
k=. (a * m) * <. @ %: b * 10x ^ 28 * y.
k <. @ % f
)
```

Given an integer argument n it finds $1+14*n$ places of pi. To solve the problem you should use a hundred or so digits, so 15 would give about the right number.

```
q =: bigpi 15
```

To work with the individual digits it is convenient to work with the character form of q.

```
w =: ": q
# w
211
```

Here are the first 210 digits:

```
7 30 $ w
314159265358979323846264338327
950288419716939937510582097494
459230781640628620899862803482
534211706798214808651328230664
709384460955058223172535940812
848111745028410270193852110555
964462294895493038196442881097
```

We need these ten at a time:

```
t =: 10 [ \ w
$ t
202 10
5 {. t
3141592653
1415926535
4159265358
1592653589
5926535897
```

To determine which of these are prime we have to convert each row into a number.

```
p =: ". t
$p
202
5 {. x: p
3141592653 1415926535 4159265358 1592653589 5926535897
```

Some of these may have had leading zeros, so that they are effectively 9-digits long. We remove them - some of these may be prime, but they don't qualify as ten-digit primes.

```
pa =: (p > 999999999) # p
$ pa
183
```

A convenient way to determine whether a number is prime is to count how many prime factors it has; if it has just one, the number is prime. We can think of using

J's *prime factors* primitive *q*: to obtain the factors of the numbers in *p*, but will find that this is not always possible; many of the numbers are outside its domain.

```

      q: 2004
2 2 3 167
  # q: 2004
4
      q: 2003
2003
  # q: 2003
1

```

So 2003 is prime, and 2004 isn't. Here is an *is prime* function *ip*:

```

      ip =: 13 : '1 = # q: y.'

      ip 2003
1
      ip 2004
0
      ip 2000000000
0
      ip 2100000000
0
      ip 2200000000
|domain error: ip
|      ip 2.2e9
| 2200000000 = * / 11 5 5 5 5 5 5 5 2 2 2 2 2 2 2 2
1

```

At this writing, the *q*: function will yield a domain error for many integers with ten or more digits, even when it is obvious the number isn't prime.

To circumvent this, we can use J's *adverse* conjunction, defined as "The result of *u* :: *v* is that of *u*, provided that *u* completes without error; otherwise the result is the result of *v*." We can write a *special is prime* function *sip* to return *_1* as result if otherwise a domain error would be reported:

```

      sip =: ip :: _1:
      sip 2200000000
_1

```

We use *sip* on *pa* to let us know which are known composites (0), which are known primes (1), and which are not determined yet (*_1*).

```

pb =: sip " 0 pa
~. pb
_1 0 1
# /. ~ pb NB. how many of each
159 23 1

```

We can remove the known composites, leaving us with just the known primes and the suspects.

```

pc =: (pb ~: 0) # pa
$ pc
160

```

Now comes the hard part: determining which of these are prime without the use of q: .

A number is composite if it has two or more prime factors. Twenty-five is composite since it has the two factors 5 and 5. The larger number 9999399973 is also composite, with the two factors 99991 and 100003.

A prime number z has only one prime factor, namely z . A composite number w must have one or more prime factors less than its square root r . It can only have one prime factor s larger than its square root r . It may be that all of its prime factors are less than r . In any case, to find whether a number is prime or not it suffices to see whether any of the primes less than its square root divides it, that is, gives a residue of zero with n . Thus, if we have a list pf of all of the primes less than r we will be able to determine whether a number n is composite by seeing if it has a residue of zero for any of pf . If it doesn't then we know that n is a prime.

We know that the numbers we are interested in will have values from 1000000000 to 9999999999, inclusive. The square root of the largest number we may find is thus less than 100000, so it suffices that pf contain just the primes less than 100000.

We can find these easily by using the function inverse to p:, that is, $p: ^: _1$.

```

p: ^: _1 [ 100000
9592
p: 9592
100003
p: <: 9592
99991
pf =: p: i. 9592
{: pf
99991

```

We need a function that finds the residues of a number with respect to each prime in `pf`, and gives 0 if the number is composite and 1 if it is not, that is, if it is prime.

```
nc =: 13 : '-. 0 e. pf | y.'0
```

This reads "not 0 in `pf` residues of `y`". We apply it to `pc`:

```
pd =: nc pc
```

How many 10-digit primes have we found?

```
9 +/pd
```

More than enough to solve the puzzle. Which are they?

```
] pe =: I. pd
2 33 36 43 73 108 128 135 149
```

And what are they?

```
pg =: x: pc { ~ pe

,. x: pg
5926535897
4197169399
1693993751
7510582097
4825342117
5822317253
2841027019
8521105559
8954930381
```

Just for fun, locate these in the digits of `pi`:

```
7 30 $ w
314159265358979323846264338327
950288419716939937510582097494
459230781640628620899862803482
534211706798214808651328230664
709384460955058223172535940812
848111745028410270193852110555
964462294895493038196442881097
```

Three of them overlap. We want the eighth, 8521105559.

Problem 2: Finding the fifth in a series

You are given five 10-digit numbers from the digits of pi, and must find the sixth. Here are the numbers:

4338327950
2795028841
6939937510
3993751058
2110555964

Here they are, embedded in pi:

7 30 \$ w
314159265358979323846264338327
950288419716939937510582097494
459230781640628620899862803482
534211706798214808651328230664
709384460955058223172535940812
848111745028410270193852110555
964462294895493038196442881097

The first and second overlap, as do the third and fourth.

I'll give two hints, the second vacuous:

Hint 1: Primarily, the sixth number has three doublets and overlaps the fifth.

Hint 2: Alternately, something for nothing.

If these hints still aren't enough to let you determine the sixth number, you can mail me at eeemcd@mac.com, and I'll give you the answer – but you'll still have to find out *why* it is.

Tools, Part 3: Object Comparison

by Dan Baronet (email: dbaronet@milinta.com)

In the following article I use terms specific to our trade. You won't find them in the dictionary but I assume the reader is familiar with words such as 'monad', 'global' (as a noun) and 'default' (verb). I use the terms 'nested' (as in 'nested array'), 'enclosed' or 'boxed' interchangeably. I also use quotes and angle brackets to help determine the type of the object I am referring to. 'Quotes' denote a variable or workspace and <angle brackets> refer to a function/operator or file. Often, the context is sufficient to remove ambiguities. Italicized words have a special meaning and often precede their definition. Optional items are shown within [brackets].

Introduction

Early in my programming career I realised there was a need to compare various APL objects. At that time (late 70s), there was no such thing in the "public domain". I needed to compare variables, functions and packages, and in files too. This is when I started thinking about having my own set of utilities to do just that.

This is the third of a series of articles on tools in APL. This is an example of code that evolved from simple to complex utility where feedback from the users made it the way it is. It is compatible with earlier versions of APLs like APL/PC.

As usual keep in mind the date this text was written as things have a tendency to change...

A Workspace to Compare Code

This is one of my first complete sets of utilities. It was first written for SHARP APL and was eventually ported to other platforms. It allows comparing code and reporting differences with various details. It was written with nested arrays and packages in mind. Today it also works on namespaces and overlays.

COMPARE utility

Keywords: COMPARE, DIFFERENCES

This workspace will show where differences occur in APL objects found in a workspace or in a file.

The core of the matter

The basis of this workspace is a function that takes the $\circ . \equiv$ of the lines of two matrices³ and returns a boolean matrix ('m') with the following properties:

1. no more than one '1' per row or column
2. each 1 is below and to the right of the previous 1, if any
3. + / , m is a maximum for the 2 previous constraints

Function <compbool> performs that operation⁴.

Basic functions

The next level function is dyad <compcm>, which takes two character matrices, compares them using <compbool> and formats the output nicely, combining the lines of the two arguments with decorators to show insertions and deletions. It also limits the comparison to the zones where the matrices differ in order to limit the amount of output. If two 100 lines matrices differ only by one line the output is limited to that region plus a few lines before and after to render the context. If the matrices are identical " is returned.

<compcm> is used by <compvar> and <compfn> to compare variables and functions respectively. These functions all return a character result that depends on the presence and value of a series of global settings. Another function, <compack>, compares *packages* using the above two functions. The term *package* refers to its namesake in SHARP APL or APL2000 (where they are emulated), a namespace in Dyalog APL or an overlay in APLX. <compack>'s reporting behaviour is like <compare>'s (see below). Instead of taking workspace names as argument it takes *packages*.

Comparing workspaces

Although the previous functions can be used on individual objects they are usually used in the context of comparison of workspaces.

In order to compare entire workspaces it is necessary to send the workspaces to a file and to use the <compfile> program using immediate execution statements as follows (compare is the comparison workspace):

³ Padded to get the same width if necessary

⁴ There may be more than one optimal solution.

```

)xload ws1
)copy compare tws
tws a transfer the workspace to file
)xload ws2
)copy compare tws
tws
)load compare
compfile '/switches (see below)' a the report is produced here
clearfile a to clear file space

```

<tws> is the Transfer WorkSpace function. It moves the variables and functions of the current workspace onto a specific file preceded by some workspace info. If the file does not exist it is created. After <compfile> has been run, <clearfile> should be used to get rid of the file.

Higher level functions

Under APL2000 or SAPL (if you have multi-task access) it might be easier to use the program <compare> as follows:

```
'ws1' compare 'ws2 /switches'
```

<compare> uses Stasks (every APL should have Stasks) to perform the previous immediate execution statements under SAPL. Under APL2000 []INBUF⁵ performs the same function. Other APLs⁶ cannot do this automatically. Yet.

Dyad <comparefiles> takes two filenames as arguments and reports on the differences of each corresponding component using the same methods. It won't report anything on identical components.

Switches

You can use the following switches with the right argument of both functions:

/norm	will left justify the fns before comparing them
/result	will return all the output as result. By default all output is displayed only.
/show=	temporary override for global SHOW (see below)
/xnames=	exclude the names that follow. A simple pattern can be given.
/xstr=	exclude the objects containing that string

⁵ there is a bug in APL2000 (V3.5) where a non empty stack interferes with the use of []INBUF so comparing workspaces with pending stacks fails using this method

⁶ some APLs like APL2 can but no version of this code exist for them. In Dyalog APL we can cheat by creating a temp namespace and []cy'ing the workspace into it but we don't get the stack this way. []CY also won't copy workspaces with a pending stack in V8.1, maybe other versions too.

The use of these switches is strict (for details on the parsing rules see the previous Vector article on this subject).

Output Specifications

The information displayed is determined by a few globals.

DELINS

When differences are displayed between objects, *deleted* lines and *new inserted* lines are preceded by a symbol, one for each case. DELINS holds both characters. By default they are '+' and '-'.

NLINES

This variable is for showing only the first N lines of a variable in the "unique symbols" area (see SHOW below). This prevents displaying too many lines if a variable is very big when displayed. The default for this variable is set at 10 (lines). This takes in account wrapped lines. If the output is incomplete the string '...(more)' is added to the truncated display.

NOLASTLINE

When set to 1 the last line of each function is ignored **if it is a comment**. This allows comparing workspaces generated by systems that *tag* functions with version control information⁷. Default 0.

SHOW

The report is divided into 3 sections:

- [1] the symbols found ONLY in the first package or workspace
- [2] the symbols found ONLY in the second one
- [3] the symbols found in both

Three things can be shown in each *section*:

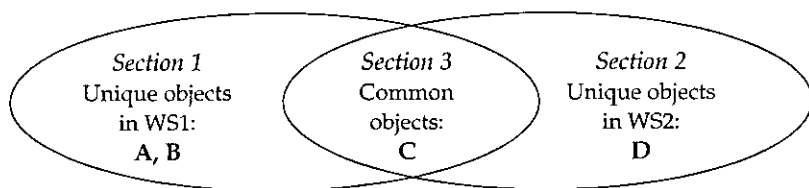
- the number of symbols
- their names
- their contents (or difference)

⁷ such tags generated by LOGOS are detected and automatically ignored in this version

The number of symbols is always displayed if there are objects in a section. The names and contents may be displayed depending on the section's value in SHOW:

- 0 nothing is shown
- 1 the names only are displayed
- 2 their contents/difference only is displayed
- 3 both names and contents are displayed

For example, if workspace WS1 contains objects A, B and C and workspace WS2 contains C and D. The first section (the objects only in WS1) would report on 2 objects: A and B. The second section (the objects only in WS2) would report on object D only and the third section (common objects) would report on object C, the only common name in both workspaces. Here's a Venn diagram to illustrate:



If `1=2|SHOW[1]` then the names 'A' and 'B' would be listed in the first section after the line "2 outstanding objects in WS1".

If `1=1+2 2+SHOW[1]` then their values would also be displayed. The same would apply to the two other sections using `SHOW[2]` and `SHOW[3]`.

SHOW's default value is 1 1 3, that is only the names of the objects in the first 2 sections (those with unique names) are displayed, not their values, and the names and the differences of the objects in the third section (the one with common names) are shown. To temporarily change this setting indicate so on the command line. For example, to only show the names for all sections:

```
'ws1' compare 'ws2 /show=2'
```

SHOW may be assigned a scalar in which case the same value applies to all 3 sections.

ZONE

This one affects the display of differences between objects. This is the number of lines shown before and after the differences found. The default is set to 2. This

allows seeing a couple of lines before and after the changes to possibly better understand the context.

Details

The display includes pseudo system variables and the stack. For all APLs the pseudo variable []SYS contains the grouping of the system variables. The first 6 elements are []IO, []CT, []PP, []PW, []RL and []LX. The remaining elements depend on the APL.

For APLs that still use groups⁸ the pseudo variable []GRPS includes all the groups' names and the objects they contain.

Example 1

Workspace WS1 contains variables 'v1' with values 0 to 8 except [3 7] which are -1 and 0, and 'v2'. It also contains functions <f1>, <f2> and <f3>, which contains 3 rows: 'one', 'two' and 'three'. Global []IO is 1.

Workspace WS2 contains a different variable 'v1' with values 0 to 8. It also contains functions <f2>, identical to the <f2> function in WS1 and <f3>, which contains 4 rows: 'one', 'new two', 'three' and 'four'. Global []IO is 0. There are no other objects in this workspace.

There is no stack in these workspaces.

The comparison output will look like this⁹:

```
'WS1' compare 'WS2'
*** comparing C:\APL\WIN\WS1 created 2003 9 14 18 19 25
    with C:\APL\WIN\WS2 created 2003 9 14 18 20 8
```

```
NOTE: - wss sizes differ by 124 bytes.
```

```
*** 2 outstanding objects in C:\APL\WIN\WS1
f1 v2
```

```
*** 4 common objects
```

⁸ groups are still used in APL2 and SHARP APL on the mainframe. They are obsolete but are included for completeness

⁹ the format may vary slightly from one APL to the other

```

#sys    f2    #f3    #v1

> Variable sys
objects are char matrices
+[0]    io =1
+       io =0
[1]    ct =1.0E-13
[2]    pp =10

> Function f3
[0]    f3
[1]    one
+[2]    two
+       new two
[3]    three
+       four

> Variable v1
objects are num vectors
elements 3 7 are different (io=1)
Var1 0 1 -1 3 4 5 0 7 8 Var2 0 1 2 3 4 5 6 7 8
use 'clearfile' to erase the file.

```

By default only the list of the objects in WS1 and WS2 is shown, not their values.

Since there are no outstanding objects in WS2 that section is skipped.

Pseudo system variable [SYS contains one row per system variable in a character matrix format. Comparing it is the same as any other character matrix.

The list of common names includes a special character, '#', for each name whose definition is different.

After the report is displayed a file exists that contains a copy of both workspaces. To rerun the report one only needs to call <compfile> like this:

```
compfile ''
```

To rerun the report with different parameters either change the globals in the workspace or specify new line parameters. For example, to display the value of the objects only in WS1 enter

```
compfile '/show=2 0 0'
```

Example 2

File <filea> has three components. File <fileb> has four components starting at component 2.

Their component three contains character matrices which are different by 2 lines. Their comparison would look like this:

```
'filea' comparefiles 'fileb'
filea has 3 components starting at 1
fileb has 4 components starting at 2
```

```
*** component 3
objects are char matrices
[1] licence
[2] note
+[3] x
-[4] y
[5] CR
[6] DELINS

*** comparing access matrices
(no difference)
```

Components that are identical or not found in the other file are not shown.

Conclusion

This code was written to help programmers do their job, not to be sold and make money (obviously). It's a tool. There have been several tools created over the years by very capable people. This one doesn't pretend to be the best but it's easily available for free and it's the same whether you use APL2000, Dyalog, APLX or SAPL.

Maybe it will be found in the new APL library under development. At any rate you can find it at www.milinta.com. Help yourselves if you think it can be of some use.

Note

Anyone looking at my code may sometimes wonder why I wrote things this or that way. To accommodate all APLs I sometime use the "lowest common code" approach, code that will work in all APLs. I sometimes also add code to circumvent problems in a particular APL. For example, in this workspace, you'll find the expression

```
,0 0 ⍋m
```

where the comma should normally be unnecessary. Unfortunately, a bug in some APLs make them return their right argument when it's empty, requiring the use of the comma.

Launching a Workspace in Windows

by David Crossley (crossley@au-village.com)

When you install your version of APL under Windows, the workspace file type will probably be registered in the Windows Registry and associated with the calling program. Thereafter you can left-double-click (or left-single-click if that option is chosen) on a workspace file in Explorer to launch APL and load that workspace. You can instead right-click to open a context menu and select Open to do the same thing.

But what if you have several installed versions of APL from the same vendor? The latest installation will probably pre-empt the Open setting leaving you apparently with no choice other than to launch the version that you wish to use and then load the workspace from APL.

However, Windows allows you to add your own file associations to the context menu. Perhaps this is old hat to you but if not, read on. In Explorer, select a workspace, then choose File/Options. from the menu and open the File Types tab. Locate and select the registered file type (eg Dyalog APL Workspace) and then choose Edit to display another dialog.

Amongst the options list, you will see **open**, possibly the only option. If you select this and then Edit, you will see the Action in a non-editable field and an Application Used input field containing something like:

```
"H:\DYALOG10\DYALOG.EXE" "%1"
```

This, of course, is the path to the application program. The "%1" indicates the selected workspace file to be passed to the application when it is launched. There is a Browse button to locate the application program you wish to use. **Leave this as it is.** Cancel to return to the previous dialog.

To add a new association, select New to present the same dialog. You may now define your own Action. For example, enter "Open Dyalog &8.1" without the quotes. The & as usual denotes a shortcut character that will appear underlined. Use Browse to locate the application file. It is advisable to enclose this in double quotes, necessary if the path includes spaces, and also the "%1" with a separating space. Press OK to save this and return to the calling menu.

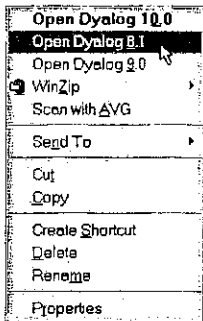
You will see the new entry in the Actions list. The default action is highlighted in bold type. You can set any you choose using the Set Default button. This is the action that is used when you left-double-click a workspace file.

There remains just one small problem. The original action is labelled **open** whereas you might prefer something like **Open Dyalog &10** so that "Open Dyalog 10" appears in the context menu. Oddly, you cannot edit the Action field. You must make the adjustment directly in the Windows Registry.

Caution: It is advisable to back-up the Registry before making amendments.

Open the Windows Registry through Run in the Start menu, type "regedit" and expand HKEY_CLASSES_ROOT.

Search for the subkey for the workspace file extension, eg .dws, and click on that. Its default value indicates the file type by its subkey code, eg dwsfile.



Search for and open the file type subkey, eg dwsfile, and then the shell subkey. You will see the coded names of the file associations, in this example **open** and **open_Dyalog_81**. Notice that the new action has a derived name. If you click on the action subkey name, you will see its displayed form as the default value. You may safely edit the default value for any action to whatever you like. Right click on the value field, choose Modify and then edit the little pop-up box. Precede the shortcut character with &.

You are now in business. The little illustration demonstrates what my context menu looks like when I right-click on a Dyalog workspace. Of course, the action will succeed only if the workspace is actually readable by that version of APL.

Using J in Excel

by Richard Hill (info@assuredsystems.com.au)

Whenever I tried to use Microsoft's Excel, I missed the ability to use Iverson's notation for a formula or a macro. Some people seem to be able to easily learn different languages, and the advice given in the documentation for Iverson's language, J, is fairly consistent, "use things like Excel's macro facility, VBA, when appropriate, J when appropriate".

Don't you find it maddening when you have to use another tool, at rare intervals, and you have to re-learn its inconsistencies? I do, so when I use Excel, I try to use the bare minimum of Excel facilities, whether or not it would be better to use more VBA and less J.

Personally, I find it enough to learn J and I don't really want to learn the other things. To put it another way, you could be trying to use a computer for a purpose other than learning programming. You might prefer to use a single multipurpose programming tool that meets your needs rather than be forever spending time trying to learn yet another programming tool just because it is the best for a particular purpose.

This means that you can use your familiar multipurpose tool to get on with what you really want to do, be it modelling factory production, actuarial research, textual analysis or whatever. If using your chosen tool is your goal, and you have chosen Iverson's notation, then you might want to temper the advice from Jsoftware, and actually try to use J everywhere you possibly can. This approach should increase the amount of practice with the language and familiarity with the tokens.

There are several ways you can use J from Excel. One of the simplest is to use it in a formula.

Take the Excel formula, (assume it is for cell D1):

```
=A1 - B1 - C1
```

(Assuming cells A1 B1 C1 hold 3 4 5, then in Excel: 3 - 4 - 5 = -6)

Instead, you can force the formula to be done in Iverson notation:

```
=jcmd(A1 & "-" & B1 & "-" & C1)
```

(Assuming cells A1 B1 C1 hold 3 4 5, then in J: $3 - 4 - 5 = 4$)

To achieve this you have to have the macros and functions to support this in your spreadsheet.

The Excel VBA macros and functions to support J are held in the text file:

```
...J5\system\examples\ole\excel\jsutil.txt
```

I found the best way to get them in, is to:

open the jsutil file with Notepad, highlight from start to end, Ctrl C, then open the spreadsheet, select Tools/Macro/Visual Basic Editor/Module/View Code and Ctrl V the code into the module.

You also need to get the spreadsheet to automatically open J.

There is an auto-open macro in the comments at the start of the jsutil text. Just delete the leading quotes from this bit of text.

Then you have to save and close the worksheet and re-open it to get it to work.

This process is documented in the J Help facility, User manual, OLE & OCX, Tutorial: J OLE Server for Excel.

To quote the tutorial, "In general, jcmd can be used for calculations which return a single value to be displayed in the current cell. The right argument is the sentence to be sent to J."

There are other macros described in the tutorial, mostly for handling spreadsheet ranges.

Using these macros, you can use Excel's VBA editor as a method of building and handling J scripts, like this:

```
Sub accum()
    ' writes the accumulated values and progressive means
    ' in adjacent columns
    ' To test, please enter some numbers
    ' in column E rows 5,6,7,8,9
    jsetr "jvar", "E5:E9"
    ' Note: the value that comes from Excel has each cell boxed
    ' then you can have a set of J statements
    jdo " a=: +/\,>jvar"
    jdo " m=: a% 1+ i.#jvar"
    ' the value to be put back into the spreadsheet
    ' has to be boxed to match the target range
    jdo "ret=: <"0 a,. m"
    jcmdr "ret", "F5:G9"
End Sub
```

With this method, you have the J execution box open to help in debugging. If there is a problem on the J side you get an error code in Excel, and you can flick back to J and see more about the error.

If you want to pass the spreadsheet on to other users, but you don't want to have the users involved with the J side, you can remove the jshow and jlog lines in the auto_run macro to hide the J part of the exercise.

This facility should enable complex applications to be built in Excel, with the developer being able to keep as much as possible within the clean consistent logic of Iverson's notation.

A More Detailed Example

Let us assume that we want to do some more complicated development, for Excel, using J.

For example, let us try to develop a useful tool for a factory production planner.

A problem in production planning is to predict what your customer will order next and when.

If you are trying to forecast what a group of customers will do, then there are many well chewed-over approaches that work quite well with aggregate demand.

However, there is a class of problems where there is only one customer for the product and the classic approaches don't work. You could, for example, be supplying a unique component to an automotive manufacturer.

Typically the manufacturer will have some sort of planning system and will be ordering components to a pattern that looks like this...

Date	Quan
4/3	101
5/3	197
7/3	208
8/3	151
11/3	103
12/3	97
15/3	209
17/3	147
20/3	205

The problem is to predict the date and quantity of the next few requirements.

The next section of these notes shows how one can go about developing a solution starting with Excel and working in J.

Let's look at the quantity problem first.

"Local knowledge" tells us that the customer has some sort of batching process internally. They could be using racks to move the parts up to the assembly line or some sort of "Kanban" lot cards to move parts within the plant. Therefore the requirements will be in multiples of the internal lot size, but there is a super-imposed variation caused by quality problems and/or odd quantities needed for replacement parts.

From the user's point of view, we want to give them the facility to highlight some figures in Excel, hit a key, and have the result returned to the spreadsheet.

Here is how I use VBA and J to make a simple user function.

The first step is to get the selected numbers out of Excel into J when the user presses a key in Excel.

In Excel, I usually make a new macro this way.

- Select Tools/Macro/Record New Macro.
- Type in the name Lotsize, and give it the shortcut key L.
- Then go back to Tools/Macro and select Stop Recording.
- Back again to Tools/Macro, select Lotsize from the list of macros and click on Edit.

Now we can type in the statements to get the selected numbers into J. The result ends up looking like this:

```
Sub Lotsize()  
'  
' Lotsize Macro  
' Macro recorded 8/01/2004 by Hill  
'  
' Keyboard Shortcut: Ctrl+L  
'  
' get selected shipments into J  
shp = Selection.Value  
' this assumes that auto_run has already built the linkages.  
ec = js.set("shp", shp)  
If ec Then MsgBox "Error code: " & Str(ec)  
  
End Sub
```

I usually find it best to save the Excel worksheet, and restart it, at this point.

When you start Excel, the auto_run macro brings up the J window. Then Alt-tab back to Excel, select the numbers to be handled, press Ctrl L.

If you now go to the ijk window, type in shp and Enter, you will find the numbers in J (boxed).

```
, shp
+-----+-----+-----+-----+-----+-----+
|101|197|208|151|103|97|209|147|205|
+-----+-----+-----+-----+-----+
...
```

Now it is time for the fun part of the job. I love playing around with numbers in J.

As soon as I've got a sentence worth saving, I start building a script file. This is a good idea, because if there is a problem between Excel and J, the quick way out is to start Excel again and you can lose the ijk window.

When you have a J script, you need it to be loaded by your Excel macro. Another couple of j statements in the macro will handle this. For example:

```
jdo "load 'g:\jex\lotsize.ijs'"
jdo "lotsize shp"
```

Finally you get a macro that contains a mix of VBA and J, like this:

```
' Lotsize Macro
' Keyboard Shortcut: Ctrl+l
'
' get selected shipments into J
shp = Selection.Value
' this assumes that auto-run has already built the linkages.
ec = js.set("shp", shp)
If ec Then MsgBox "Error code: " & Str(ec)
' bring in a J script
jdo "load 'g:\jex\lotsize.ijs'"
' execute J functions within the script
jdo "ltsz=: lotsize shp"
' return the calculated lotsize to the spreadsheet
' the value to be put back into the spreadsheet
' has to be boxed to match the target range
jdo "ltsz=: <ltsz"
jcmdir "ltsz", "A1:A1"

End Sub
```

The bulk of the logic is in the J script. This is the script written for this case.

```

Lotsize=: 3 : 0
NB. <calc lotsize>=: lotsize <vector of ship quantities>
NB. Changes:
NB. a 040114 improved text

NB. summary of algorithm
NB. remove sports eg outside n*sigma limit
NB. take newest
NB. assume lots per day 1 to n
NB. get remainders for various n
NB. round to units
NB. nub of possible lot sizes
NB. remainders with each lot size
NB. if > 50 pct, use complement
NB. get propor to ship size
NB. lot size with min total propor diff

NB. allow for incoming boxed arrays
s=: >,y.
NB. mean (from Phrases)
A=: # %~ +/
mn=: A s
NB. Sample variance (dispersion) of y (from Phrases)
m27=: (+/@(*:@[] - +/ % #)) % #)"1
NB. Standard deviation of y (from Phrases)
m28=: %:@m27"1
sd=: m28 s
NB. filter out shipments outside limits
f=: (mn- f),mn+ f=: 2*sd
s=: ((s>0{f})*(s<1{f}))#s
NB. select last few usable shipments for further work
s=: (-12 <. #s){. s
NB. assume a limited number of lots per shipment
NB. get possible lot sizes
b=: ~. , <. 0.5+ s % / 1+i.9
NB. which lot size gives minimum remainder?
r=: b | / s
u=: r> 0.5*b
c=: (r* -. u) + u* b -r
d=: +/ |: c %"1 s
(d i. <. / d){b
)

```

Acknowledgements

A number of people have commented on a draft of these notes. In most cases their comments have been incorporated. The exception is Oleg Kobchenko's suggestion of a "japply" macro which would apply a J phrase to a given range in the spreadsheet. This would be a very useful extension. An idea for someone else, or another day....

Conclusion

You can use Excel and Iverson Notation together without having to learn VBA at all, by using jcmd in the Excel formula box.

Unfortunately, once the user interaction is slightly complicated, you have to write some VBA. However, you can keep it to a minimum.

I like to do the fun part in J and the least that I can get away with in VBA.

More on Execute Inverse

by David Crossley (crossley@au-village.com)

In Volume 20.3, the Zark Newsletter discussed Execute Inverse and also included submissions by Nicholas Small and myself as separate articles. I learnt that `''', ((1+⊖AV=''))/⊖AV), '''` may not be executable. Dyalog APL reports a SYNTAX ERROR due to just a single character, the first or *nul* character.

Rex Swain offered two solutions for APLs respectively without and with nested structures as originally conceived in APL*PLUS NARS by STSC. Oddly, the solution without nesting overlooked the problem of non-executable characters. The second devoted quite a lot of code in substituting the APL*PLUS's `⊖TCnnn` characters (e.g. `⊖TCNUL`) within a text string, e.g. `'...',⊖TCNUL,'...'`.

With the benefit of hindsight, I have improved my original submissions to handle non-executable characters by substituting an indexed `⊖AV[⊖IO+n n n]` code string rather than using APL*PLUS system variables.

The most concise coding would produce output with no special-casing; with nested elements as `(c...), (c...)`; and with the explicit reshape of all sub-arrays. It would have redundant parentheses. Altogether not very readable but with the fastest execution time, typically 25-50% faster.

The results from the functions below are clear, much as one might enter an expression via the desktop. There are no redundant parentheses or spaces. Text null is presented without an explicit reshape as `''` while the numeric null is presented as `θ` (zildeg) in the DFn (Dyalog-specific) but as `0p0` in the more general function. The unit vector is presented as e.g. `.5` or `'x'`. There is no explicit reshape for scalars or vectors of length greater than 1. Strand notation is used. Items of text vectors are not enclosed in parentheses but separated by space, except for strings with non-executable characters. No attempt is made to compress, say, an item of repeated elements.

The first submission is a Dyalog APL dynamic function (DFn). There are no local variables at all apart from a single static (*nu1*). DFns encourage functional programming.

Notice how the operands of a dynamic operator (DOP) may be used to pass effectively one or two extra array arguments; operand $\alpha\alpha$ of DOP αd is the current sub-array whose α 'th element is being processed.

```

ei←{[PP-15 ⋄ nul+θp[AV                                     A String that reproduces w when executed
  sh←{α≡θ:'θ' ⋄ α≡':':'''''' ⋄ (,1)≡pα:',' ,w           A reSHApe - special cases
    1ε(0 1=pρα)^1,1<θpρα:w ⋄ (*ρα),'p',w               A condit'l explicit reshape
  }
  ad←{1=x/pαα:'c',w ⋄ np α>αα:{α=[IO]!}' ,w ⋄ 1φ'}(' ,w) A ADorn nested(dop)
  np←{(~nulεw)^{0≡εw}v{w≡θ}v{' '≡θp0ρw)^{1=pρw)^1≠θpρw} A No Parens
  qu←{({1+w='''')/w}                                     A double-up QUotes
  pt←{[ML+0 ⋄ {w≡-1+1,w}w=nul}c w}                       A ParTition on nul chars
  tx←{nul=θpρw:6φ'[IO],''', [AV[',(1<p,w)/(*ρw),'p' ⋄ w} A process TeXt
  tr←sh←{                                                  A TRaverse array
    326=[DR w:t,/({tpw)(w ad)''w tr''w                 A nested or mixed simple
    0=θp0ρw:w                                             A numeric
    ~nulεw:1φqu''',,,w                                   A normal text
    t,/tx''pt 1φqu''',,,w                                A text with nul chars
  }∘{(1fp,w)ρw}                                           A expose prototype
}
w tr w                                                    A do it

```

The second, slightly slower, solution is a more general function that should work, perhaps with minor adjustments, in most APLs that support control structures which may, of course, be re-coded easily to use branches and labels. Non-executable characters are identified as the origin-0 indices into $[AV$ in line [1] to suit the version of APL.

This example is fine for Dyalog APL.

```

      v r←ei_n b;i;l;ne;x;y          A String that reproduces b when executed

[1]  ne←⊂AV[⊂IO+0]                  A non-executable chars - inc as appropriate
[2]  y←(1[p,b])pb                    A expose null
[3]  :If 326=⊂DR b ⋄ r←ei_n''y        A nested or mixed simple
[4]      :For i :In lpr
[5]          :If 1=×/py ⋄ (i>r)←'c',i>r      A single strand item
[6]          :ElseIf (~1×ene)^(0=≡x)∨(' '≡p0px)^(1=ppx)^(1≠0ppx)←i>y
[7]              (i>r)←(i=⊂IO)↓' ',i>r      A no parens around strand item
[8]          :Else ⋄ (i>r)←1ϕ'(' ',i>r      A parens required
[9]              :EndIf
[10]         :EndFor
[11]         r←t,/r
[12]     :ElseIf 0=0p0pb ⋄ r←y          A numeric
[13]     :Else                          A text...
[14]         r←1ϕ(1+r=0pr)/r←''',y      A do quotes
[15]         :If lene≠r                  A modify runs of non-exec chars
[16]             r←(1≠~1↓1,l←rene)cr    A partition on non-exec chars
[17]             :For i :In lpr
[18]                 :If lene=0pi>r      A as ',⊂AV[n n n],'
[19]                     (i>r)←3ϕ',''',⊂AV[⊂IO+',';⊂IO-⊂AVti>r
[20]                 :EndIf
[21]             :EndFor
[22]             r←t,/r
[23]         :EndIf
[24]     :EndIf
[25]     :If b≡'' ⋄ r←''''''              A special-case ''
[26]     :ElseIf (,1)≡pb ⋄ r←',',r        A single item vector
[27]     :Else ⋄ r←((~1∈(0 1=ppb)∧1,1<0ppb)/(≠pb),'p'),r    A reshape
[28]     :EndIf

```

▽

[Note: A Dyalog 8.2 workspace (22KB) containing these functions may be downloaded from www.aplforum.com]

Index to Advertisers

Vector Back Numbers

6

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Email: apl385@compuserve.com.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (hardcopy with diskette as appropriate) to the Vector Working Group via:

Vector Administration, c/o Gill Smith
Brook House
Gilling East
YORK, YO62 4JJ
Tel: +44 (0) 1439-788385
Email: apl385@compuserve.com

Authors wishing to use Word for Windows should contact Vector Production for a copy of the APL2741 TrueType font, and a suitable Winword template. These may also be downloaded from the Vector web site at www.vector.org.uk

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO62 4JJ. Please also copy us with all electronically submitted material so that we have early warning of possible problems.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

2004/2005 Committee

Chairman	Adrian Smith 01439-788385 apl385@compuserve.com	Brook House Gilling East YORK YO62 4JJ
Secretary	Anthony Camacho 0117-9730036 acam@blueyonder.co.uk	11 Auburn Road Redland BRISTOL, BS6 6LS
Treasurer	Nicholas Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ
Journal Editor	Stefano Lanzavecchia stf@apl.it	c/o APL Italiana Via Monferrato, 1 20144 - Milano, Italy
Education	Dr Alan Mayer 01792-295779 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Webmaster	Bob Hoekstra 01483-771028 bob.hoekstra@hoekstrasystems.ltd.uk	Dominique, Salisbury Road WOKING, Surrey GU33 7UR
Activities	Ray Cannon 01252-874697 ray_cannon@compuserve.com	21 Woodbridge Rd, Blackwater Camberley, Surrey GU17 0BS
Projects	Stephen Taylor sjt@lambenttechnology.com	
Administration	Rowena Small 01223-570850 treas.apl@bcs.org.uk	12 Cambridge Road, Waterbeach, Cambridge CB5 9NJ

Journal Working Group

Editor:	Stefano Lanzavecchia	see above
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Anthony & Sylvia Camacho (0117-9730036), Ray Cannon (01252-874697), Stephen Taylor (077-1340 0852) Bob Hoekstra (01483-771028), Marc Griffiths	

Typeset by APL-385 with MS Word for Windows

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" – an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Causeway Graphical Systems Ltd
The Maltings, Castlegate,
MILTON, North Yorks YO17 7DP
Tel: 01653-696760
Fax: 01653-697719
Email: sales@causeway.co.uk
Web: www.causeway.co.uk

Dyalog Ltd
Grove House, Lutyens Close,
Chinriam Court, BASINGSTOKE,
Hants, RG24 8AG
Tel: 01256-338461
Fax: 01256-316559
Email: sales@dyalog.com
Web: www.dyalog.com

Insight Systems ApS
Nordre Strandvej 119G
DK-3150 Hellebæk
Denmark
Tel: +45 70 26 13 26
Fax: +45 70 26 13 25
Email: info@insight.dk
Web: www.insight.dk

MicroAPL Ltd
The Roller Mill, Mill Lane
Uckfield
E Sussex TN22 5AA
Tel: 01825-768050. Fax: 01825-749472
Email: MicroAPL@microapl.demon.co.uk
Web: www.microapl.co.uk/apl

Kx Systems
555 Bryant St., No. 375
Palo Alto CA 94301 USA
Tel: +1 866 kdb fast
Email (general enquiries): info@kx.com

In Europe: Simon Garland, SVP Technology
Email: simon@kx.com

Compass Ltd
Compass House
60 Priestley Road
GUILDFORD, Surrey GU2 5YU
Tel: 01483-514500

APL2000 Inc
One Research Court
Suite 140
Rockville MD 20850
USA
Tel: +1 (301) 208 7150
Email: sales@apl2000.com
Web: www.apl2000.com

HMW Computing
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel: 0870-1010-469
Email: HMW@4extra.com

Soliton Inc
44 Victoria Street, Suite 2100
Toronto
Ontario M5C 1Y2
Canada
Tel: +1 (416) 364 9355
Email: sales@soliton.com
Web: www.soliton.com